

**Imperial College
London**

Reinforcement Learning for Portfolio Management

MEng Dissertation

Angelos Filos
CID: 00943119

June 20, 2018

Supervisor: Professor Danilo Mandic

Second Marker: Professor Pier Luigi Dragotti

Advisors: Bruno Scalzo Dees, Gregory Sidier

Department of Electrical and Electronic Engineering, Imperial College London

Acknowledgement

I would like to thank Professor Danilo Mandic for agreeing to supervise this self-proposed project, despite the uncertainty about the viability of the topic. His support and guidance contributed to the delivery of a challenging project.

I would also like to take this opportunity and thank Bruno Scalzo Dees for his helpful comments, suggestions and enlightening discussions, which have been instrumental in the progress of the project.

Lastly, I would like to thank Gregory Sidier for spending time with me, out of his working hours. His experience, as a practitioner, in Quantitative Finance helped me demystify and engage with topics, essential to the project.

Abstract

The challenges of modelling the behaviour of financial markets, such as non-stationarity, poor predictive behaviour, and weak historical coupling, have attracted attention of the scientific community over the last 50 years, and has sparked a permanent strive to employ engineering methods to address and overcome these challenges. Traditionally, mathematical formulations of dynamical systems in the context of Signal Processing and Control Theory have been a lynchpin of today's Financial Engineering. More recently, advances in sequential decision making, mainly through the concept of Reinforcement Learning, have been instrumental in the development of multistage stochastic optimization, a key component in sequential portfolio optimization (asset allocation) strategies. In this thesis, we develop a comprehensive account of the expressive power, modelling efficiency, and performance advantages of so called trading agents (i.e., Deep Soft Recurrent Q-Network (DSRQN) and Mixture of Score Machines (MSM)), based on both traditional system identification (model-based approach) as well as on context-independent agents (model-free approach). The analysis provides a conclusive support for the ability of model-free reinforcement learning methods to act as universal trading agents, which are not only capable of reducing the computational and memory complexity (owing to their linear scaling with size of the universe), but also serve as generalizing strategies across assets and markets, regardless of the trading universe on which they have been trained. The relatively low volume of daily returns in financial market data is addressed via data augmentation (a generative approach) and a choice of pre-training strategies, both of which are validated against current state-of-the-art models. For rigour, a risk-sensitive framework which includes transaction costs is considered, and its performance advantages are demonstrated in a variety of scenarios, from synthetic time-series (sinusoidal, sawtooth and chirp waves),

simulated market series (surrogate data based), through to real market data (S&P 500 and EURO STOXX 50). The analysis and simulations confirm the superiority of universal model-free reinforcement learning agents over current portfolio management model in asset allocation strategies, with the achieved performance advantage of as much as 9.2% in annualized cumulative returns and 13.4% in annualized Sharpe Ratio.

Contents

1	Introduction	1
1.1	Problem Definition	1
1.2	Motivations	2
1.3	Report Structure	2
I	Background	4
2	Financial Signal Processing	5
2.1	Financial Terms & Concepts	5
2.1.1	Asset	6
2.1.2	Portfolio	6
2.1.3	Short Sales	7
2.2	Financial Time-Series	8
2.2.1	Prices	8
2.2.2	Returns	9
2.3	Evaluation Criteria	13
2.3.1	Statistical Moments	14
2.3.2	Financial Risk and Performance Metrics	20
2.4	Time-Series Analysis	23
2.4.1	Vector Autoregression (VAR)	24
2.4.2	Recurrent Neural Networks (RNN)	26
3	Portfolio Optimization	31
3.1	Markowitz Model	33
3.1.1	Mean-Variance Optimization	33
3.1.2	Quadratic Programming	34
3.2	Generic Objective Functions	36

3.2.1	Risk Aversion	36
3.2.2	Sharpe Ratio	37
3.3	Transaction Costs	37
3.3.1	Mathematical Formalization	38
3.3.2	Multi-Stage Decision Problem	39
4	Reinforcement Learning	40
4.1	Dynamical Systems	40
4.1.1	Agent & Environment	41
4.1.2	Action	41
4.1.3	Reward	41
4.1.4	State & Observation	42
4.2	Major Components of Reinforcement Learning	43
4.2.1	Return	44
4.2.2	Policy	44
4.2.3	Value Function	44
4.2.4	Model	45
4.3	Markov Decision Process	45
4.3.1	Markov Property	45
4.3.2	Definition	46
4.3.3	Optimality	46
4.3.4	Bellman Equation	47
4.3.5	Exploration-Exploitation Dilemma	48
4.4	Extensions	48
4.4.1	Infinite Markov Decision Process	48
4.4.2	Partially Observable Markov Decision Process	49
II	Innovation	50
5	Financial Market as Discrete-Time Stochastic Dynamical System	51
5.1	Assumptions	51
5.1.1	Sufficient Liquidity	52
5.1.2	Zero Slippage	52
5.1.3	Zero Market Impact	52
5.2	Action Space	52
5.3	State & Observation Space	53
5.3.1	Observation	53
5.3.2	State	54
5.4	Reward Signal	56

6	Trading Agents	58
6.1	Model-Based Reinforcement Learning	59
6.1.1	System Identification	60
6.1.2	Vector Autoregression (VAR)	61
6.1.3	Recurrent Neural Network (RNN)	64
6.1.4	Weaknesses	65
6.2	Model-Free Reinforcement Learning	67
6.2.1	Deep Soft Recurrent Q-Network (DSRQN)	68
6.2.2	Monte-Carlo Policy Gradient (REINFORCE)	74
6.2.3	Mixture of Score Machines (MSM)	79
7	Pre-Training	86
7.1	Baseline Models	87
7.1.1	Quadratic Programming with Transaction Costs	87
7.1.2	Data Generation	88
7.2	Model Evaluation	88
7.2.1	Convergence to Quadratic Programming	89
7.2.2	Performance Gain	89
III	Experiments	91
8	Synthetic Data	92
8.1	Deterministic Processes	92
8.1.1	Sinusoidal Waves	93
8.1.2	Sawtooth Waves	94
8.1.3	Chirp Waves	94
8.2	Simulated Data	95
8.2.1	Amplitude Adjusted Fourier Transform (AAFT)	96
8.2.2	Results	97
9	Market Data	98
9.1	Reward Generating Functions	98
9.1.1	Log Rewards	99
9.1.2	Differential Sharpe Ratio	99
9.2	Standard & Poor's 500	100
9.2.1	Market Value	100
9.2.2	Evaluation	101
9.3	EURO STOXX 50	101
9.3.1	Sequential Markowitz Model	102
9.3.2	Results	102

CONTENTS

10 Conclusion	104
10.1 Contributions	104
10.2 Future Work	105
Bibliography	106

Chapter 1

Introduction

Engineering methods and systems are routinely used in financial market applications, including signal processing, control theory and advanced statistical methods. The computerization of the markets (Schinckus, 2017) encourages automation and algorithmic solutions, which are now well-understood and addressed by the engineering communities. Moreover, the recent success of Machine Learning has attracted interest of the financial community, which permanently seeks for the successful techniques from other areas, such as computer vision and natural language processing to enhance modelling of financial markets. In this thesis, we explore how the asset allocation problem can be addressed by Reinforcement Learning, a branch of Machine Learning that optimally solves sequential decision making problems via direct interaction with the environment in an episodic manner.

In this introductory chapter, we define the objective of the thesis and highlight the research and application domains from which we draw inspiration.

1.1 Problem Definition

The aim of this report is to investigate the effectiveness of Reinforcement Learning agents on asset allocation¹. A finite universe of financial instruments, assets, such as stocks, is selected and the role of an agent is to construct an internal representation (model) of the market, allowing it to determine how to optimally allocate funds of a finite budget to those assets. The agent is trained on both synthetic and real market data. Then, its performance is compared with standard portfolio management algorithms on an

¹The terms *Asset Allocation* and *Portfolio Management* are used interchangeably throughout the report.

out-of-sample dataset; data that the agent has not been trained on (i.e., test set).

1.2 Motivations

From the IBM TD-Gammon (Tesauro, 1995) and the IBM Deep Blue (Campbell *et al.*, 2002) to the Google DeepMind Atari (Mnih *et al.*, 2015) and the Google DeepMind AlphaGo (Silver and Hassabis, 2016), reinforcement learning is well-known for its effectiveness in board and video games. Nonetheless, reinforcement learning applies to many more domains, including Robotics, Medicine and Finance, applications of which align with the mathematical formulation of portfolio management. Motivated by the success of some of these applications, an attempt is made to improve and adjust the underlying methods, such that they are applicable to the asset allocation problem settings. In particular special attention is given to:

- **Adaptive Signal Processing**, where Beamforming has been successfully addressed via reinforcement learning by Almeida *et al.* (2015);
- **Medicine**, where a data-driven medication dosing system (Nemati *et al.*, 2016) has been made possible thanks to model-free reinforcement agents;
- **Algorithmic Trading**, where the automated execution (Noonan, 2017) and market making (Spooner *et al.*, 2018) have been recently revolutionized by reinforcement agents.

Without claiming equivalence of portfolio management with any of the above applications, their relatively similar optimization problem formulation encourages the endeavour to develop reinforcement learning agents for asset allocation.

1.3 Report Structure

The report is organized in three Parts: the **Background** (Part I), the **Innovation** (Part II) and the **Experiments** (Part III). The readers are advised to follow the sequence of the parts as presented, however, if comfortable with Modern Portfolio Theory and Reinforcement Learning, they can focus on the last two parts, following the provided references to background material when necessary. A brief outline of the project structure and chapters is provided below:

Chapter 2: Financial Signal Processing The objective of this chapter is to introduce essential financial terms and concepts for understanding the methods developed later in the report.

Chapter 3: Portfolio Optimization Providing the basics of Financial Signal Processing, this chapter proceeds with the mathematical formulation of static Portfolio Management, motivating the use of Reinforcement Learning to address sequential Asset Allocation via multi-stage decision making.

Chapter 4: Reinforcement Learning This chapter serves as an important step toward demystifying Reinforcement Learning concepts, by highlighting their analogies to Optimal Control and Systems Theory. The concepts developed in this chapter are essential to the understanding of the trading algorithms and agents developed later in the report.

Chapter 5: Financial Market as Discrete-Time Stochastic Dynamical System This chapter parallels Chapters 3 and 4, introducing a unified, versatile framework for training agents and investment strategies.

Chapter 6: Trading Agents This objectives of this chapter are to: (1) introduce traditional model-based (i.e., system identification) reinforcement learning trading agents; (2) develop model-free reinforcement learning trading agents; (3) suggest a flexible universal trading agent architecture that enables pragmatic applications of Reinforcement Learning for Portfolio Management; (4) assess performance of developed trading agents on a small scale experiment (i.e., 12-asset S&P 500 market)

Chapter 7: Pre-Training In this chapter, a pre-training strategy is suggested, which addresses the local optimality of the Policy Gradient agents, when only a limited number of financial market data samples is available.

Chapter 8: Synthetic Data In this chapter, the effectiveness of the trading agents of Chapter 6 is assessed on synthetic data - from deterministic time-series (sinusoidal, sawtooth and chirp waves) to simulated market series (surrogate data based). The superiority of model-based or model-free agents is highlighted in each scenario.

Chapter 9: Market Data This chapter parallels Chapter 9, evaluating the performance of the trading agents of Chapter 6 on real market data, from two distinct universes: (1) the underlying U.S. stocks of S&P 500 and (2) the underlying European stocks of EURO STOXX 50.

Part I

Background

Chapter 2

Financial Signal Processing

Financial applications usually involve the manipulation and analysis of sequences of observations, indexed by time order, also known as time-series. Signal Processing, on the other hand, provides a rich toolbox for systematic time-series analysis, modelling and forecasting (Mandic and Chambers, 2001). Consequently, signal processing methods can be employed to mathematically formulate and address fundamental economics and business problems. In addition, Control Theory studies discrete dynamical systems, which form the basis of Reinforcement Learning, the set of algorithms used in this report to solve the asset allocation problem. The links between signal processing algorithms, systems and control theory motivate their integration with finance, to which we refer as **Financial Signal Processing** or **Financial Engineering**.

In this chapter, the overlap between signal processing and control theory with finance is explored, attempting to bridge their gaps and highlight their similarities. Firstly, in Section 2.1, essential financial terms and concepts are introduced, while In Section 2.2, the time-series in the context of finance are formalized. In Section 2.3 the evaluation criteria used throughout the report to assess the performance of the different algorithms and strategies are explained, while in Section 2.4 signal processing methods for modelling sequential data are studied.

2.1 Financial Terms & Concepts

In order to better communicate ideas and gain insight into the economic problems, basic terms are defined and explained in this section. However, useful definitions are also provided by Johnston and Djurić (2011).

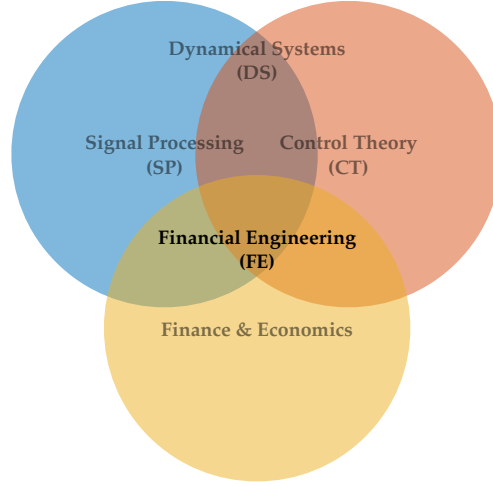


Figure 2.1: Financial Engineering relative to Signal Processing and Control Theory.

2.1.1 Asset

An **asset** is an item of economic value. Examples of assets are cash (in hand or in a bank), stocks, loans and advances, accrued incomes etc. Our main focus on this report is on cash and stocks, but general principles apply to all kinds of assets.

Assumption 2.1 *The assets under consideration are liquid, hence they can be converted into cash quickly, with little or no loss in value. Moreover, the selected assets have available historical data in order to enable analysis.*

2.1.2 Portfolio

A **portfolio** is a collection of multiple financial assets, and is characterized by its:

- **Constituents:** M assets of which it consists;
- **Portfolio vector, w_t :** its i -th component represents the ratio of the total budget invested to the i -th asset, such that:

$$w_t = \begin{bmatrix} w_{1,t}, & w_{2,t}, & \dots, & w_{M,t} \end{bmatrix}^T \in \mathbb{R}^M \quad \text{and} \quad \sum_{i=1}^M w_{i,t} = 1 \quad (2.1)$$

For fixed constituents and portfolio vector w_t , a portfolio can be treated as a single master asset. Therefore, the analysis of single simple assets can be

applied to portfolios upon determination of the constituents and the corresponding portfolio vector.

Portfolios are more powerful, general representation of financial assets since the single asset case can be represented by a portfolio; the j -th asset is equivalent to the portfolio with vector $e^{(j)}$, where the j -th term is equal to unity and the rest are zero. Portfolios are also preferred over single assets in order to minimize risk, as illustrated in Figure 2.2.

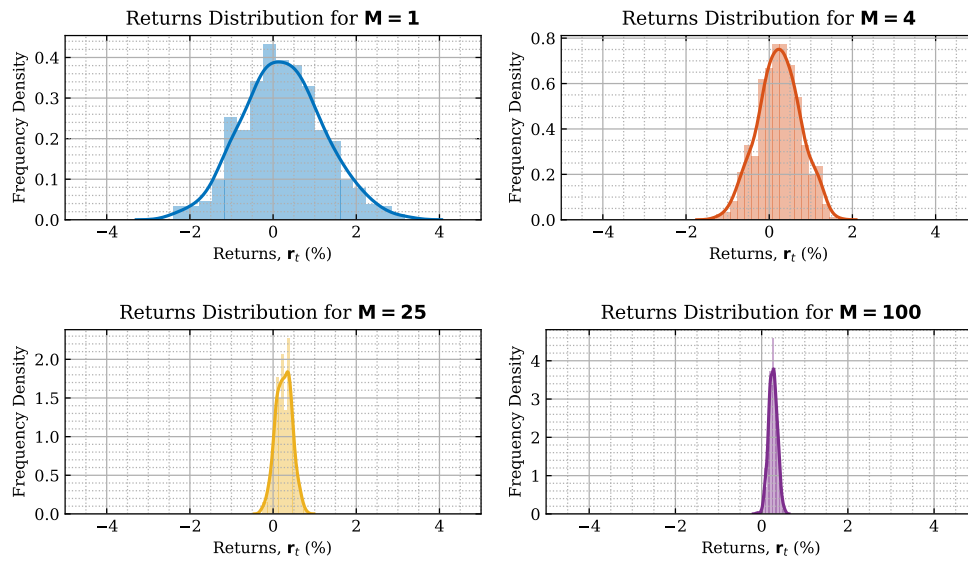


Figure 2.2: Risk for a single asset and a number of uncorrelated portfolios. Risk is represented by the standard deviation or the width of the distribution curves, illustrating that a large portfolio ($M = 100$) can be significantly less risky than a single asset ($M = 1$).

2.1.3 Short Sales

Sometimes it is possible to sell an asset that we do not own. This process is called **short selling** or **shorting** (Luenberger, 1997). The exact shorting mechanism varies between markets, but it can be generally summarized as:

1. **Borrowing** an asset i from someone who owns it at time t ;
2. **Selling** it immediately to someone else at price $p_{i,t}$;
3. **Buying back** the asset at time $(t + k)$, where $k > 0$, at price $p_{i,t+k}$;
4. **Returning** the asset to the lender

Therefore, if one unit of the asset is shorted, the **overall** absolute return is $p_{i,t} - p_{i,t+k}$ and as a result short selling is profitable only if the asset price declines between time t and $t + k$ or $p_{i,t+k} < p_{i,t}$. Nonetheless, note that the potential loss of short selling is unbounded, since asset prices are not bounded from above ($0 \leq p_{i,t+k} < \infty$).

Remark 2.2 *If short selling is allowed, then the portfolio vector satisfies (2.1), but w_i can be negative, if the i -th asset is shorted. As a consequence, w_j can be greater than 1, such that $\sum_{i=1}^M w_i = 1$.*

For instance, in case of a two-assets portfolio, the portfolio vector $w_t = [-0.5, 1.5]^T$ is valid and can be interpreted as: 50% of the budget is short sold on the first asset ($w_{1,t} = -0.5$) and 150% of the budget is invested on the second asset ($w_{2,t} = 1.5$). Note that the money received from shorting asset 1 are used in the investment on asset 2, enabling $w_{2,t} > 1$.

Usually the terms **long** and **short** position to an asset are used to refer to investments where we buy or short sell the asset, respectively.

2.2 Financial Time-Series

The dynamic nature of the economy, as a result of the non-static supply and demand balance, causes prices to evolve over time. This encourages to treat market dynamics as time-series and employ technical methods and tools for analysis and modelling.

In this section, asset prices are introduced, whose definition immediately reflect our intuition, as well as other time-series, derived to ease analysis and evaluation.

2.2.1 Prices

Let $p_t \in \mathbb{R}$ be the **price** of an asset at discrete time index t (Feng and Palomar, 2016), then the sequence $p_1, p_2, \dots, p_T$ is a univariate time-series. The equivalent notations $p_{i,t}$ and $p_{\text{asset},i,t}$ are also used to distinguish between the prices of the different assets. Hence, the T -samples price time-series of an

asset i , is the column vector $\vec{p}_{i,1:T}$, such that:

$$\vec{p}_{i,1:T} = \begin{bmatrix} p_{i,1} \\ p_{i,2} \\ \vdots \\ p_{i,T} \end{bmatrix} \in \mathbb{R}_+^T \quad (2.2)$$

where the arrow highlights the fact that it is a time-series. For convenience of portfolio analysis, we define the **price vector** p_t , such that:

$$p_t = \begin{bmatrix} p_{1,t} & p_{2,t} & \dots & p_{M,t} \end{bmatrix} \in \mathbb{R}_+^M \quad (2.3)$$

where the i -th element is the asset price of the i -th asset in the portfolio at time t . Extending the single-asset time-series notation to the multivariate case, we form the asset **price matrix** $\vec{P}_{1:T}$ by stacking column-wise the T -samples price time-series of the M assets of the portfolio, then:

$$\vec{P}_{1:T} = \begin{bmatrix} \vec{p}_{1,1:T} & \vec{p}_{2,1:T} & \dots & \vec{p}_{M,1:T} \end{bmatrix} = \begin{bmatrix} p_{1,1} & p_{2,1} & \dots & p_{M,1} \\ p_{1,2} & p_{2,2} & \dots & p_{M,2} \\ \vdots & \vdots & \ddots & \vdots \\ p_{1,T} & p_{2,T} & \dots & p_{M,T} \end{bmatrix} \in \mathbb{R}_+^{T \times M} \quad (2.4)$$

This formulation enables cross-asset analysis and consideration of the inter-dependencies between the different assets. We usually relax notation by omitting subscripts when they can be easily inferred from context.

Figure 2.3 illustrates examples of asset prices time-series and the corresponding distribution plots. At a first glance, note the highly non-stationary nature of asset prices and hence the difficulty to interpret distribution plots. Moreover, we highlight the unequal scaling between prices, where for example, GE (General Electric) average price at 23.14\$ and BA (Boeing Company) average price at 132.23\$ are of different order and difficult to compare.

2.2.2 Returns

Absolute asset prices are not directly useful for an investor. On the other hand, prices changes over time are of great importance, since they reflect the investment profit and loss, or more compactly, its **return**.

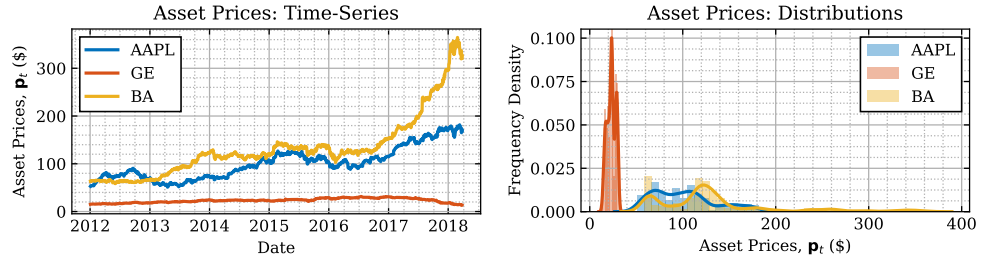


Figure 2.3: Asset prices time-series (left) and distributions (right) for AAPL (Apple), GE (General Electric) and BA (Boeing Company).

Gross Return

The **gross return** R_t of an asset represents the scaling factor of an investment in the asset at time $(t - 1)$ (Feng and Palomar, 2016). For example, a B dollars investment in an asset at time $(t - 1)$ will worth BR_t dollars at time t . It is given by the ratio of its prices at times t and $(t - 1)$, such that:

$$R_t \triangleq \frac{p_t}{p_{t-1}} \in \mathbb{R} \quad (2.5)$$

Figure 2.4 illustrates the benefit of using gross returns over asset prices.

Remark 2.3 *The asset gross returns are concentrated around unity and their behaviour does not vary over time for all stocks, making them attractive candidates for stationary autoregressive (AR) processes (Mandic, 2018a).*

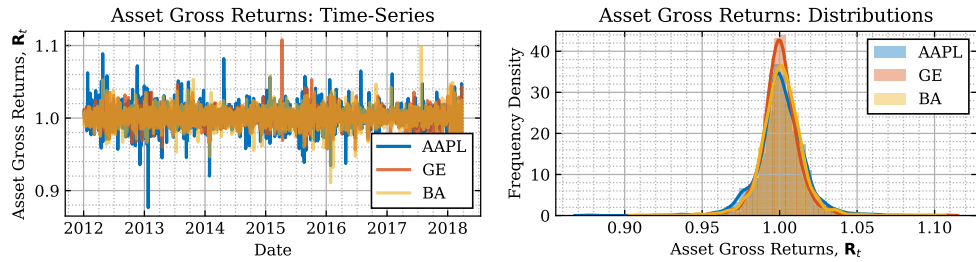


Figure 2.4: Asset gross returns time-series (left) and distributions (right).

Simple Return

A more commonly used term is the **simple return**, r_t , which represents the percentage change in asset price from time $(t - 1)$ to time t , such that:

$$r_t \triangleq \frac{p_t - p_{t-1}}{p_{t-1}} = \frac{p_t}{p_{t-1}} - 1 \stackrel{(2.5)}{=} R_t - 1 \in \mathbb{R} \quad (2.6)$$

The gross and simple returns are straightforwardly connected, but the latter is more interpretable, and thus more frequently used.

Figure 2.5 depicts the example asset simple returns time-series and their corresponding distributions. Unsurprisingly, simple returns possess the representation benefits of gross returns, such as stationarity and normalization. Therefore, we can use simple returns as a comparable metric for all assets, thus enabling the evaluation of analytic relationships among them, despite originating from asset prices of different scale.

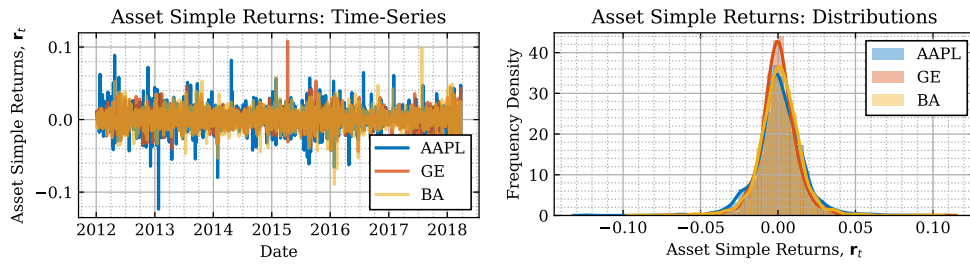


Figure 2.5: Single Assets Simple Returns

The T -samples simple returns time-series of the i -th asset is given by the column vector $\vec{r}_{i,1:T}$, such that:

$$\vec{r}_{i,1:T} = \begin{bmatrix} r_{i,1} \\ r_{i,2} \\ \vdots \\ r_{i,T} \end{bmatrix} \in \mathbb{R}^T \quad (2.7)$$

while the **simple returns vector** $\mathbf{r}_t$:

$$\mathbf{r}_t = \begin{bmatrix} r_{1,t} \\ r_{2,t} \\ \vdots \\ r_{M,t} \end{bmatrix} \in \mathbb{R}^M \quad (2.8)$$

where $r_{1,t}$ the simple return of the i -th asset at time index t .

Remark 2.4 Exploiting the representation advantage of the portfolio over single assets, we define the **portfolio simple return** as the linear combination of the simple returns of each constituents, weighted by the portfolio vector.

Hence, at time index t , we obtain:

$$r_t \triangleq \sum_{i=1}^M w_{i,t} r_{i,t} = \mathbf{w}_t^T \mathbf{r}_t \in \mathbb{R} \quad (2.9)$$

Combining the price matrix in (2.4) and the definition of simple return (2.6), we construct the **simple return matrix** $\vec{\mathbf{R}}_{1:T}$ by stacking column-wise the T -samples simple returns time-series of the M assets of the portfolio, to give:

$$\vec{\mathbf{R}}_{1:T} = \begin{bmatrix} \vec{r}_{1,1:T} & \vec{r}_{2,1:T} & \cdots & \vec{r}_{M,1:T} \end{bmatrix} = \begin{bmatrix} r_{1,1} & r_{2,1} & \cdots & r_{M,1} \\ r_{1,2} & r_{2,2} & \cdots & r_{M,2} \\ \vdots & \vdots & \ddots & \vdots \\ r_{1,T} & r_{2,T} & \cdots & r_{M,T} \end{bmatrix} \in \mathbb{R}^{T \times M} \quad (2.10)$$

Collecting the portfolio (column) vectors for the time interval $t \in [1, T]$ into a **portfolio weights matrix** $\vec{\mathbf{W}}_{1:T}$, we obtain the portfolio returns time-series by multiplication of $\vec{\mathbf{R}}_{1:T}$ with $\vec{\mathbf{W}}_{1:T}$ and extraction of the T diagonal elements of the product, such that:

$$\mathbf{r}_{1:T} = \text{diag}(\vec{\mathbf{R}}_{1:T} \vec{\mathbf{W}}_{1:T}) \in \mathbb{R}^T \quad (2.11)$$

Log Return

Despite the interpretability of the simple return as the percentage change in asset price over one period, it is asymmetric and therefore practitioners tend to use log returns instead (Kennedy, 2016), in order to preserve interpretation and to yield a symmetric measure. Using the example in Table 2.1, a 15% increase in price followed by a 15% decline does not result in the initial price of the asset. On the contrary, a 15% log-increase in price followed by a 15% log-decline returns to the initial asset price, reflecting the symmetric behaviour of log returns.

Let the **log return** ρ_t at time t be:

$$\rho_t \triangleq \ln\left(\frac{p_t}{p_{t-1}}\right) \stackrel{(2.5)}{=} \ln(R_t) \in \mathbb{R} \quad (2.12)$$

Note the very close connection of gross return to log return. Moreover, since gross return is centered around unity, the logarithmic operator makes log returns concentrated around zero, clearly observed in Figure 2.6.

time t	simple return	price (\$)	log return
0	-	100	-
1	+0.15	110	+0.13
2	-0.15	99	-0.16
3	+0.01	100	+0.01
4	-0.14	86	-0.15
5	+0.16	100	+0.15

Table 2.1: Simple Return Asymmetry & Log Return Symmetry

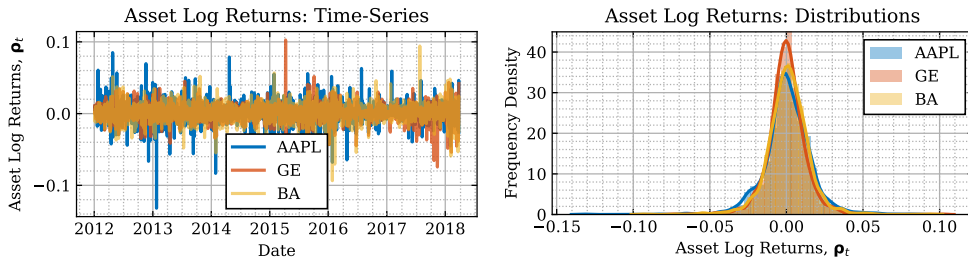


Figure 2.6: Single Assets Log Returns

Comparing the definitions of simple and log returns in (2.6) and (2.12), respectively, we obtain the relationship:

$$\rho_t = \ln(1 + r_t) \quad (2.13)$$

hence we can define all time-series and convenient portfolio representations of log returns by substituting simple-returns in (2.13). For example, the **portfolio log return** is given by substitution of (2.13) into (2.9), such that:

$$\rho_t \triangleq \ln(1 + \mathbf{w}_t^T \mathbf{r}_t) \in \mathbb{R} \quad (2.14)$$

2.3 Evaluation Criteria

The end goal is the construction of portfolios, linear combinations of individual assets, whose properties (e.g., returns, risk) are optimal under provided conditions and constraints. As a consequence, a set of evaluation criteria and metrics is necessary in order to evaluate the performance of the generated portfolios. Due to the uncertainty of the future dynamics of the financial markets, we study the statistical properties of the assets returns, as well as other risk metrics, motivated by signal processing.

2.3.1 Statistical Moments

Future prices and hence returns are inherently unknown and uncertain (Kennedy, 2016). To mathematically capture and manipulate this stochasticity, we treat future market dynamics (i.e., prices, cross-asset dependencies) as random variables and study their properties. Qualitative visual analysis of probability density functions is a labour-intensive process and thus impractical, especially when high-dimensional distributions (i.e., 4D and higher) are under consideration. On the other hand, quantitative measures, such as moments, provide a systematic way to analyze (joint) distributions (Meucci, 2009).

Mean, Median & Mode

Suppose that we need to summarize all the information regarding a random variable X in only one number, the one value that best represents the whole range of possible outcomes. We are looking for a location parameter that provides a fair indication of where on the real axis the random variable X will end up taking its value.

An immediate choice for the location parameter is the center of mass of the distribution, i.e., the weighted average of each possible outcome, where the weight of each outcome is provided by its respective probability. This corresponds to computing the **expected value** or **mean** of the random variable:

$$\mathbb{E}[X] = \mu_X \triangleq \int_{-\infty}^{+\infty} x f_X(x) dx \in \mathbb{R} \quad (2.15)$$

Note that the mean is also the first order statistical moment of the distribution f_X . When a finite number of observations T is available, and there is no closed form expression for the probability density function f_X , the **sample mean** or **empirical mean** is used as an unbiased estimate of the expected value, according to:

$$\mathbb{E}[X] \approx \frac{1}{T} \sum_{t=1}^T x_t \quad (2.16)$$

Investor Advice 2.1 (Greedy Criterion) *For the same level of risk, choose the portfolio that maximizes the expected returns (Wilmott, 2007).*

Figure 2.7 illustrates to cases where the **Greedy Criterion 2.1** is applied. In case of assets with equal risk levels (i.e., left sub-figure) we prefer the one that maximizes expected returns, thus the red ($\mu_{\text{blue}} = 1 < \mu_{\text{red}} = 4$). On the

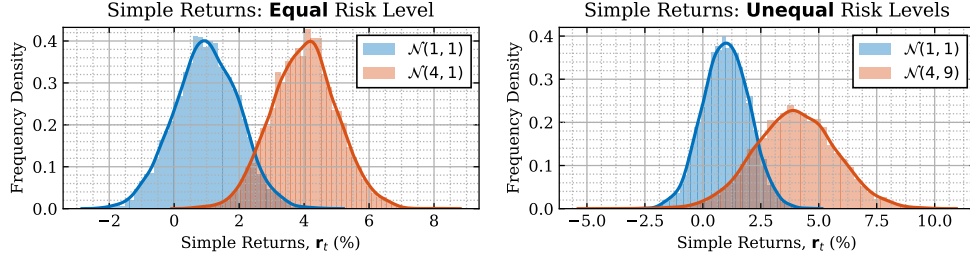


Figure 2.7: Greedy criterion for equally risky assets (left) and unequally risky assets (right).

other hand, when the assets have unequal risk levels (i.e., right sub-figure) the criterion does not apply and we cannot draw any conclusions without employing other metrics as well.

The definition of the mean value is extended to the multivariate case as the juxtaposition of the mean value (2.15) of the marginal distribution of each entry:

$$\mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}_{\mathbf{X}} \triangleq \begin{bmatrix} \mathbb{E}[X_1] \\ \mathbb{E}[X_2] \\ \vdots \\ \mathbb{E}[X_M] \end{bmatrix} \in \mathbb{R}^M \quad (2.17)$$

A portfolio with vector $\mathbf{w}_t$ and single asset mean simple returns $\boldsymbol{\mu}_r$ has expected simple returns:

$$\mu_r = \mathbf{w}_t^T \boldsymbol{\mu}_r \quad (2.18)$$

An alternative choice for the location parameter is the **median**, which is the quantile relative to the specific cumulative probability $p = 1/2$:

$$\text{Med}[X] \triangleq Q_X\left(\frac{1}{2}\right) \in \mathbb{R} \quad (2.19)$$

The juxtaposition of the median, or any other quantile, of each entry of a random variable does not satisfy the affine equivariance property¹ (Meucci, 2009) and therefore it does not define a suitable location parameter.

¹ $\text{Med}[\mathbf{a} + \mathbf{B}\mathbf{X}] \neq \mathbf{a} + \mathbf{B}\text{Med}[\mathbf{X}]$.

A third parameter of location is the mode, which refers to the shape of the probability density function f_X . Indeed, the mode is defined as the point that corresponds to the highest peak of the density function:

$$\text{Mod}[X] \triangleq \underset{x \in \mathbb{R}}{\text{argmax}} f_X(x) \in \mathbb{R} \quad (2.20)$$

Intuitively, the mode is the most frequently occurring data point in the distribution. It is trivially extended to multivariate distributions, namely as the highest peak of the joint probability density function:

$$\text{Mod}[\mathbf{X}] \triangleq \underset{\mathbf{x} \in \mathbb{R}^M}{\text{argmax}} f_{\mathbf{X}}(\mathbf{x}) \in \mathbb{R}^M \quad (2.21)$$

Note that the relative position of the location parameters provide qualitative information about the symmetry, the tails and the concentration of the distribution. Higher-order moments quantify these properties.

Figure 2.8 illustrates the distribution of the prices and the corresponding simple returns of the asset BA (Boeing Company), along with their location parameters. In case of the simple returns, we highlight that the mean, the median and the mode are very close to each other, reflecting the symmetry and the concentration of the distribution, properties that motivated the selection of returns over raw asset prices.

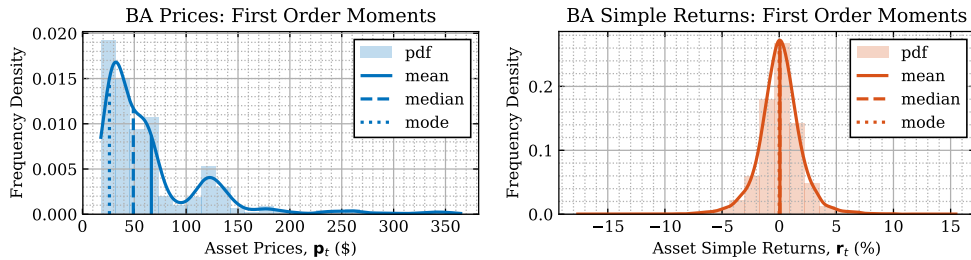


Figure 2.8: First order moments for BA (Boeing Company) prices (left) and simple returns (right).

Volatility & Covariance

The dilemma we faced in selecting between assets in Figure 2.7 motivates the introduction of a metric that quantifies risk level. On other words, we are looking for a dispersion parameter that yields an indication of the extent

to which the location parameter (i.e., mean, median) might be wrong in guessing the outcome of the random variable X .

The **variance** is the benchmark dispersion parameter, measuring how far the random variable X is spread out of its mean, given by:

$$\text{Var}[X] = \sigma_X^2 \triangleq \mathbb{E}[(X - \mathbb{E}[X])^2] \in \mathbb{R} \quad (2.22)$$

The square root of the variance, σ_X , namely the **standard deviation** or **volatility** in finance, is a more physically interpretable parameter, since it has the same units as the random variable under consideration (i.e., prices, simple returns).

Note that the variance is also the second order statistical central moment of the distribution f_X . When a finite number of observations T is available, and there is no closed form expression for the probability density function, f_X , the Bessel's correction formula (Tsay, 2005) is used as an unbiased estimate of the variance, according to:

$$\text{Var}[X] \approx \frac{1}{T-1} \sum_{t=1}^T (x_t - \mu_X)^2 \quad (2.23)$$

Investor Advice 2.2 (Risk-Aversion Criterion) *For the same expected returns, choose the portfolio that minimizes the volatility (Wilmott, 2007).*

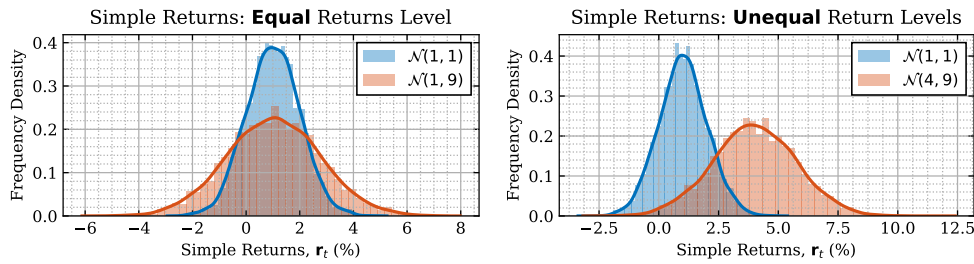


Figure 2.9: Risk-aversion criterion for equal returns (left) and unequal returns (right).

According to the **Risk-Aversion Criterion**, in Figure 2.9, for the same returns level (i.e., left sub-figure) we choose the less risky asset, the blue, since the red is more spread out ($\sigma_{\text{blue}}^2 = 1 < \sigma_{\text{red}}^2 = 9$). However, in case of unequal return levels (i.e., right sub-figure) the criterion is inconclusive.

The definition of variance is extended to the multivariate case by introducing **covariance**, which measures the joint variability of two variables, given by:

$$\text{Cov}[\mathbf{X}] = \mathbf{\Sigma}_X \triangleq \mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X}])(\mathbf{X} - \mathbb{E}[\mathbf{X}])^T] \in \mathbb{R}^{M \times M} \quad (2.24)$$

or component-wise:

$$\text{Cov}[X_m, X_n] = [\text{Cov}[\mathbf{X}]]_{mn} = \Sigma_{mn} \triangleq \mathbb{E}[(X_m - \mathbb{E}[X_m])(X_n - \mathbb{E}[X_n])] \in \mathbb{R} \quad (2.25)$$

By direct comparison of (2.22) and (2.25), we note that:

$$\text{Var}[X_m] = \text{Cov}[X_m, X_m] = [\text{Cov}[\mathbf{X}]]_{mm} = \Sigma_{mm} \quad (2.26)$$

hence the m -th diagonal element of the covariance matrix Σ_{mm} is the variance of the m -th component of the multivariate random variable $\mathbf{X}$, while the non-diagonal terms Σ_{mn} represent the joint variability of the m -th with the n -th component of $\mathbf{X}$. Note that, by definition (2.24), the covariance is a symmetric and real matrix, thus it is semi-positive definite (Mandic, 2018b).

Empirically, we estimate the covariance matrix entries using again the Bessel's correction formula (Tsay, 2005), in order to obtain an unbiased estimate:

$$\text{Cov}[X_m, X_n] \approx \frac{1}{T-1} \sum_{t=1}^T (x_{m,t} - \mu_{X_m})(x_{n,t} - \mu_{X_n}) \quad (2.27)$$

A portfolio with vector $\mathbf{w}_t$ and covariance matrix of assets simple returns $\mathbf{S}$ has variance:

$$\sigma_r^2 = \mathbf{w}_t^T \mathbf{\Sigma} \mathbf{w}_t \quad (2.28)$$

The **correlation coefficient** is also frequently used to quantify the linear dependency between random variables. It takes values in the range $[-1, 1]$ and hence it is a normalized way to compare dependencies, while covariances are highly influenced by the scale of the random variables' variance. The correlation coefficient is given by:

$$\text{corr}[X_m, X_n] = [\text{corr}[\mathbf{X}]]_{mn} = \rho_{mn} \triangleq \frac{\text{Cov}[X_m, X_n]}{\sigma_{X_m} \sigma_{X_n}} \in [-1, 1] \subset \mathbb{R} \quad (2.29)$$

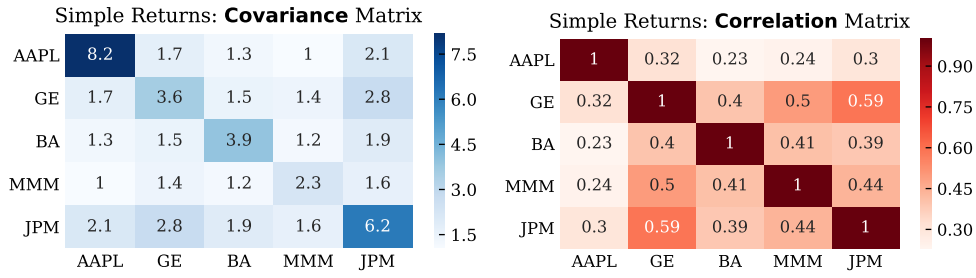


Figure 2.10: Covariance and correlation matrices for assets simple returns.

Skewness

The standard measure of symmetry of a distribution is the **skewness**, which is the third central moment normalized by the standard deviation, in such a way to make it scale-independent:

$$\text{skew}[X] \triangleq \frac{\mathbb{E}[(X - \mathbb{E}[X])^3]}{\sigma_X^3} \quad (2.30)$$

In particular, a distribution whose probability density function is symmetric around its expected value has null skewness. If the skewness is positive (negative), occurrences larger than the expected value are less (more) likely than occurrences smaller than the expected value.

Investor Advice 2.3 (Negatively Skewed Criterion) *Choose negatively skewed returns, rather than positively skewed. (Wilmott, 2007).*

Kurtosis

The fourth moment provides a measure of the relative weight of the tails with respect to the central body of a distribution. The standard quantity to evaluate this balance is the **kurtosis**, defined as the normalized fourth central moment:

$$\text{kurt}[X] \triangleq \frac{\mathbb{E}[(X - \mathbb{E}[X])^4]}{\sigma_X^4} \quad (2.31)$$

The kurtosis gives an indication of how likely it is to observe a measurement far in the tails of the distribution: a large kurtosis implies that the distribution displays "fat tails".

2.3.2 Financial Risk and Performance Metrics

Despite the insight into the statistical properties we gain by studying moments of returns distribution, we can combine them in such ways to fully capture the behaviour of our strategies and better assess them. Inspired by standard metrics used in signal processes (e.g. signal-to-noise ratio) and sequential decision making we introduce the following performance evaluators: cumulative returns, sharpe ratio, drawdown and value at risk.

Cumulative Returns

In subsection 2.2.2 we defined returns relative to the change in asset prices in one time period. Nonetheless, we usually get involved into a multi-period investment, hence we are extending the definition of vanilla returns to the **cumulative returns**, which represent the change in asset prices over larger time horizons.

Based on (2.5), the **cumulative gross return** $R_{t \rightarrow T}$ between time indexes t and T is given by:

$$R_{t \rightarrow T} \triangleq \frac{p_T}{p_t} = \left(\frac{p_T}{p_{T-1}} \right) \left(\frac{p_{T-1}}{p_{T-2}} \right) \cdots \left(\frac{p_{t+1}}{p_t} \right) \stackrel{(2.5)}{=} R_T R_{T-1} \cdots R_{t+1} = \prod_{i=t+1}^T R_i \in \mathbb{R} \quad (2.32)$$

The cumulative gross return is usually also termed **Profit & Loss (PnL)**, since it represents the wealth level of the investment. If $R_{t \rightarrow T} > 1$ (< 1) the investment was profitable (lossy).

Investor Advice 2.4 (Profitability Criterion) *Aim to maximize profitability of investment.*

Moreover, the **cumulative simple return** $r_{t \rightarrow T}$ is given by:

$$r_{t \rightarrow T} \triangleq \frac{p_T}{p_t} - 1 \stackrel{(2.32)}{=} \left[\prod_{i=t+1}^T R_i - 1 \right] \stackrel{(2.6)}{=} \left[\prod_{i=t+1}^T (1 + r_i) - 1 \right] \in \mathbb{R} \quad (2.33)$$

while the **cumulative log return** $\rho_{t \rightarrow T}$ is:

$$\rho_{t \rightarrow T} \triangleq \ln\left(\frac{p_T}{p_t}\right) \stackrel{(2.32)}{=} \ln\left(\prod_{i=t+1}^T R_i\right) = \sum_{i=t+1}^T \ln(R_i) \stackrel{(2.12)}{=} \sum_{i=t+1}^T \rho_i \in \mathbb{R} \quad (2.34)$$

Figure 2.11 demonstrates the interpretation power of cumulative returns over simple returns. Simple visual inspection of simple returns is inadequate for comparing the performance of the different assets. On the other hand, cumulative simple returns exhibit that BA's (Boeing Company) price increased by $\approx 400\%$, while GE's (General Electric) price declines by $\approx 11\%$, in the time period 2012 to 2018.

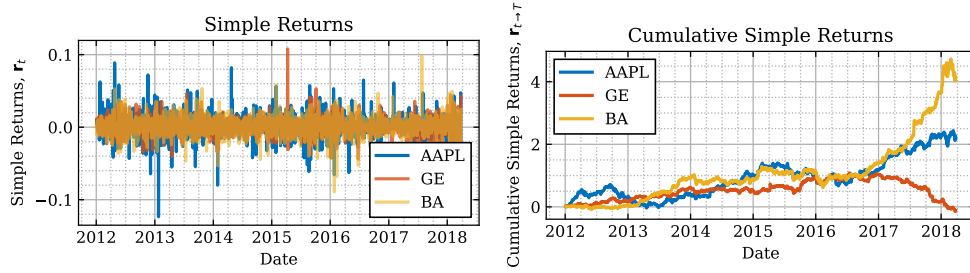


Figure 2.11: Assets cumulative simple returns.

Sharpe Ratio

Remark 2.5 *The criteria 2.1 and 2.2 can sufficiently distinguish and prioritize investments which either have the same risk level or returns level, respectively. Nonetheless, they fail in all other cases, when risk or return levels are unequal.*

The failure of greedy criterion and risk-aversion criterion is demonstrated in both examples in Figures 2.7 and 2.9, where it can be observed that the more risky asset, the red one, has a higher expected returns (i.e., the red distribution is wider, hence has larger variance, but it is centered around a larger value, compared to the blue distribution). Consequently, none of the criteria applies and the comparison is inconclusive.

In order to address this issue and motivated by **Signal-to-Noise Ratio (SNR)** (Zhang and Wang, 2017; Feng and Palomar, 2016), we define **Sharpe Ratio (SR)** as the ratio of expected returns (i.e., signal power) to their standard deviation (i.e., noise power²), adjusted by a scaling factor:

$$\mathbf{SR}_{1:T} \triangleq \sqrt{T} \frac{\mathbb{E}[\mathbf{r}_{1:T}]}{\sqrt{\text{Var}[\mathbf{r}_{1:T}]}} \in \mathbb{R} \quad (2.35)$$

²The variance of the noise is equal to the noise power. Standard deviation is used in the definition of SR to provide a unit-less metric.

where T is the number of samples considered in the calculation of the empirical mean and standard deviation.

Investor Advice 2.5 (Sharpe Ratio Criterion) *Aim to maximize Sharpe Ratio of investment.*

Considering now the example in Figures 2.7, 2.9, we can quantitatively compare the two returns streams and select the one that maximizes the Sharpe Ratio:

$$\mathbf{SR}_{blue} = \sqrt{T} \frac{\mu_{blue}}{\sigma_{blue}} = \sqrt{T} \frac{1}{1} = \sqrt{T} \quad (2.36)$$

$$\mathbf{SR}_{red} = \sqrt{T} \frac{\mu_{red}}{\sigma_{red}} = \sqrt{T} \frac{4}{3} \quad (2.37)$$

$$\mathbf{SR}_{blue} < \mathbf{SR}_{red} \Rightarrow \text{choose red} \quad (2.38)$$

Drawdown

The **drawdown** (DD) is a measure of the decline from a historical peak in cumulative returns (Luenberger, 1997). A drawdown is usually quoted as the percentage between the peak and the subsequent trough and is defined as:

$$\mathbf{DD}(t) = -\max\{0, [\max_{\tau \in (0,t)} r_{0 \rightarrow \tau}] - r_{0 \rightarrow t}\} \quad (2.39)$$

The **maximum drawdown** (MDD) up to time t is the maximum of the drawdown over the history of the cumulative returns, such that:

$$\mathbf{MDD}(t) = -\max_{x \in (0,t)} \{ [\max_{\tau \in (0,T)} r_{0 \rightarrow \tau}] - r_{0 \rightarrow T} \} \quad (2.40)$$

The drawdown and maximum drawdown plots are provided in Figure 2.12 along with the cumulative returns of assets GE and BA. Interestingly, the decline of GE's cumulative returns starting in early 2017 is perfectly reflected by the (maximum) drawdown curve.

Value at Risk

The **value at risk** (VaR) is another commonly used metric to assess the performance of a returns time-series (i.e., stream). Given daily simple returns

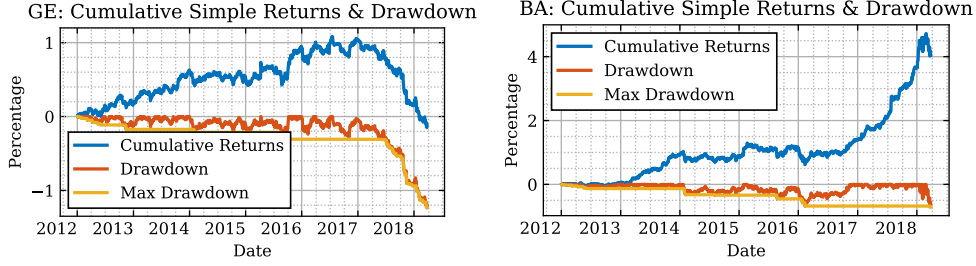


Figure 2.12: (Maximum) drawdown and cumulativer returns for GE and BA.

r_t and cut-off $c \in (0, 1)$, the value at risk is defined as the c quantile of their distribution, representing the worst $100c\%$ case scenario:

$$\mathbf{VaR}(c) \triangleq Q_r(c) \in \mathbb{R} \quad (2.41)$$

Figure 2.13 depicts GE's value at risk at -1.89% for cut-off parameter $c = 0.05$. We interpret this as "5% of the trading days, General Electric's stock declines more than 1.89% ".

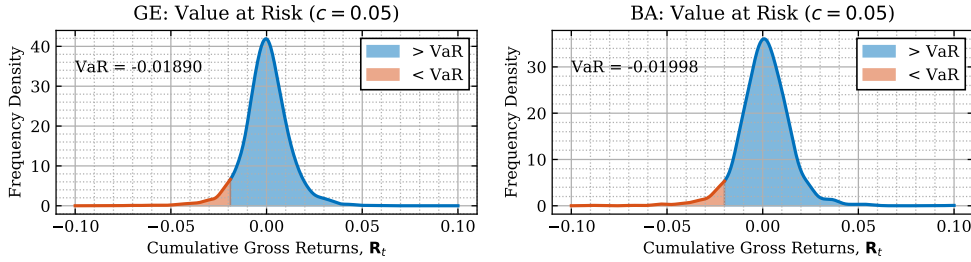


Figure 2.13: Illustration of the 5% value at risk (VaR) of GE and BA stocks.

2.4 Time-Series Analysis

Time-series analysis is of major importance in a vast range of research topics, and many engineering applications. This relates to analyzing time-series data for estimating meaningful statistics and identifying patterns of sequential data. Financial time-series analysis deals with the extraction of underlying features to analyze and predict the temporal dynamics of financial assets (Navon and Keller, 2017). Due to the inherent uncertainty and non-analytic structure of financial markets (Tsay, 2005), the task is proven challenging, where classical linear statistical methods such as the VAR model, and statistical machine learning models have been widely applied (Ahmed

et al., 2010). In order to efficiently capture the non-linear nature of the financial time-series, advanced non-linear function approximators, such as RNN models (Mandic and Chambers, 2001) and Gaussian Processes (Roberts *et al.*, 2013) are also extensively used.

In this section, we introduce the VAR and RNN models, which comprise the basis for the model-based approach developed in Section 6.1.

2.4.1 Vector Autoregression (VAR)

Autoregressive (AR) processes can model univariate time-series and specify that future values of the series depend linearly on the past realizations of the series (Mandic, 2018a). In particular, a p -order autoregressive process $AR(p)$ satisfies:

$$\begin{aligned} x_t &= a_1 x_{t-1} + a_2 x_{t-2} + \cdots + a_p x_{t-p} + \varepsilon_t \\ &= \sum_{i=1}^p a_i x_{t-i} + \varepsilon_t \\ &= \mathbf{a}^T \vec{\mathbf{x}}_{t-p:t-1} + \varepsilon_t \in \mathbb{R} \end{aligned} \tag{2.42}$$

where ε_t is a stochastic term (an imperfectly predictable term), which is usually treated as white noise and $\mathbf{a} = [a_1, a_2, \cdots, a_p]^T$ the p model parameters/coefficients.

Extending the AR model for multivariate time-series, we obtain the **vector autoregressive** (VAR) process, which enables us to capture the cross-dependencies between series. For the general case of a M -dimensional p -order vector autoregressive process $VAR_M(p)$, it follows that:

$$\begin{aligned}
 \begin{bmatrix} x_{1,t} \\ x_{2,t} \\ \vdots \\ x_{M,t} \end{bmatrix} &= \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_M \end{bmatrix} + \begin{bmatrix} a_{1,1}^{(1)} & a_{1,2}^{(1)} & \cdots & a_{1,M}^{(1)} \\ a_{2,1}^{(1)} & a_{2,2}^{(1)} & \cdots & a_{2,M}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M,1}^{(1)} & a_{M,2}^{(1)} & \cdots & a_{M,M}^{(1)} \end{bmatrix} \begin{bmatrix} x_{1,t-1} \\ x_{2,t-1} \\ \vdots \\ x_{M,t-1} \end{bmatrix} + \\
 &\quad \begin{bmatrix} a_{1,1}^{(2)} & a_{1,2}^{(2)} & \cdots & a_{1,M}^{(2)} \\ a_{2,1}^{(2)} & a_{2,2}^{(2)} & \cdots & a_{2,M}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M,1}^{(2)} & a_{M,2}^{(2)} & \cdots & a_{M,M}^{(2)} \end{bmatrix} \begin{bmatrix} x_{1,t-2} \\ x_{2,t-2} \\ \vdots \\ x_{M,t-2} \end{bmatrix} + \cdots + \\
 &\quad \begin{bmatrix} a_{1,1}^{(p)} & a_{1,2}^{(p)} & \cdots & a_{1,M}^{(p)} \\ a_{2,1}^{(p)} & a_{2,2}^{(p)} & \cdots & a_{2,M}^{(p)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M,1}^{(p)} & a_{M,2}^{(p)} & \cdots & a_{M,M}^{(p)} \end{bmatrix} \begin{bmatrix} x_{1,t-p} \\ x_{2,t-p} \\ \vdots \\ x_{M,t-p} \end{bmatrix} + \begin{bmatrix} e_{1,t} \\ e_{2,t} \\ \vdots \\ e_{M,t} \end{bmatrix} \quad (2.43)
 \end{aligned}$$

or equivalently in compact a form:

$$\mathbf{x}_t = \mathbf{c} + \mathbf{A}_1 \mathbf{x}_{t-1} + \mathbf{A}_2 \mathbf{x}_{t-2} + \cdots + \mathbf{A}_p \mathbf{x}_{t-p} + \mathbf{e}_t = \mathbf{c} + \sum_{i=1}^p \mathbf{A}_i \mathbf{x}_{t-i} + \mathbf{e}_t \in \mathbb{R}^M \quad (2.44)$$

where $\mathbf{c} \in \mathbb{R}^M$ a vector of constants (intercepts), $\mathbf{A}_i \in \mathbb{R}^{M \times M}$ for $i = 1, 2, \dots, p$, the p parameter matrices and $\mathbf{e}_t \in \mathbb{R}^M$ a stochastic term, noise. Hence, VAR processes can adequately capture the dynamics of linear systems, under the assumption that they follow a Markov process of finite order, at most p (Murphy, 2012). In other words, the effectiveness of a p -th order VAR process relies on the assumption that the last p observations have all the sufficient statistics and information to predict and describe the future realizations of the process. As a result, we *enforce* a memory mechanism, keeping the p last values, $\vec{X}_{t-p:t-1}$, of the multivariate time-series and making predictions according to (2.44). Increasing the order of the model p , results in increased computational and memory complexity as well as a tendency to overfit the noise of the observed data. A $\text{VAR}_M(p)$ process has:

$$|P|_{\text{VAR}_M(p)} = M \times M \times p + M \quad (2.45)$$

parameters, hence they increase linearly with the model order. The systematic selection of the model order p can be achieved by minimizing an information criterion, such as the **Akaike Information Criterion** (AIC) (Mandic, 2018a), given by:

$$p_{\text{AIC}} = \min_{p \in \mathbb{N}} \left[\ln(\text{MSE}) + \frac{2p}{N} \right] \quad (2.46)$$

where MSE the mean squared error of the model and N the number of samples.

After careful investigation of equation (2.44), we note that the target $\vec{x}_t$ is given by an ensemble (i.e., linear combination) of p linear regressions, where the i -th regressor³ has (trainable) weights A_i and features $\mathbf{x}_{t-i}$. This interpretation of a VAR model allows us to interpret its strengths and weaknesses on a common basis with the neural network architectures, covered in subsequent parts. Moreover, this enables adaptive training (e.g., via Least-Mean-Square (Mandic, 2004) filter), which will prove useful in online learning, covered in Section 6.1.

Figure 2.14 illustrates a fitted $\text{VAR}_4(12)$ process, where the $p = p_{\text{AIC}} = 12$. We note that both in-sample (i.e., training) performance and out-of-sample (i.e., testing) performance are poor (see Table 2.2), despite the large number of parameters $|P|_{\text{VAR}_4(12)} = 196$.

2.4.2 Recurrent Neural Networks (RNN)

Recurrent neural networks (RNN) (Mandic and Chambers, 2001), are a family of neural networks with feedback loops which are very successful in processing sequential data. Most recurrent networks can also process sequences of variable length (Goodfellow *et al.*, 2016).

Consider the classical form of a dynamical system:

$$\mathbf{s}_t = f(\mathbf{s}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}) \quad (2.47)$$

where $\mathbf{s}_t$ and $\mathbf{x}_t$ the system state and input signal at time step t , respectively, while f a function parametrized by $\boldsymbol{\theta}$ that maps the previous state and the

³Let one of the regressors to have a bias vector that corresponds to c .

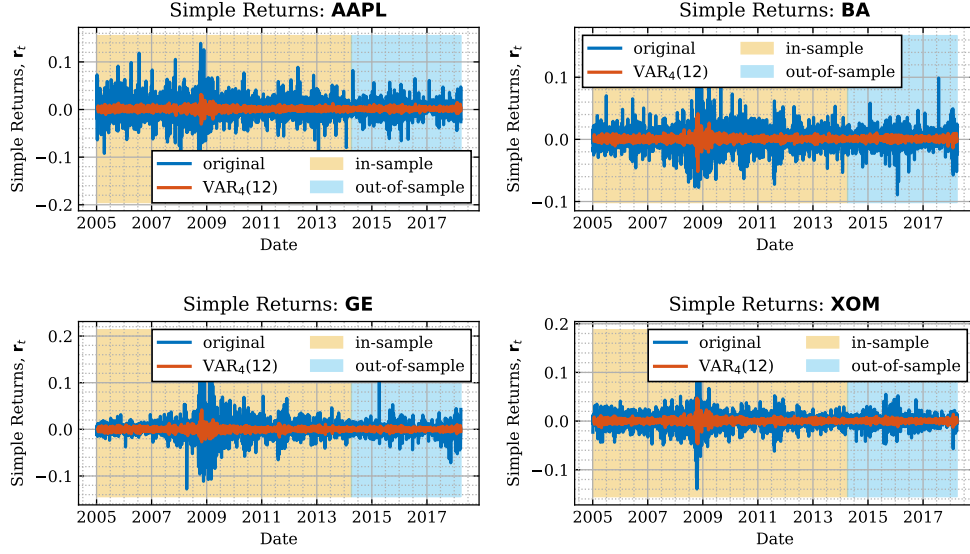


Figure 2.14: Vector autoregressive (VAR) time-series predictive model for assets simple returns. One step prediction is performed, where the realized observations are used as they come.

input signal to the new state. Unfolding the recursive definition in (2.47) for a finite value of t :

$$\begin{aligned} s_t &= f(s_{t-1}, x_t; \theta) \\ s_t &= f(f(s_{t-2}, x_{t-1}; \theta), x_t; \theta) \\ s_t &= f(f(f(\cdots (f(\cdots), x_{t-1}; \theta), x_t; \theta))) \end{aligned} \quad (2.48)$$

In general, f can be a highly non-linear function. Interestingly, a composite function of nested applications of f is responsible for generating the next state.

Many recurrent neural networks use equation (2.49) or a similar equation to define the values of their hidden units. To indicate that the state is the hidden units of the network, we now rewrite equation (2.47) using the variable h to represent the state:

$$h_t = f(h_{t-1}, x_t; \theta) \quad (2.49)$$

Then the hidden state h_t can be used to obtain the output signal y_t (i.e., observation) at time index t , assuming a non-linear relationship, described by function g that is parametrized by φ :

$$\hat{y}_t = g(h_t; \varphi) \quad (2.50)$$

The computational graph corresponding to (2.49) and (2.50) is provided in Figure 2.15. It can be shown that recurrent neural networks⁴ are universal function approximators (Cybenko, 1989), which means that if there is a relationship between past states and current input with next states, RNNs have the capacity to model it.

Another important aspect of RNNs is *parameter sharing*. Note in (2.48) that θ are the only parameters, shared between time steps. Consequently, the number of parameters of the model decreases significantly, enabling faster training and limiting model overfitting (“Supervised sequence labelling with recurrent neural networks. 2012”), compared to feedforward neural networks (i.e., multi-layer-perceptrons), which do not allow loops or any recursive connection. Feedforward networks can be also used with sequential data when memory is brute-forced⁵, leading to very large and deep architectures, and requiring a lot more time to train and effort to avoid overfitting, in order to achieve similar results with smaller RNNs (Mandic and Chambers, 2001).

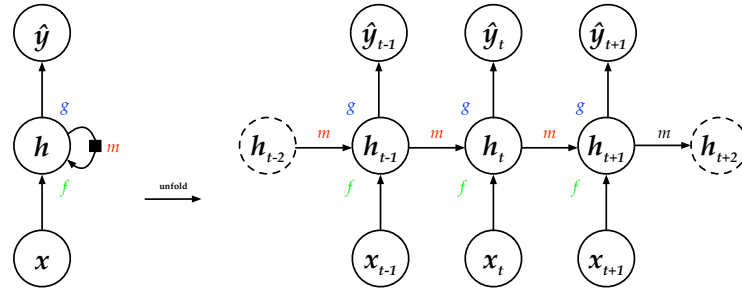


Figure 2.15: A generic recurrent network computational graph. This recurrent network processes information from the input x by incorporating it into the state h that is passed forward through time, which in turn is used to predict the target variable $\hat{y}$. (Left) Circuit diagram. The black square indicates a delay of a single time step. (Right) The same network seen as an unfolded computational graph, where each node is now associated with one particular time instance (Goodfellow *et al.*, 2016).

Simple RNNs, such that the one implementing equation (2.48), are not used because of the vanishing gradient problem (Hochreiter, 1998; Pascanu *et al.*, 2012), but instead variants, such as the **Gated Rectified Units** (GRU),

⁴And any neural network with certain non-linear activation functions, in general.

⁵Similar to VAR process memory mechanics.

are preferred because of their low computational complexity⁶ and simple architecture. The most variants introduce some filter/forget mechanism, responsible for selectively filtering out (or “forgetting”) past states, shaping the hidden state $\mathbf{h}$ in a highly non-linear fashion, but without accumulating the effects from all the history of observations, alleviating the vanishing gradients problem. Consulting the schematic in Figure 2.15, this filtering operation is captured by m (in red), which stands for selective “memory”.

Given a loss function $\mathcal{L}$ and historic data $\mathcal{D}$, then the process of training involves minimization of $\mathcal{L}$ conditioned on $\mathcal{D}$, where the parameters θ and φ are the decision variables of the optimization problem:

$$\underset{\theta, \varphi}{\text{minimize}} \quad \mathcal{L}(\theta, \varphi; \mathcal{D}) \quad (2.51)$$

which is usually addressed by adaptive variants of Stochastic Gradient Descent (SGD), such as **Adam** (Kingma and Ba, 2014), to ensure faster convergence and avoid saddle points. All these optimization algorithms rely on (estimates of) descent directions, obtained by the gradients of the loss function $\mathcal{L}$ with respect to the network parameters (θ, φ) , namely $\nabla_{\theta}(\mathcal{L})$ and $\nabla_{\varphi}(\mathcal{L})$. Due to the parameter sharing mechanics of RNNs, obtaining the gradients is non-trivial and thus **Backpropagation Through Time** (BPTT) (Werbos, 1990) algorithm is used, which efficiently calculates the contribution of each parameter to the loss function across all time steps.

The selection of the hyperparameters, such as the size of the hidden state $\mathbf{h}$ and the activation functions, can be performed using cross-validation or other empirical methods. Nonetheless, we choose to allow excess degrees of freedom to our model but regularize it using weight decay in the form of L^1 and L^2 norms, as well as dropout, according to the Gal and Ghahramani (2016) guidelines.

As any neural network layer, recurrent layers can be stack together or connected with other layers (i.e., affine or convolutional layers) forming deep architectures, capable of dealing with complex datasets.

For comparison with the VAR₄(12) process in Section 2.4.1, we train an RNN, comprised of two layers, one GRU layer followed by an affine layer, where the size of the hidden state is 3 or $\mathbf{h} \in \mathbb{R}^3$. The number of model parameters is $|P|_{\text{GRU-RNN}(4 \rightarrow 3 \rightarrow 4)} = 88$, but it significantly outperforms the VAR model, as suggested by Figure 2.16 and summary in Table 2.2.

⁶Compared to LSTM (Ortiz-Fuentes and Forcada, 1997).

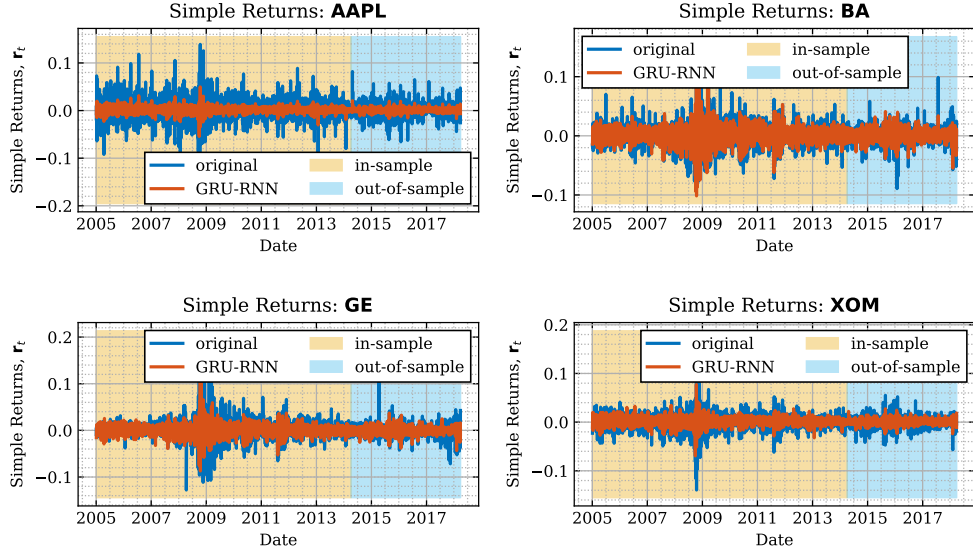


Figure 2.16: Gated recurrent unit recurrent neural network (GRU-RNN) time-series predictive model for assets simple returns. One step prediction is performed, where the realized observations are used as they come (i.e., set observation y_{t-1} equal to x_t , rather than predicted value $\hat{y}_{t-1}$).

	Training Error		Testing Error	
	VAR	GRU-RNN	VAR	GRU-RNN
AAPL	0.000411	0.000334	0.000520	0.000403
BA	0.000315	0.000090	0.000415	0.000124
GE	0.000284	0.000132	0.000392	0.000172
XOM	0.000210	0.000140	0.000341	0.000198

Table 2.2: Training and testing Mean Square Error (MSE) of VAR and GRU-RNN for Figures 2.14, 2.16. Despite the lower number of parameters, the GRU-RNN model can capture the non-linear dependencies and efficiently construct a "memory" (hidden state) to better model temporal dynamics.

Chapter 3

Portfolio Optimization

The notion of a portfolio has already been introduced in subsection 2.1.2, as a master asset, highlighting its representation advantage over single assets. Nonetheless, portfolios allow also investors to combine properties of individual assets in order to “amplify” the positive aspects of the market, while “attenuating” its negative impacts on the investment.

Figure 3.1 illustrates three randomly generated portfolios (with fixed portfolio weights over time given in Table 3.2), while Table 3.1 summarizes their performance. Importantly, we note the significant differences between the random portfolios, highlighting the importance that portfolio construction and asset allocation plays in the success of an investment. As Table 3.2 implies, short-selling is allowed in the generation of the random portfolios owing to the negative portfolio weights, such as $p_{AAPL}^{(0)}$ and $p_{MMM}^{(2)}$. Regardless, portfolio vector definition (2.1) is satisfied in all cases, since the portfolio vectors’ elements sum to one (column-wise addition).

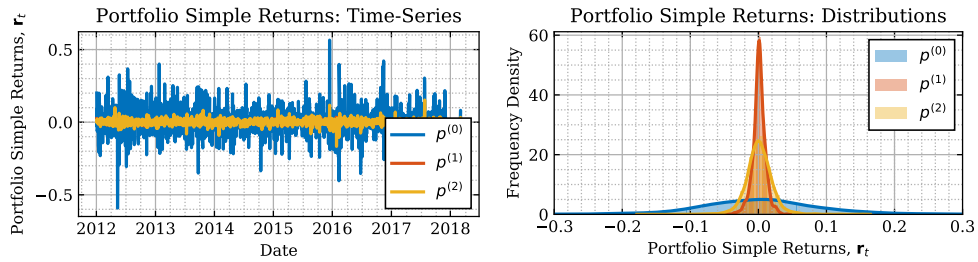


Figure 3.1: Simple returns of randomly allocated portfolios.

Portfolio Optimization aims to address the allocation problem in a systematic way, where an objective function reflecting the investor’s preferences is

3. PORTFOLIO OPTIMIZATION

Randomly Allocated Portfolios Performance Summary			
Performance Metrics	$p^{(0)}$	$p^{(1)}$	$p^{(2)}$
Mean Returns (%)	0.334685	0.097516	0.0869129
Cumulative Returns (%)	-83.8688	297.869	167.605
Volatility (%)	9.49456	0.94646	2.02911
Sharpe Ratio	1.35885	3.97176	1.65115
Max Drawdown (%)	173.656	44.8894	101.782
Average Drawdown Time (days)	70	7	24
Skewness	0.162362	-0.242675	-0.0217292
Kurtosis	3.27544	1.90174	7.83347
Value at Risk, $c = 0.05$ (%)	-14.0138	-1.52798	-2.88773
Conditional Value at Risk (%)	-20.371	-2.18944	-4.44684
Hit Ratio (%)	50.9091	56.633	51.4478
Average Win to Average Loss	1.06156	1.01662	1.06534

Table 3.1: Performance summary of Figure 3.1, illustrating a tremendous impact of portfolio construction on performance. Portfolio 0 ($p^{(0)}$) is outperformed by portfolio 1 ($p^{(1)}$) in all metrics, motivating the introduction of portfolio optimization.

	$p^{(0)}$	$p^{(1)}$	$p^{(2)}$
AAPL	-2.833049	0.172436	0.329105
GE	-2.604941	-0.061177	0.467233
BA	3.622328	0.313936	1.484903
JPM	6.764848	0.233765	0.092537
MMM	-3.949186	0.341040	-1.373778

Table 3.2: Random Portfolio Vectors of Figure 3.1.

constructed and optimized with respect to the portfolio vector.

In this chapter, we introduce the Markowitz Model (section 3.1), the first attempt to mathematically formalize and suggest an optimization method to address portfolio management. Moreover, we extend this framework to generic utility and objective functions (section 3.2), by taking transaction costs into account (section 3.3). The shortcomings of the methods discussed here encourage the development of context-agnostic agents, which is the focus of this thesis (section 6.2). However, the simplicity and robustness of the traditional portfolio optimization methods have motivated the supervised pre-training (Chapter 7) of the agents with Markowitz-like models as ground truths.

3.1 Markowitz Model

The **Markowitz model** (Markowitz, 1952; Kroll *et al.*, 1984) mathematically formulates the portfolio allocation problem, namely finding a portfolio vector w in a universe of M assets, according to the investment Greedy (Investment Advice 2.1) and Risk-Aversion (Investment Advice 2.2) criteria, constrained on the portfolio vector definition (2.1). Hence, the Markowitz model gives the optimal portfolio vector w_* which minimizes volatility for a given returns level, such that:

$$\sum_{i=1}^M w_{*,i} = 1, \quad w_* \in \mathbb{R}^M \quad (3.1)$$

3.1.1 Mean-Variance Optimization

For a trading universe of M assets, provided historical data, we obtain empirical estimates of the expected returns $\mu = [\mu_1, \mu_2, \dots, \mu_M]^T \in \mathbb{R}^M$, where μ_i the sample mean (2.16) of the i -th asset and the covariance $\Sigma \in \mathbb{R}^{M \times M}$, such that Σ_{ij} the empirical covariance (2.27) of the i -th and the j -th assets.

For given target expected return $\bar{\mu}_{target}$, determine the portfolio vector $w \in \mathbb{R}^M$ such that:

$$\underset{w}{\text{minimize}} \quad \frac{1}{2} w^T \Sigma w \quad (3.2)$$

$$\text{subject to} \quad w^T \mu = \bar{\mu}_{target} \quad (3.3)$$

$$\text{and} \quad \mathbf{1}_M^T w = 1 \quad (3.4)$$

where $\sigma^2 = w^T \Sigma w$ is the portfolio variance, $\mu = w^T \mu$ the portfolio expected return and the M -dimensional column vector of ones is denoted by $\mathbf{1}_M$.

For Lagrangian multipliers $\lambda, \kappa \in \mathbb{R}$, we form the Lagrangian function $\mathcal{L}$ (Papadimitriou and Steiglitz, 1998) such that:

$$\mathcal{L}(w, \lambda, \kappa) = \frac{1}{2} w^T \Sigma w - \lambda (w^T \mu - \bar{\mu}_{target}) - \kappa (\mathbf{1}_M^T w - 1) \quad (3.5)$$

We differentiate the Lagrangian function $\mathcal{L}$ ¹ and apply the first order necessary condition of optimality:

¹The covariance matrix Σ is by definition (2.24) symmetric, so $\frac{\partial(w^T \Sigma w)}{\partial w} = \Sigma w$.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{\Sigma} \mathbf{w} - \lambda \boldsymbol{\mu} - \kappa \mathbf{1}_M = 0 \quad (3.6)$$

Upon combining the optimality condition equations (3.3), (3.4) and (3.6) into a matrix form, we have:

$$\begin{bmatrix} \mathbf{\Sigma} & \boldsymbol{\mu} & \mathbf{1}_M \\ \boldsymbol{\mu}^T & 0 & 0 \\ \mathbf{1}_M^T & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ -\lambda \\ -\kappa \end{bmatrix} = \begin{bmatrix} 0 \\ \bar{\mu}_{target} \\ \mathbf{1}_M \end{bmatrix} \quad (3.7)$$

Under the assumption that $\mathbf{\Sigma}$ is full rank and $\boldsymbol{\mu}$ is not a multiple of $\mathbf{1}_M$, then equation (3.7) is solvable by matrix inversion (Boyd and Vandenberghe, 2004). The resulting portfolio vector $\mathbf{w}_{MVP}$ defines the **mean-variance optimal portfolio**.

$$\begin{bmatrix} \mathbf{w}_{MVP} \\ -\lambda \\ -\kappa \end{bmatrix} = \begin{bmatrix} \mathbf{\Sigma} & \boldsymbol{\mu} & \mathbf{1} \\ \boldsymbol{\mu}^T & 0 & 0 \\ \mathbf{1}^T & 0 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ \bar{\mu}_{target} \\ \mathbf{1} \end{bmatrix} \quad (3.8)$$

Note that mean-variance optimization is also used in signal processing and wireless communications in order to determine the optimal beamformer (Almeida *et al.*, 2015), using, for example, Minimum Variance Distortion Response filters (Xia and Mandic, 2013).

3.1.2 Quadratic Programming

Notice that the constraint (3.4) suggests that short-selling is allowed. This simplifies the formulation of the problem by relaxing conditions, enabling a closed form solution (3.8) as a set of linear equations. If short sales are prohibited, then an additional constraint should be added, and in this case, the optimization problem becomes:

$$\begin{aligned} & \underset{\mathbf{w}}{\text{minimize}} && \mathbf{w}^T \mathbf{\Sigma} \mathbf{w} \\ & \text{subject to} && \mathbf{w}^T \boldsymbol{\mu} = \bar{\mu}_{target} \\ & && \text{and } \mathbf{1}_M^T \mathbf{w} = 1 \\ & && \text{and } \mathbf{w} \succeq 0 \end{aligned} \quad (3.9)$$

where $\succeq$ designates an element-wise inequality operator. This problem cannot be reduced to the solution of a set of linear equations. It is termed a

quadratic program and it is solved numerically using gradient-based algorithms (Gill *et al.*, 1981). Optimization problems with quadratic objective functions and linear constraints fall into this framework.

Remark 3.1 Figure 3.2 illustrates the volatility to expected returns dependency of an example trading universe (i.e., yellow scatter points), along with all optimal portfolio solutions with and without short selling, left and right subfigures, respectively. The blue part of the solid curve is termed **Efficient Frontier** (Luenberger, 1997), and the corresponding portfolios are called efficient. These portfolios are obtained by the solution of (3.2).

Interestingly, despite the inferior performance of the individual assets' performance, appropriate linear combinations of them results in less volatile and more profitable master assets, demonstrating once again the power of portfolio optimization. Moreover, we note that the red part of the solid curve is inefficient, since for the same risk level (i.e., standard deviation) there are portfolios with higher expected returns, which aligns with the Greedy Criterion 2.1. Finally, we highlight that in case of short-selling (left subfigure) there are feasible portfolios, which have higher expected returns than any asset from the universe. This is possible since the low-performing assets can be shorted and the inventory from them can be invested in high-performance assets, amplifying their high returns. On the other hand, when short-selling is not allowed, the expected returns of any portfolio is restricted in the interval defined by the lowest and the highest empirical returns of the assets in the universe.

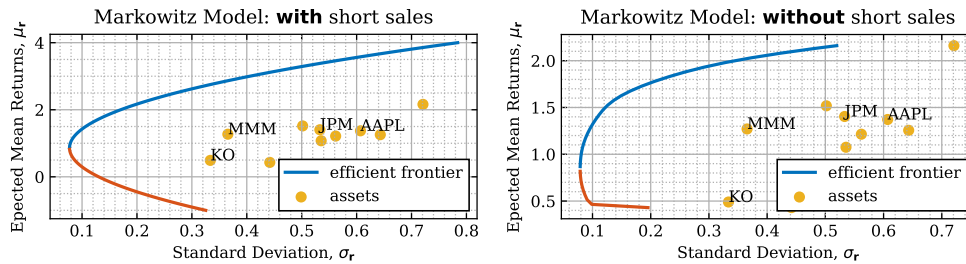


Figure 3.2: Efficient frontier for Markowitz model the with (Left) and without (Right) short-selling. The yellow points are the projections of single assets historic performance on the $\sigma - \mu$ (i.e., volatility-returns) plane. The solid lines are the portfolios, obtained by solving the Markowitz model optimization problem (3.2) for different values of $\bar{\mu}_{target}$. Note that the red points are rejected and only the blue loci is efficient.

3.2 Generic Objective Functions

In Section 2.3 we introduced various evaluation metrics that reflect our investment preferences. Extending the vanilla Markowitz model which minimizes risk (i.e., variance), constraint on a predetermined profitability level (i.e., expected returns), we can select any metric as the optimization objective function, constrained on the budget (3.4) and any other criteria we favor, as long as the fit in the quadratic programming framework. More complex objectives may be solvable with special non-linear optimizers, but there is no general principle. In this Section we exhibit how to translate evaluation metrics to objective functions, suitable for quadratic programming. Any of the functions presented can be used with and without short-selling, so in order to address the more difficult of the two cases, we will consider that long positions are only allowed. The other case is trivially obtained by ignoring the relevant constraint in the sign of weights.

3.2.1 Risk Aversion

Motivated by the Lagrangian formulation of the Markowitz model (3.5), we define the **Risk Aversion** portfolio, named after the risk aversion coefficient $\alpha \in \mathbb{R}_+$, given by the solution of the program:

$$\begin{aligned} & \underset{w}{\text{maximize}} && w^T \mu - \alpha w^T \Sigma w \\ & \text{subject to} && \mathbf{1}_M^T w = 1 \\ & && \text{and } w \succeq 0 \end{aligned} \tag{3.10}$$

The risk aversion coefficient α is model hyperparameter, which reflect the trade-off between portfolio expected returns ($w^T \mu$) and risk level ($w^T \Sigma w$) (Wilmott, 2007). For $\alpha \rightarrow 0$ the investor is infinitely greedy, they do not consider volatility and aim to maximize only returns. On the other hand, for $\alpha \rightarrow \infty$, the investor is infinitely risk-averse and selects the least risky portfolio, regardless its returns performance. Any positive value for α results in a portfolio which balances the two objectives weighted by the risk aversion coefficient. Figure 3.3 illustrates the volatility-returns plane for the same trading universe as in Figure 3.2, but the curve is obtained by the solution of (3.10). As expected, high values of α result in less volatile portfolios, while low risk aversion coefficient leads to higher returns.

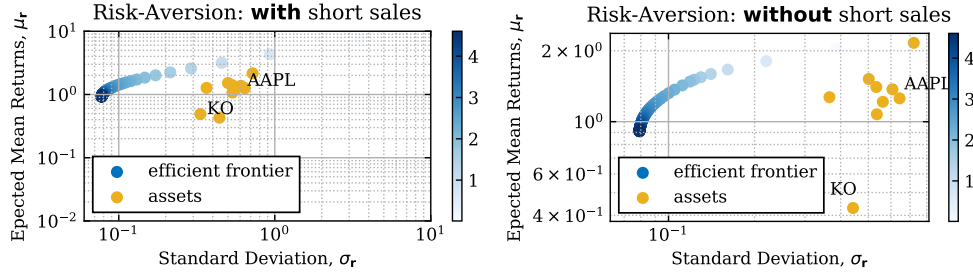


Figure 3.3: Risk-Aversion optimization efficient frontier with (*Right*) and without (*left*) short-selling. The *yellow* points are the projections of single assets historic performance on the $\sigma - \mu$ (i.e., volatility-returns) plane. The *blue* points are the efficient portfolios, or equivalently the solutions of the risk-aversion optimization problem (3.10) for different values of α , designated by the opacity of the blue color (see colorbar).

3.2.2 Sharpe Ratio

Both objective functions so far require hyperparameter tuning ($\bar{\mu}_{target}$ or α), hence either cross-validation or hand-picked selection is required (Kennedy, 2016). On the other hand, in Section 2.3 we motivated the use of Sharpe Ratio (2.35) as a Signal-to-Noise Ratio equivalent for finance, which is not parametric. Considering the Sharpe Ratio as the objective function of the program:

$$\begin{aligned} & \underset{w}{\text{maximize}} && \frac{w^T \mu}{\sqrt{w^T \Sigma w}} \\ & \text{subject to} && \mathbf{1}_M^T w = 1 \\ & && \text{and } w \succeq 0 \end{aligned} \quad (3.11)$$

Unlike the aforementioned methods, by solving the optimization problem in (3.11) we obtain a single portfolio which is guaranteed to be optimal for the provided universe.

3.3 Transaction Costs

In real stock exchanges, such as NYSE (Wikipedia, 2018c), NASDAQ (Wikipedia, 2018b) and LSE (Wikipedia, 2018a), trading activities (buying or selling) are accompanied with expenses, including brokers' commissions and spreads (Investopedia, 2018g; Quantopian, 2017), they are usually referred to as **transaction costs**. Therefore every time a new portfolio vector is determined

(portfolio re-balancing), the corresponding transaction costs should be subtracted from the budget.

In order to simplify the analysis around transaction costs we will use the rule of thumb (Quantopian, 2017), charging 0.2% for every activity. For example, if three stocks A are bought with price 100\$ each, then the transaction costs will be 0.6\$. Let the price of the stock A raising at price 107\$, when we decide to sell all three stocks, then the transaction costs will be 0.642\$.

3.3.1 Mathematical Formalization

Given any objective function, $\mathcal{J}$, the transaction costs are subtracted from the returns term in order to adjust the profit & loss, accounting for the expenses of trading activities. Therefore, we solve the optimization program:

$$\begin{aligned} & \underset{w}{\text{maximize}} && \mathcal{J} - \mathbf{1}_M^T \beta \|w_0 - w\|_1 \\ & \text{subject to} && \mathbf{1}_M^T w = 1 \\ & && \text{and } w \succeq 0 \end{aligned} \tag{3.12}$$

where $\beta \in \mathbb{R}$ the transactions cost (i.e., 0.002 for standard 0.2% commissions), and $w_0 \in \mathbb{R}^M$ the initial portfolio, since the last re-balancing. All the parameters of the model² (3.12) are given, since β is market-specific, and w_0 the current position. Additionally, the transactions cost term can be seen as a regularization term which penalizes excessive trading and restricts large trades (i.e., large $\|w_0 - w\|$).

The objective function $\mathcal{J}$ can be any function that can be optimized according to the framework developed in Section 3.2. For example the risk-aversion with transaction costs optimization program is given by:

$$\underset{w}{\text{maximize}} \quad w^T \mu - \alpha w^T \Sigma w - \mathbf{1}_M^T \beta \|w_0 - w\|_1 \tag{3.13}$$

while the Sharpe Ratio optimization with transaction costs is:

$$\underset{w}{\text{maximize}} \quad \frac{w^T \mu - \mathbf{1}_M^T \beta \|w_0 - w\|_1}{\sqrt{w^T \Sigma w}} \tag{3.14}$$

We highlight that in (3.13) is subtracted from $\mathcal{J}$ directly since all terms have the same units³, while in (3.14) the transaction cost term is subtracted di-

²Not considering the objective function's $\mathcal{J}$ parameters.

³The parameter α is not dimensionless.

rectly from the expected returns, since Sharpe Ratio is unitless.

3.3.2 Multi-Stage Decision Problem

The involvement of transaction costs make Portfolio Management a **Multi-Stage Decision Problem** (Neuneier, 1996), which in simple terms means that two sequences of states with the same start and end state will have different value. For instance, imagine two investors, both of which have an initial budget of 100\$. On Day 1, the first investor uses all of his budget to construct a portfolio according to his preferred objective function, paying 0.2\$ for transaction costs, according to Quantopian (2017). By Day 3, the market prices have changed but the portfolio of the first investor has not changed in value and decided to liquidate all of his investment, paying another 0.2\$ for selling his portfolio. On Day 5 both of the investors (re-)enter the market make identical investments and hence pay the same commission fees. Obviously, the two investors have the same start and end states but their intermediate trajectories lead to different reward streams.

From (3.12) it is obvious that w_0 affects the optimal allocation w . In a sequential portfolio optimization setting, the past decisions (i.e., w_0) will have a direct impact on the optimality of the future decisions (i.e., w), therefore apart from the maximization of immediate rewards, we should also focus on eliminating the negative effects on the future decisions. As a consequence, sequential asset allocation is a multi-stage decision problem where myopic optimal actions can lead to sub-optimal cumulative rewards. This setting encourages the use of Reinforcement Learning agents which aim to maximize long-term rewards, even if that means acting sub-optimally in the near-future from the traditional portfolio optimization point of view.

Reinforcement Learning

Reinforcement learning (RL) refers to both a learning problem and a sub-field of machine learning (Goodfellow *et al.*, 2016). As a learning problem (Szepesvári, 2010), it refers to learning to control a system (*environment*) so as to maximize some numerical value, which represents a long-term objective (*discounted cumulative reward signal*). Recalling the analysis in Section 3.3.2, sequential portfolio management

In this chapter we introduce the necessary tools to analyze stochastic dynamical systems (Section 4.1). Moreover, we review the major components of a reinforcement learning algorithm (Section 4.2, as well as extensions of the formalization of dynamical systems (Section 4.4), enabling us to reuse some of those tools to more general and intractable otherwise problems.

4.1 Dynamical Systems

Reinforcement learning is suitable in optimally controlling dynamical systems, such as the general one illustrated in Figure 4.1: A controller (*agent*) receives the controlled *state* of the system and a *reward* associated with the last *state transition*. It then calculates a control signal (*action*) which is sent back to the system. In response, the system makes a transition to a *new state* and the cycle is repeated. The goal is to learn a way of controlling the system (*policy*) so as to maximize the total reward. The focus of this report is on **discrete-time** dynamical systems, though most of the notions developed extend to continuous-time systems.

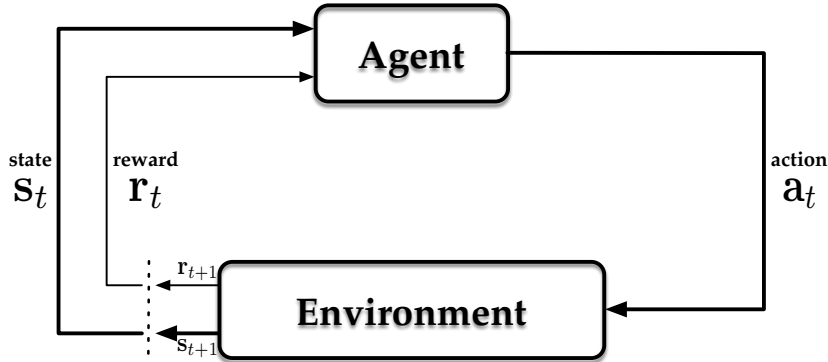


Figure 4.1: High-level stochastic dynamical system schematic (Sutton and Barto, 1998).

4.1.1 Agent & Environment

The term **agent** is used to refer to the controller, while **environment** is used interchangeably with the term system. The goal of a reinforcement learning algorithm is the development (training) of an agent capable of successfully interacting with the environment, such that it maximizes some scalar objective over time.

4.1.2 Action

Action $a_t \in \mathcal{A}$ is the control signal that the agent sends back to the system at time index t . It is the only way that the agent can influence the environment state and as a result, lead to different reward signal sequences. The **action space** $\mathcal{A}$ refers to the set of actions that the agent is allowed to take and it can be:

- Discrete: $\mathcal{A} = \{a_1, a_2, \dots, a_M\}$;
- Continuous: $\mathcal{A} \subseteq [c, d]^M$.

4.1.3 Reward

A **reward** $r_t \in \mathbb{B} \subseteq \mathbb{R}$ is a scalar feedback signal, which indicates how well the agent is doing at discrete time step t . The agent aims to maximize cumulative reward, over a sequence of steps.

Reinforcement learning addresses **sequential decision making** tasks (Silver, 2015b), by training agents that optimize delayed rewards and can evaluate the long-term consequences of their actions, being able to sacrifice immediate reward to gain more long-term reward. This special property of rein-

forcement learning agents is very attractive for financial applications, where investment horizons range from few days and weeks to years or decades. In the latter cases, myopic agents can perform very poorly since evaluation of long-term rewards is essential in order to succeed (Mnih *et al.*, 2016). However, the applicability of reinforcement learning depends vitally on the hypothesis:

Hypothesis 4.1 (Reward Hypothesis) *All goals can be described by the maximization of expected cumulative reward.*

Consequently, the selection of the appropriate reward signal for each application is very crucial. It influences the agent learned strategies since it reflects its goals. In Section 5.4, a justification for the selected reward signal is provided, along with an empirical comparison between other metrics mentioned in Section 2.3.

4.1.4 State & Observation

The **state**, $s_t \in \mathbb{S}$, is also a fundamental element of reinforcement learning, but it is usually used to refer to both the environment state and the agent state.

The agent does not always have direct access to the state, but at every time step, t , it receives an observation, $o_t \in \mathbb{O}$.

Environment State

The **environment state** s_t^e is the internal representation of the system, used in order to determine the next observation o_{t+1} and reward r_{t+1} . The environment state is usually invisible to the agent and even if it visible, it may contain irrelevant information (Sutton and Barto, 1998).

Agent State

The **history** $\vec{h}_t$ at time t is the sequence of observations, actions and rewards up to time step t , such that:

$$\vec{h}_t = (o_1, a_1, r_1, o_2, a_2, r_2, \dots, o_t, a_t, r_t) \quad (4.1)$$

The **agent state** (a.k.a **state**) s_t^a is the internal representation of the agent about the environment, used in order to select the next action a_{t+1} and it can be any function of the history:

$$s_t^a = f(\vec{h}_t) \quad (4.2)$$

The term **state space** S is used to refer to the set of possible states the agents can observe or construct. Similar to the action space, it can be:

- Discrete: $S = \{s_1, s_2, \dots, s_n\}$;
- Continuous: $S \subseteq \mathbb{R}^N$.

Observability

Fully observable environments allow the agent to directly observe the environment state, hence:

$$o_t = s_t^e = s_t^a \quad (4.3)$$

Partially observable environments offer indirect access to the environment state, therefore the agent has to construct its own state representation s_t^a (“Supervised sequence labelling with recurrent neural networks. 2012”), using:

- Complete history: $s_t^a \equiv \vec{h}_t$;
- Recurrent neural network: $s_t^a \equiv f(s_{t-1}^a, o_t; \theta)$.

Upon modifying the basic dynamical system in Figure 4.1 in order to take partial observability into account, we obtain the schematic in Figure 4.2. Note that f is function unknown to the agent, which has access to the observation o_t but not to the environment state s_t . Moreover, $\mathcal{R}_s^a$ and $\mathcal{P}_{ss'}^a$ are the reward generating function and the transition probability matrix (function) of the MDP, respectively. Treating the system as a probabilistic graphical model, the state s_t is a latent variable that either deterministically or stochastically (depending on the nature of f) determines the observation o_t . In a partially observable environment, the agent needs to reconstruct the environment state, either by using the complete history h_t or a stateful sequential model (i.e., recurrent neural network, see (2.49)).

4.2 Major Components of Reinforcement Learning

Reinforcement Learning agents may include one or more of the following components (Silver, 2015b):

- **Policy:** agent’s behavior function;
- **Value function:** how good is each state, or state-action pair;
- **Model:** agent’s representation of the environment.

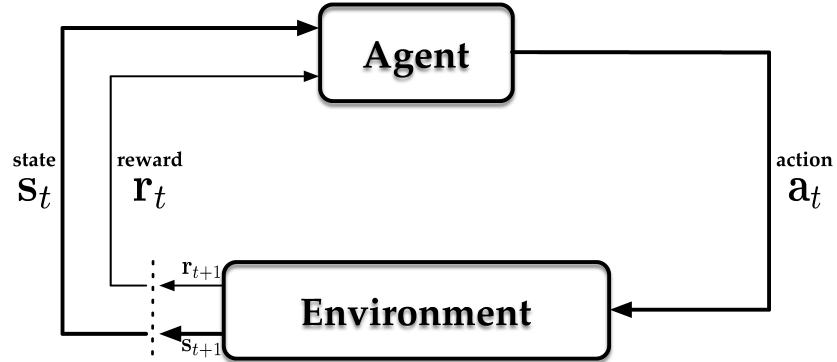


Figure 4.2: High-level stochastic partially observable dynamical system schematic.

In this Section, we discuss these components and highlight their importance and impact on algorithm design.

4.2.1 Return

Let γ be the **discount factor** of future rewards, then the **return** G_t (also known as **future discounted reward**) at time index t is given by:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad \gamma \in [0, 1] \quad (4.4)$$

4.2.2 Policy

Policy, π , refers to the behavior of an agent. It is a mapping function from "state to action" (Witten, 1977), such that:

$$\pi : \mathcal{S} \rightarrow \mathcal{A} \quad (4.5)$$

where $\mathcal{S}$ and $\mathcal{A}$ are respectively the state space and the action space. A policy function can be:

- Deterministic: $A_{t+1} = \pi(s_t)$;
- Stochastic: $\pi(a|s) = \mathbb{P}[a_t = a | s_t = s]$.

4.2.3 Value Function

State-value function, v_π , is the expected return, G_t , starting from state s , which then follows a policy π (Szepesvári, 2010), that is:

$$v_\pi : \mathcal{S} \rightarrow \mathbb{B}, \quad v_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s] \quad (4.6)$$

where $\mathcal{S}$ and $\mathcal{B}$ are respectively the state space and the rewards set ($\mathcal{B} \subseteq \mathbb{R}$).

Action-value function, q_π , is the expected return, G_t , starting from state s , upon taking action a , which then follows a policy π (Silver, 2015b):

$$q_\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{B}, \quad q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a] \quad (4.7)$$

where $\mathcal{S}, \mathcal{A}, \mathcal{B}$ the state space, the action space and the reward set, respectively.

4.2.4 Model

A **model** predicts the next state of the environment, s_{t+1} , and the corresponding reward signal, r_{t+1} , given the current state, s_t , and the action taken, a_t , at time step, t . It can be represented by a **state transition probability matrix** $\mathcal{P}$ given by:

$$\mathcal{P}_{ss'}^a : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}, \quad \mathcal{P}_{ss'}^a = \mathbb{P}[s_{t+1} = s' | s_t = s, a_t = a] \quad (4.8)$$

and a **reward generating function** $\mathcal{R}$:

$$\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{B}, \quad \mathcal{R}_s^a = \mathbb{E}[r_{t+1} | s_t = s, a_t = a] \quad (4.9)$$

where $\mathcal{S}, \mathcal{A}, \mathcal{B}$ the state space, the action space and the reward set, respectively.

4.3 Markov Decision Process

A special type of discrete-time stochastic dynamical systems are Markov Decision Processes (MDP). They possess strong properties that guarantee convergence to the global optimum policy (i.e., strategy), while by relaxing some of the assumption, they can describe any dynamical system, providing a powerful representation framework and a common way of controlling dynamical systems.

4.3.1 Markov Property

A state S_t (Silver, 2015c) satisfies the **Markov property** if and only if (iff):

$$\mathbb{P}[s_{t+1} | s_t, s_{t-1}, \dots, s_1] = \mathbb{P}[s_{t+1} | s_t] \quad (4.10)$$

This implies that the previous state, s_t , is a sufficient statistic for predicting the future, therefore the longer-term history, $\vec{h}_t$, can be discarded.

4.3.2 Definition

Any fully observable environment, which satisfies equation (4.3), can be modeled as a **Markov Decision Process** (MDP). A Markov Decision Process (Poole and Mackworth, 2010) is an object (i.e., 5-tuple) $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ where:

- $\mathcal{S}$ is a *finite* set of states (state space), such that they satisfy the Markov property, as in definition (4.10)
- $\mathcal{A}$ is a *finite* set of actions (action space);
- $\mathcal{P}$ is a state transition probability matrix;;
- $\mathcal{R}$ is a reward generating function;
- γ is a discount factor.

4.3.3 Optimality

Apart from the expressiveness of MDPs, they can be optimally solved, making them very attractive.

Value Function

The **optimal state-value function**, v_* , is the maximum state-value function over all policies:

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad \forall s \in \mathcal{S} \quad (4.11)$$

The **optimal action-value function**, q_* , is the maximum action-value function over all policies:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \quad (4.12)$$

Policy

Define a partial ordering over policies (Silver, 2015c)

$$\pi \leq \pi' \iff v_{\pi}(s) \leq v_{\pi'}(s), \quad \forall s \in \mathcal{S} \quad (4.13)$$

For an MDP the following theorems¹ are true:

Theorem 4.2 (Policy Optimality) *There exists an optimal policy, π_* , that is better than or equal to all other policies, such that $\pi_* \geq \pi, \forall \pi$.*

¹The proofs are based on the contraction property of Bellman operator (Poole and Mackworth, 2010).

Theorem 4.3 (State-Value Function Optimality) *All optimal policies achieve the optimal state-value function, such that $v_{\pi_*}(s) = v_*(s)$, $\forall s \in \mathcal{S}$.*

Theorem 4.4 (Action-Value Function Optimality) *All optimal policies achieve the optimal action-value function, such that $q_{\pi_*}(s, a) = q_*(s, a)$, $\forall s \in \mathcal{S}, a \in \mathcal{A}$.*

4.3.4 Bellman Equation

Given a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, because of the Markov property (4.10) that states in $\mathcal{S}$ satisfy:

- The policy π is a distribution over actions given states

$$\pi(s|a) = \mathbb{P}[a_t = a | s_t = s] \quad (4.14)$$

Without loss of generality we assume that the policy π is stochastic because of the state transition probability matrix $\mathcal{P}$ (Sutton and Barto, 1998). Owing to the Markov property, MDP policies depend only on the current state and are time-independent, stationary (Silver, 2015c), such that

$$a_t \sim \pi(\cdot | s_t), \quad \forall t > 0 \quad (4.15)$$

- The state-value function v_π can be decomposed into two parts: the immediate reward and the discounted reward of successor state γr_{t+1} :

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | s_t = s] \\ &\stackrel{(4.4)}{=} \mathbb{E}_\pi[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) | s_t = s] \\ &\stackrel{(4.4)}{=} \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} | s_t = s] \\ &\stackrel{(4.10)}{=} \mathbb{E}_\pi[r_{t+1} + \gamma v_\pi(s_{t+1}) | s_t = s] \end{aligned} \quad (4.16)$$

- The action-value function q_π can be similarly decomposed to

$$q_\pi(s) = \mathbb{E}_\pi[r_{t+1} + \gamma q_\pi(s_{t+1}, s_{t+1}) | s_t = s, a_t = a] \quad (4.17)$$

Equations (4.16) and (4.17) are the **Bellman Expectation Equations** for Markov Decision Processes formulated by Bellman (1957).

4.3.5 Exploration-Exploitation Dilemma

Search, or seeking a goal under uncertainty, is a ubiquitous requirement of life² (Hills *et al.*, 2015). Not only machines but also humans and animals usually face the trade-off between *exploiting* known opportunities and *exploring* for better opportunities elsewhere. This is a fundamental dilemma in reinforcement learning, where the agent may need to act “sub-optimally” in order to explore new possibilities, which may lead it to better strategies. Every reinforcement learning algorithm takes into account this trade-off, trying to balance search for new opportunities (exploration) with secure³ actions (exploitation). From an optimization point of view, if an algorithm is greedy and only exploits, it may converge fast, but it runs the risk of sticking to a local minimum. Exploration, may at first slow down convergence, but it can lead to previously unexplored regions of the search space, resulting in an improved solution. Most algorithms perform exploration either by artificially adding noise to the actions, which is attenuated while the agent gains experience, or modelling the uncertainty of each action (Gal, 2016) in the Bayesian optimization framework.

4.4 Extensions

Markov Decisions Processes can be exploited by reinforcement learning agents, who can optimally solve them (Sutton and Barto, 1998; Szepesvári, 2010). Nonetheless, most real-life applications are not satisfying one or more of the conditions stated in Section 4.3.2. As a consequence, modifications of them lead to other types of processes, such as Infinite MDP and Partially Observable MDP, which in turn can realistically fit a lot of application domains.

4.4.1 Infinite Markov Decision Process

In the case of either the state space S , or the action space A , or both being infinite⁴ then the environment can be modelled as an **Infinite Markov Decision Process** (IMDP). Therefore, in order to implement the policy π or/and the action-value function q_π in a computer, a differentiable function approximation method must be used (Sutton *et al.*, 2000a), such as a least

²Metaphorically speaking, an agent “lives” in the environment.

³This does not reflect any risk-sensitive metric or strategy, “secure” is used here to describe actions that have been tried in the past and their outcomes are predictable to some extend.

⁴Countably infinite (discrete) or continuous.

squares function approximation or a neural network (Michalski *et al.*, 2013). An IMDP action-state dynamics are described by a **transition probability function** $\mathcal{P}_{ss'}^a$ and not a matrix, since the state or/and the action spaces are continuous.

4.4.2 Partially Observable Markov Decision Process

If $s_t^e \neq s_t^a$ then the environment is partially observable and it can be modeled as a **Partially Observable Markov Decision Process (POMDP)**. POMDP is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{R}, \mathcal{Z}, \gamma \rangle$ where:

- $\mathcal{O}$ is a *finite* set of observations (observation space)
- $\mathcal{Z}$ is an observation function, $\mathcal{Z}_{s'o}^a = \mathbb{P}[O_{t+1} | s_{t+1} = s', a_t = a]$

It is important to notice that, any dynamical system can be viewed as a POMDP and all the algorithms used for MDPs are applicable, *without convergence guarantees* though.

Part II

Innovation

Financial Market as Discrete-Time Stochastic Dynamical System

In Chapter 3, the task of static asset allocation as well as traditional methods of its assessment were introduced. Our interest in dynamically (i.e., sequentially) constructing portfolios led to studying Reinforcement Learning basic components and concepts in Chapter 4, which suggest a framework to deal with sequential decision making tasks. However, in order to leverage the reinforcement learning tools, it is necessary to translate the problem (i.e., asset allocation) into a discrete-time stochastic dynamical system and, in particular, into a Markov Decision Process (MDP). Note that not all of the strong assumptions of an MDP (Section 4.3.2) can be satisfied, hence we resort to the relaxation of some of the assumptions and consideration of the MDP extensions, discussed in Section 4.4. However, the convergence and optimality guarantees are obviously not applicable under this formalization.

In this chapter, we mathematically formalize financial markets as discrete-time stochastic dynamical systems. Firstly, we consider the necessary assumptions for this formalization (Section 5.1), followed by the framework (Sections 5.2, 5.3, 5.4) which enables reinforcement learning agents to interact with the financial market in order to optimally address portfolio management.

5.1 Assumptions

Back-test tradings are only considered, where the trading agent pretends to be back in time at a point in the market history, not knowing any “future” market information, and does paper trading from then onward (Jiang *et*

al., 2017). As a requirement for the back-test experiments, the following three assumptions must apply: *sufficient liquidity*, *zero slippage* and *zero market impact*, all of which are realistic if the traded assets' volume in a market is high enough (Wilmott, 2007).

5.1.1 Sufficient Liquidity

An asset is termed **liquid** if it can be converted into cash quickly, with little or no loss in value (Investopedia, 2018c).

Assumption 5.1 (Sufficient Liquidity) *All market assets are liquid and every transaction can be executed under the same conditions.*

5.1.2 Zero Slippage

Slippage refers to the difference between the expected price of a trade and the price at which the trade is actually executed (Investopedia, 2018e).

Assumption 5.2 (Zero Slippage) *The liquidity of all market assets is high enough that, each trade can be carried out immediately at the last price when an order is placed.*

5.1.3 Zero Market Impact

Asset prices are determined by the **Law of Supply and Demand** (Investopedia, 2018b), therefore any trade impacts the balance between them, hence affects the price of the asset being traded.

Assumption 5.3 (Zero Market Impact) *The capital invested by the trading agent is so insignificant that it has no influence on the market.*

5.2 Action Space

In order to solve the asset allocation task, the trading agent should be able to determine the portfolio vector $\mathbf{w}_t$ at every time step t , therefore the action $\mathbf{a}_t$ at time t is the portfolio vector $\mathbf{w}_{t+1}$ at time $t + 1$:

$$\mathbf{a}_t \equiv \mathbf{w}_{t+1} \stackrel{(2.1)}{\triangleq} \begin{bmatrix} w_{1,t+1}, & w_{1,t+1}, & \dots, & w_{M,t+1} \end{bmatrix} \quad (5.1)$$

hence the action space $\mathbb{A}$ is a subset of the continuous M -dimensional real space $\mathbb{R}^M$:

$$\mathbf{a}_t \in \mathbb{A} \subseteq \mathbb{R}^M, \quad \forall t \geq 0 \quad \text{subject to} \quad \sum_{i=1}^M a_{i,t} = 1 \quad (5.2)$$

If short-selling is prohibited, the portfolio weights are strictly non-negative, or:

$$\mathbf{a}_t \in \mathbb{A} \subseteq [0, 1]^M, \quad \forall t \geq 0 \quad \text{subject to} \quad \sum_{i=1}^M a_{i,t} = 1 \quad (5.3)$$

In both cases, nonetheless, the action space is infinite (continuous) and hence the financial market needs to be treated as an Infinite Markov Decision Process (IMDP), as described in Section 4.4.1.

5.3 State & Observation Space

5.3.1 Observation

At any time step t , we can only observe asset prices, thus the price vector $\mathbf{p}_t$ (2.3) is the observation $\mathbf{o}_t$, or equivalently:

$$\mathbf{o}_t \equiv \mathbf{p}_t \stackrel{(2.3)}{\triangleq} \begin{bmatrix} p_{1,t} & p_{2,t} & \cdots & p_{M,t} \end{bmatrix} \quad (5.4)$$

hence the observation space $\mathcal{O}$ is a subset of the continuous M -dimensional positive real space $\mathbb{R}_+^M$, since prices are non-negative real values:

$$\mathbf{o}_t \in \mathcal{O} \subseteq \mathbb{R}_+^M, \quad \forall t \geq 0 \quad (5.5)$$

Since one-period prices do not fully capture the market state¹, *financial markets are partially observable* (Silver, 2015b). As a consequence, equation (4.3) is not satisfied and we should construct the agent's state $\mathbf{s}_t^a$ by processing the observations $\mathbf{o}_t \in \mathcal{O}$. In Section 4.1.4, two alternatives to deal with partial observability were suggested, considering:

- (4.1)
1. Complete history: $\mathbf{s}_t^a \equiv \vec{\mathbf{h}}_t \triangleq (\mathbf{o}_1, \mathbf{a}_1, r_1, \dots, \mathbf{o}_t, \mathbf{a}_t, r_t);$
2. Recurrent neural network: $\mathbf{s}_t^a \equiv f(\mathbf{s}_{t-1}^a, \mathbf{o}_t; \boldsymbol{\theta});$

¹If prices were a VAR(1) process (Mandic, 2018a), then financial markets are pure MDPs.

where in both cases we assume that the agent state approximates the environment state $s_t^a = \hat{s}_t^e \approx s_t^e$. While the first option may contain all the environment information by time t , it does not scale well, since the memory and computational load grow linearly with time t . A GRU-RNN (see Section 2.4.2), on the other hand, can store and process efficiently the historic observation in an adaptive manner as they arrive, filtering out any uninformative observations out. We will be referring to this recurrent layer as the **state manager**, since it is responsible for constructing (i.e., managing) the agent state. This layer can be part of any neural network architecture, enabling end-to-end differentiability and training.

Figure 5.1 illustrates examples of a financial market observations o_t and the corresponding actions a_t of a random agent.

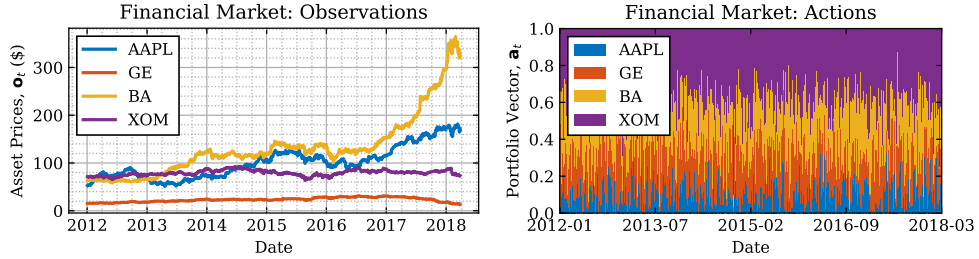


Figure 5.1: Example universe of assets as dynamical system, including AAPL (Apple), GE (General Electric), BA (Boeing Company) and XOM (Exxon Mobil Corporation). (Left) Financial market asset prices, observations o_t . (Right) Portfolio manager, agent, portfolio vectors, actions a_t ; the portfolio coefficients are illustrated in a stacked bar chat, where at each time step, they sum to unity according to equation (2.1).

5.3.2 State

In order to assist and speed-up the training of the state manager, we process the raw observations o_t , obtaining $\hat{s}_t$. In particular, thanks to the representation and statistical superiority of log returns over asset prices and simple returns (see 2.2.2), we use log returns matrix $\vec{p}_{t-T:t}$ (2.10), of fixed window size² T . We also demonstrate another important property of log returns, which suits the nature of operations performed by neural networks, the function approximators used for building agents. Neural network layers apply non-linearities to weighted *sums* of input features, hence the features do not

²Expanding windows are more appropriate in case the RNN state manager is replaced by complete the history $\vec{h}_t$.

multiply (Nasrabadi, 2007) with each other³, but only with the layer weights (i.e., parameters). Nonetheless, by summing the log returns, we equivalently multiply the gross returns, hence the networks are learning non-linear functions of the products of returns (i.e., asset-wise and cross-asset) which are the building blocks of the covariances between assets. Therefore, by simply using the log returns we enable cross-asset dependencies to be easily captures.

Moreover, transaction costs are taken into account, and since (3.12) suggests that the previous time step portfolio vector w_{t-1} affects transactions costs, we also append the w_t , or equivalently a_{t-1} by (5.1), to the agent state, obtaining the 2-tuple:

$$\hat{s}_t \triangleq \langle w_t, \vec{\rho}_{t-T:t} \rangle = \left\langle \begin{bmatrix} w_{1,t} \\ w_{2,t} \\ \vdots \\ w_{M,t} \end{bmatrix}, \begin{bmatrix} \rho_{1,t-T} & \rho_{1,t-T+1} & \cdots & \rho_{1,t} \\ \rho_{2,t-T} & \rho_{2,t-T+1} & \cdots & \rho_{2,t} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{M,t-T} & \rho_{M,t-T+1} & \cdots & \rho_{M,t} \end{bmatrix} \right\rangle \quad (5.8)$$

where $\rho_{i,(t-\tau) \rightarrow t}$ the log cumulative returns of asset i between the time interval $[t - \tau, t]$. Overall, the agent state is given by:

$$s_t^a \equiv f(s_{t-1}^a, \hat{s}_t; \theta) \quad (5.9)$$

where f the state manager non-linear mapping function. When an observation arrives, we calculate⁴ $\hat{s}_t$ and feed it to the state manager (GRU-RNN).

Therefore the state space S is a subset of the continuous K -dimensional real space $\mathbb{R}^K$, where K the size of the hidden state in the GRU-RNN state manager:

³This is the issue that multiplicative neural networks (Salinas and Abbott, 1996) try to address. Consider two scalar feature variables x_1 and x_2 and the target scalar variable y such that:

$$f(x_1, x_2) \triangleq y = x_1 * x_2, \quad x_1, x_2 \in \mathbb{R} \quad (5.6)$$

It is very hard for a neural network to learn this function, but a logarithmic transformation of the features transforms the problem to a very simple sum of logarithms using the property:

$$\log(x_1) + \log(x_2) = \log(x_1 * x_2) = \log(y) \quad (5.7)$$

⁴Most terms can be stored or pre-calculated.

$$\mathbf{s}_t^a \in \mathcal{S} \subseteq \mathbb{R}^K, \quad \forall t \geq 0 \quad (5.10)$$

Figure 5.2 illustrates two examples of the processed observation $\hat{\mathbf{s}}_t$. The agent uses this as input to determine its internal state, which in turn drives its policy. They look meaningless and impossible to generalize from, with the naked eye, nonetheless, in Chapter 6, we demonstrated the effectiveness of this representation, especially thanks to the combination of the convolutional and recurrent layers combination.

Overall, the financial market should be modelled as an Infinite Partially Observable Markov Decision Process (IPOMDP), since:

- The action space is continuous (infinite), $\mathcal{A} \subseteq \mathbb{R}^M$;
- The observations $\mathbf{o}_t$ are not sufficient statistics (partially observable) of the environment state;
- The state space is continuous (infinite), $\mathcal{S} \subseteq \mathbb{R}^K$.

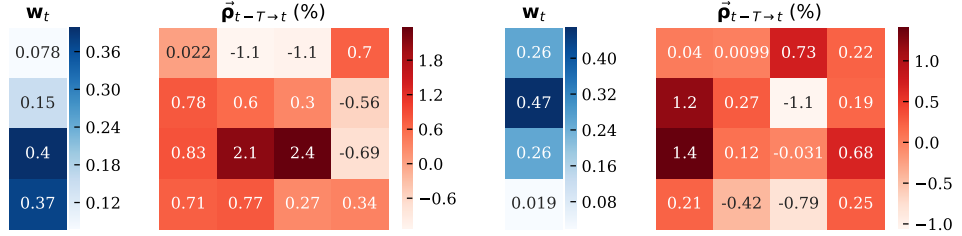


Figure 5.2: Examples of processed observation 2-tuples for two randomly selected time steps.

5.4 Reward Signal

The determination of the reward signal is usually the most challenging step in the design of a reinforcement learning problem. According to the Reward Hypothesis 4.1, the reward is a scalar value, which fully specifies the goals of the agent, and the maximization of the expected cumulative reward leads to the optimal solution of the task. Specifying the optimal reward generating function is the field of study of Inverse Reinforcement Learning (Ng and Russell, 2000) and Inverse Optimal Control (Moylan and Anderson, 1973).

In our case, we develop a generic, modular framework, which enables comparison of various reward generating functions⁵, including log returns, (negative) volatility and Sharpe Ratio. Section 2.3 motivates a few reward function candidates, most of which are implemented and tested in Chapter 9. It is worth highlighting that the reinforcement learning methods, by default, aim to maximize the *expected cumulative reward signal*, hence the optimization problem that the agent (parametrized by θ) solves is given by:

$$\underset{\theta}{\text{maximize}} \quad \sum_{t=1}^T \mathbb{E}[\gamma^t r_t] \quad (5.11)$$

For instance, when we refer to log returns (with transaction costs) as the reward generating function, the agent solves the optimization problem:

$$\underset{\theta}{\text{maximize}} \quad \sum_{t=1}^T \mathbb{E}[\gamma^t \ln(1 + \mathbf{w}_t^T \mathbf{r}_t - \beta \|\mathbf{w}_{t-1} - \mathbf{w}_t\|_1)] \quad (5.12)$$

where the argument of the logarithm is the adjusted by the transaction costs gross returns at time index t (see (2.12) and (3.3)).

⁵Transaction costs are included in all case.

Trading Agents

Current state-of-the-art algorithmic portfolio management methods:

- Address the decision making task of asset allocation by solving a prediction problem, heavily relying on the accuracy of predictive models for financial time-series (Aldridge, 2013; Heaton *et al.*, 2017), which are usually unsuccessful, due to the stochasticity of the financial markets;
- Make unrealistic assumptions about the second and higher-order statistical moments of the financial signals (Necchi, 2016; Jiang *et al.*, 2017);
- Deal with binary trading signals (i.e., BUY, SELL, HOLD) (Neuneier, 1996; Deng *et al.*, 2017), instead of assigning portfolio weights to each asset, and hence limiting the scope of their applications.

On the other hand, the representation of the financial market as a discrete-time stochastic dynamical system, as derived in Chapter 5 enables the development of a unified framework for training reinforcement learning trading agents. In this chapter, this framework is exploited by:

- Model-based Reinforcement Learning agents, as in Section 6.1, where vector autoregressive processes (VAR) and recurrent neural networks (RNN) are fitted to environment dynamics, while the derived agents perform planning and control (Silver, 2015a). Similar to (Aldridge, 2013; Heaton *et al.*, 2017), these agents are based on a predictive model of the environment, which is in turn used for decision making. Their performance is similar to known algorithms and thus they are used as baseline models for comparison;
- Model-free Reinforcement Learning agents, as in Section 6.2, which directly address the decision making task of sequential and multi-step

optimization. Modifications to the state-of-the-art reinforcement learning algorithms, such as Deep Q-Network (DQN) (Mnih *et al.*, 2015) and Deep Deterministic Policy Gradient (DDPG) (Lillicrap *et al.*, 2015), enable their incorporation to the trading agents training framework.

Algorithm 1 provides the general setup for reinforcement learning algorithms discussed in this chapter, based on which, experiments on a small universe of real market data are carried out, for testing their efficacy and illustration purposes. In Part III, all the different agents are compared on a larger universe of assets with different reward functions, a more realistic and practical setting.

Algorithm 1: General setup for trading agents.

inputs : trading universe of M -assets
 initial portfolio vector $w_1 = a_0$
 initial asset prices $p_0 = o_0$
 objective function $\mathcal{J}$

output: optimal agent parameters θ_*, φ_*

```

1 repeat
2   for  $t = 1, 2, \dots T$  do
3     observe 2-tuple  $\langle o_t, r_t \rangle$ 
4     calculate gradients  $\nabla_{\theta} \mathcal{J}(r_t)$  and  $\nabla_{\varphi} \mathcal{J}(r_t)$            // BPTT
5     update agent parameters  $\theta, \varphi$ 
6       using adaptive gradient optimizers           // ADAM
7     get estimate of agent state:  $s_t \approx f(\cdot, o_t)$            // (5.9)
8     sample and take action:  $a_t \sim \pi(\cdot | s_t; \varphi)$            // portfolio
       rebalance
9   end
10 until convergence
11 set  $\theta_*, \varphi_* \leftarrow \theta, \varphi$ 

```

6.1 Model-Based Reinforcement Learning

Upon a revision of the schematic of a generic partially observable environment (i.e., dynamical system) as in Figure 4.2, it is noted that given the transition probability function $\mathcal{P}_{ss'}^a$ of the system, the reinforcement learning task reduces to *planning* (Atkeson and Santamaria, 1997); simulate future states by recursively calling $\mathcal{P}_{ss'}^a$ L times and choose the roll-outs (i.e., trajectories) which maximize cumulative reward, via dynamic programming (Bertsekas *et al.*, 1995):

$$\mathbf{s}_t \xrightarrow{\mathcal{P}_{ss'}^a} \mathbf{s}_{t+1} \xrightarrow{\mathcal{P}_{ss'}^a} \dots \xrightarrow{\mathcal{P}_{ss'}^a} \mathbf{s}_{t+L} \quad (6.1)$$

$$\mathbf{a}_{t+1} \equiv \max_{a \in \mathcal{A}} \mathcal{J}(\mathbf{a} | \mathbf{a}_t, \mathbf{s}_t, \dots, \mathbf{s}_{t+L}) \quad (6.2)$$

Note that due to the assumptions made in Section 5.1, and especially the Zero Market Impact assumption 5.3, the agent actions do not affect the environment state transitions, or equivalently the financial market is an *open loop system* (Feng and Palomar, 2016), where the agent actions do not modify the system state, but only the received reward:

$$p(\mathbf{s}_{t+1} | \mathbf{s}, \mathbf{a}) = p(\mathbf{s}_{t+1} | \mathbf{s}) \Rightarrow \mathcal{P}_{ss'}^a = \mathcal{P}_{ss'} \quad (6.3)$$

Moreover, the reward generating function is known, as explained in section 5.4, hence *a model of the environment is obtained by learning only the transition probability function $\mathcal{P}_{ss'}$* .

6.1.1 System Identification

In the area of Signal Processing and Control Theory, the task under consideration is usually termed as **System Identification** (SI), where an approximation of the environment, the “model”, is fitted such that it captures the environment dynamics:

$$\underbrace{\hat{\mathcal{P}}_{ss'}}_{\text{model}} \approx \underbrace{\mathcal{P}_{ss'}}_{\text{environment}} \quad (6.4)$$

Figure 6.1 illustrates schematically the system identification wiring of the circuit, where the model, represented by $\hat{\mathcal{P}}_{ss'}$, is compared against the true transition probability function $\mathcal{P}_{ss'}$ and the loss function $\mathcal{L}$ (i.e., mean squared error) is minimized by optimizing with respect to the model parameters θ . It is worth highlighting that the transition probability function is by definition stochastic (4.8) hence the candidate fitted models should ideally be able to capture and incorporate this uncertainty. As a result, model-based reinforcement learning usually (Deisenroth and Rasmussen, 2011; Levine *et al.*, 2016; Gal *et al.*, 2016) relies on probabilistic graphical models, such as Gaussian Processes (Rasmussen, 2004) or Bayesian Networks (Ghahramani, 2001), which are non-parametric models that do not output point estimates, but learn the generating process of the data p_{data} and hence enable sampling

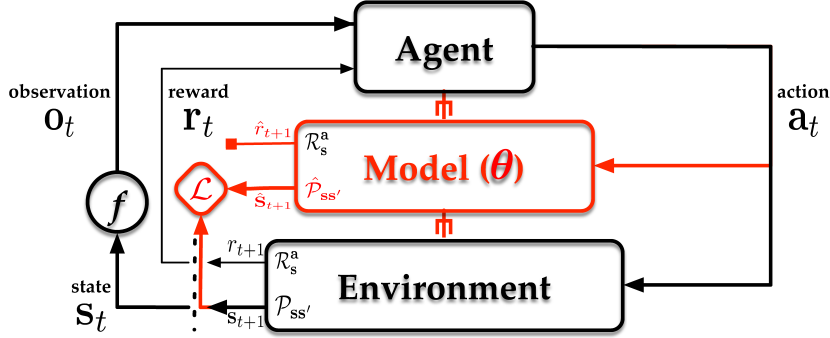


Figure 6.1: General setup for System Identification (SI) (i.e., model-based reinforcement learning) for solving a discrete-time stochastic partially observable dynamical system.

from the posterior distribution. Sampling from the posterior distribution allows us to have stochastic predictions that respect model dynamics.

In this section we shall focus, nonetheless, only on vector autoregressive processes (VAR) and recurrent neural networks (RNN) for modelling $\mathcal{P}_{ss'}$, trained on an adaptive fashion (Mandic and Chambers, 2001), given by Algorithm 2. An extension of vanilla RNNs to bayesian RNNs could be also tried using the MC-dropout trick from (Gal and Ghahramani, 2016).

6.1.2 Vector Autoregression (VAR)

Following on the introduction of the vector autoregressive processes (VAR) in Section 2.4.1, and using the fact that the transition probability model $\hat{\mathcal{P}}_{ss'}$ is a one-step time-series predictive model, we investigate the effectiveness of VAR processes as time-series predictors.

Agent Model

The vector autoregressive processes (VAR) regress past values of multivariate time-series with the future values (see equation (2.44)). In order to satisfy the covariance stationarity assumption (Mandic, 2018a), we fit a VAR process on the log-returns ρ_t , and not the raw observations $\mathbf{o}_t$ (i.e., price vectors), since the latter is known to be highly non-stationary¹. The model is pre-trained on historic data (i.e., batch supervised learning training (Murphy, 2012)) and it is updated online, following the gradient $\nabla_{\theta} \mathcal{L}(\hat{\rho}_{tt}, \rho_t)$, as described in Algorithm 2. The model takes the form:

¹In the wide-sense (Bollerslev, 1986).

Algorithm 2: General setup for adaptive model-based trading agents.

inputs : trading universe of M -assets
 initial portfolio vector $w_1 = a_0$
 initial asset prices $p_0 = o_0$
 loss function $\mathcal{L}$
 historic dataset $\mathcal{D}$
output: optimal model parameters θ_*

```

1 batch training on  $\mathcal{D}$ 
2    $\theta \leftarrow \operatorname{argmax}_{\theta} p(\theta|\mathcal{D})$  // MLE
3 repeat
4   for  $t = 1, 2, \dots T$  do
5     predict next state  $\hat{s}_t$  // via  $\hat{\mathcal{P}}_{ss'}$ 
6     observe tuple  $\langle o_t, r_t \rangle$ 
7     get estimate of agent state:  $s_t \approx f(\cdot, o_t)$  // (5.9)
8     calculate gradients:  $\nabla_{\theta} \mathcal{L}(\hat{s}_t, s_t)$  // backprop
9     update model parameters  $\theta$ 
10    using adaptive gradient optimizers // ADAM
11    plan and take action  $a_t$  // portfolio rebalance
12  end
13 until convergence
14 set  $\theta_* \leftarrow \theta$ 

```

$$\mathcal{P}_{ss'} : \quad \rho_t \stackrel{(2.44)}{\approx} c + \sum_{i=1}^p A_i \rho_{t-i} \quad (6.5)$$

$$\text{planning :} \quad \left(s_t \xrightarrow{\mathcal{P}_{ss'}^a} \dots \xrightarrow{\mathcal{P}_{ss'}^a} s_{t+L} \right) \stackrel{(6.2)}{\Rightarrow} a_{t+1} \equiv \max_{a \in \mathbb{A}} \mathcal{J}(a|a_t, s_t, \dots, s_{t+L}) \quad (6.6)$$

Related Work

Vector autoregressive processes have been widely used for modelling financial time-series, especially returns, due to their pseudo-stationary nature (Tsay, 2005). In the context of model-based reinforcement learning there is no results in the open literature on using VAR processes in this context, nonetheless, control engineering applications (Akaike, 1998) have extensively used autoregressive models to deal with dynamical systems.

Evaluation

Figure 6.2 illustrates the cumulative rewards and the prediction error power are illustrated. Observe that the agent is highly correlated with the market (i.e., S&P 500) and overall collects lower cumulative returns. Moreover, note that the market crash in 2009 (Farmer, 2012), affects the agent significantly, leading to a decline by 179.6% (drawdown), taking as many as 2596 days to recover, from 2008-09-12 to 2015-10-22.

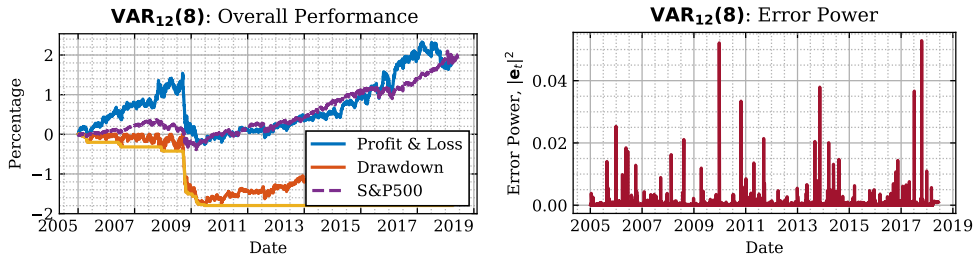


Figure 6.2: Order-eight vector autoregressive model-based reinforcement learning agent on a 12-asset universe (i.e., $\text{VAR}_{12}(8)$), pre-trained on historic data between 2000-2005 and trained online onward. (Left) Cumulative rewards and (maximum) drawdown of the learned strategy, against the S&P 500 index (traded as SPY). (Right) Mean squared prediction error for single-step predictions.

Weaknesses

The order- p VAR model, $\text{VAR}(p)$, assumes that the underlying generating process:

1. Is covariance stationary;
2. Satisfies the order- p Markov property;
3. Is linear, conditioned on past samples.

Unsurprisingly, most of these assumptions are not realistic for real market data, which reflects the poor performance of the method illustrated in Figure 6.2.

Expected Properties 6.1 (Non-Stationary Dynamics) *The agent should be able to capture non-stationary dynamics.*

Expected Properties 6.2 (Long-Term Memory) *The agent should be able to selectively remember past events without brute force memory mechanisms (e.g., using lagged values as features).*

Expected Properties 6.3 (Non-Linear Model) *The agent should be able to learn non-linear dependencies between features.*

6.1.3 Recurrent Neural Network (RNN)

The limitations of the vector autoregressive processes regarding stationarity, linearity and finite memory assumptions are overcome by the recurrent neural network (RNN) environment model. Inspired by the effectiveness of recurrent neural networks in time-series prediction (Gers *et al.*, 1999; Mandic and Chambers, 2001; Hénaff *et al.*, 2011) and the encouraging results obtained from the initial one-step predictive GRU-RNN model in Figure 2.16, we investigate the suitability of RNNs in the context of model-based reinforcement learning, used as environment predictors.

Agent Model

Revisiting Algorithm 2 along with the formulation of RNNs in Section 2.4.2, we highlight the steps:

$$\text{state manager : } \overset{(4.2)}{s_t = f(s_{t-1}, \rho_t)} \quad (6.7)$$

$$\text{prediction : } \hat{\rho}_{t+1} \approx V\sigma(s_t) + b \quad (6.8)$$

$$\text{planning : } \left(s_t \xrightarrow{\mathcal{P}_{ss'}^a} \dots \xrightarrow{\mathcal{P}_{ss'}^a} s_{t+L} \right) \overset{(6.2)}{\Rightarrow} a_{t+1} \equiv \max_{a \in A} \mathcal{J}(a | a_t, s_t, \dots, s_{t+L}) \quad (6.9)$$

where V and b the weights matrix and bias vector of the output affine layer² of the network and σ a non-linearity (i.e., rectified linear unit (Nair and Hinton, 2010), hyperbolic tangent, sigmoid function.). Schematically the network is depicted in Figure 6.3.

Related Work

Despite the fact that RNNs were first used decades ago (Hopfield, 1982; Hochreiter and Schmidhuber, 1997; Mandic and Chambers, 2001), recent ad-

²In Deep Learning literature (Goodfellow *et al.*, 2016), the term **affine** refers to a neural network layer with parameters W and b that performs a mapping from an input matrix X to an output vector y according to

$$f_{\text{affine}}(X; W, b) = \hat{y} \triangleq XW + b \quad (6.10)$$

The terms *affine*, *fully-connected* (FC) and *dense* refer to the same layer configuration.

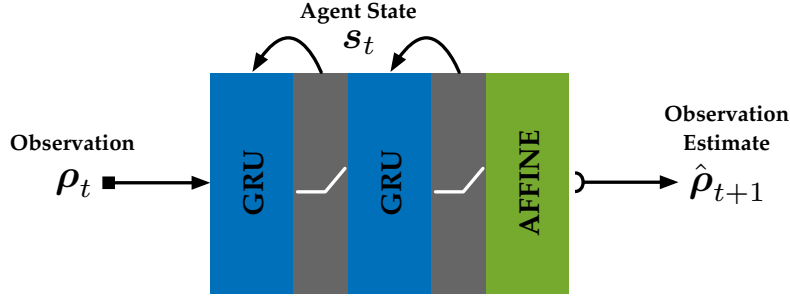


Figure 6.3: Two layer gated recurrent unit recurrent neural network (GRU-RNN) model-based reinforcement learning agent; receives log returns ρ_t as input, builds internal state s_t and estimates future log returns $\hat{\rho}_{t+1}$. Regularized mean squared error is used as the loss function, optimized with the ADAM (Kingma and Ba, 2014) adaptive optimizer.

vances in adaptive optimizers (e.g., RMSProp (Tieleman and Hinton, 2012), ADAM (Kingma and Ba, 2014)) and deep learning have enabled the development of *deep* recurrent neural networks for sequential data modelling (e.g., time-series, text). Since financial markets are dominated by dynamic structures and time-series, RNNs have been extensively used for modelling dynamic financial systems (Tino *et al.*, 2001; Chen *et al.*, 2015; Heaton *et al.*, 2016; Bao *et al.*, 2017). In most cases, feature engineering prior to training is performed so that meaningful financial signals are combined, instead of raw series. Our approach was rather context-free, performing pure technical analysis of the series, without involving manual extraction and validation of high-order features.

Evaluation

Figure 6.4 illustrates the performance of a two-layer gated recurrent unit recurrent neural network, which is not outperforming the vector autoregressive predictor as much as it was expected. Again, we note the strong correlation with the market (i.e., S&P 500). The 2008 market crash affects the RNN agent as well, which manages to recover faster than the VAR agent, leading to an overall 221.1% cumulative return, slightly higher than the market index.

6.1.4 Weaknesses

Having developed both vector autoregressive and recurrent neural network model-based reinforcement learning agents, we conclude that despite the ar-

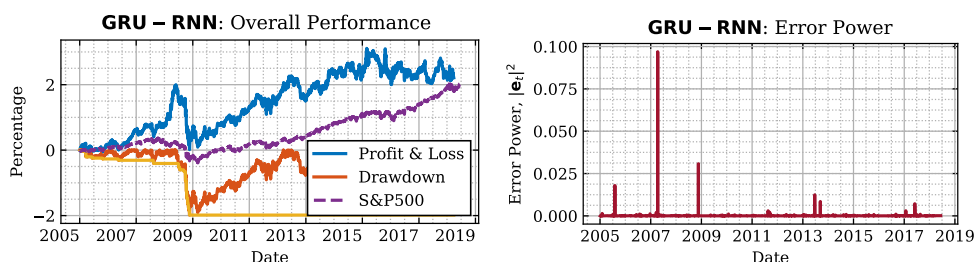


Figure 6.4: Two-layer gated recurrent unit recurrent neural network (GRU-RNN) model-based reinforcement learning agent on a 12-asset universe, pre-trained on historic data between 2000-2005 and trained online onward (*Left*) Cumulative rewards and (maximum) drawdown of the learned strategy, against the S&P 500 index (traded as SPY). (*Right*) Mean square prediction error for single-step predictions.

chitectural simplicity of system identification, who are under-performing. The inherent randomness (i.e., due to uncertainty) of the financial time-series (e.g., prices, returns) affects the model training and degrades predictability.

In spite of the promising results in Figures 2.14, 2.16, where one-step predictions are considered, control and planning (6.2) are only effective when accurate multi-step predictions are available. Therefore, the process of first fitting a model (e.g., VAR, RNN or Gaussian Process) and then use an external optimization step, results in two sources of approximation error, where the error propagates over time and reduces performance.

A potential improvement of these methods would be *manually feature engineering the state space*, such as extracting meaningful econometric signals (Greene, 2003) (e.g., volatility regime shifts, earning or dividends announcements, fundamentals) which in turn are used for predicting the returns. This is, in a nutshell, the traditional approach that quantitative analysts (LeBaron, 2001) have been using for the past decades. The computational power has been radically improved over the years, which permits larger (i.e., deeper and wider) models to be fitted, while elaborate algorithms, such as variational inference (Archer *et al.*, 2015), have made previously intractable tasks possible. Nonetheless, this approach involves a lot of tweaks and human intervention, which are the main aspects we aim to attenuate.

6.2 Model-Free Reinforcement Learning

In the final Section, we assume that solving a system identification problem (i.e., explicitly inferring environment dynamics) is easier than addressing directly the initial objective; the maximization of a cumulative reward signal (e.g., log returns, negative volatility, sharpe ratio). Nonetheless, predicting accurately the evolution of the market was proven challenging, resulting in ill-performing agents.

In this section, we adapt an orthogonal approach, where we do not rely on an explicit model of the environment, but we parametrize the agent value function or/and policy directly. At first glance, it may seem counter-intuitive how skipping the modelling of the environment can lead to a meaningful agent at all, but consider the following example from daily life. Humans are able to easily handle objects or move them around. Unarguably, this is a consequence of experience that we have gained over time, however, if we are asked to explain the environment model that justifies our actions, it is much more challenging, especially for an one year old kid, who can successfully play with toys but fails to explain this task using Newtonian physics.

Another motivating example from the portfolio management and trading field: *pairs trading* is a simple trading strategy (Gatev *et al.*, 2006), which relies on the assumption that historically correlated assets will preserve this relationship over time³. Hence when the two assets start deviating from one another, this is considered an arbitrage opportunity (Wilmott, 2007), since they are expected to return to a correlated state. This opportunity is exploited by taking a long position for the rising stock and a short position for the falling. If we would like to train a model-based reinforcement learning agent to perform pairs trading, it would be almost impossible or too unstable, regardless the algorithm simplicity. On the other hand, a value-based or policy gradient agent could perform this task with minimal effort, replicating the strategy, because the pairs trading strategy does not rely on future value prediction, but much simpler statistical analysis (i.e., cross-correlation), which, in case of model-based approaches, should be translated into an optimization problem of an unrelated objective - the prediction error. Overall, using model-free reinforcement learning improves efficiency (i.e., only one episode fitting) and also allows finer control over the policy, but it also limits the policy to only be as good as the learned model. More importantly, for the task under consideration (i.e., asset allocation) it is shown to be easier to

³Pairs trading is selected for educational purposes only, we are not claiming that it is optimal in any sense.

represent a good policy than to learn an accurate model.

The model-free reinforcement learning agents are summarized in Algorithm 1, where different objective functions and agent parametrizations lead to different approaches and hence strategies. We classify these algorithms as:

- **Value-based:** learn a state value function v (4.16), or an action-value function q (4.17) and use it with an *implicit* policy (e.g., ϵ -greedy (Sutton and Barto, 1998));
- **Policy-based:** learn a policy directly by using the reward signal to guide adaptation.

In this chapter, we will, first, focus on value-based algorithms, which exploit the state (action) value function, as defined in equations (4.6), (4.7) as an estimate for expected cumulative rewards. Then policy gradient methods will be covered, which parametrize directly the policy of the agent, and perform gradient ascent to optimize performance. Finally, a universal agent will be introduced, which reduces complexity (i.e., computational and memory) and generalizes strategies across assets, regardless the trained universe, based on parameter sharing (Bengio *et al.*, 2003) and transfer learning (Pan and Yang, 2010a) principles.

6.2.1 Deep Soft Recurrent Q-Network (DSRQN)

A wide range of value-based reinforcement learning algorithms (Sutton and Barto, 1998; Silver, 2015d; Szepesvári, 2010) have been suggested and used over time. The Q-Learning is one of the simplest and best performing ones (Liang *et al.*, 2016), which motivates us to extend it to continuous action spaces to fit our system formulation.

Q-Learning

Q-Learning is a simple but very effective value-based model-free reinforcement learning algorithm (Watkins, 1989). It works by successively improving its evaluations of the action-value function q , and hence the name. Let $\hat{q}$, be the estimate of the true action-value function, then $\hat{q}$ is updated online (every time step) according to:

$$\hat{q}(s_t, a_t) \leftarrow \hat{q}(s_t, a_t) + \alpha \underbrace{\left[r_t + \gamma \max_{a' \in \mathcal{A}} \hat{q}(s_{t+1}, a') - \hat{q}(s_t, a_t) \right]}_{\text{TD error, } \delta_{t+1}} \quad (6.11)$$

where $\alpha \geq 0$ the learning rate and $\gamma \in [0, 1]$ the discount factor. In the literature, the term in the square brackets is usually called **Temporal Difference Error** (TD error), or δ_{t+1} (Sutton and Barto, 1998).

Theorem 6.1 *For a Markov Decision Process, Q-learning converges to the optimum action-values with probability 1, as long as all actions are repeatedly sampled in all states and the action-values are represented discretely.*

The proof of Theorem 6.1 is provided by Watkins (1989) and relies on the contraction property of the Bellman Operator⁴ (Sutton and Barto, 1998), showing that:

$$\hat{q}(s, a) \rightarrow q_*(s, a) \quad (6.13)$$

Note that equation (6.11) is practical only in the cases that:

1. The state space is discrete, and hence the action-value function can be stored in a digital computer, as a grid of scalars;
2. The action space is discrete, and hence at each iteration, the maximization over actions $a \in \mathcal{A}$ is tractable.

The Q-Learning steps are summarized in Algorithm 3.

Related Work

Due to the success of the Q-Learning algorithm, early attempts to modify it to fit the asset allocation task were made by Neuneier (1996), who attempted to use a differentiable function approximator (i.e., neural network) to represent the action-value function, and hence enabled the use of Q-Learning in *continuous state spaces*. Nonetheless, he was restricted to discrete action spaces and thus was acting on *buy* and *sell* signals only. In the same year, Moody *et al.* (1998) used a similar approach but introduced new reward signals, namely general utility functions and the differential Sharpe Ratio, which are considered in Chapter 9.

Recent advances in deep learning (Goodfellow *et al.*, 2016) and stochastic optimization methods (Boyd and Vandenberghe, 2004) led to the first practical

⁴The Bellman Operator, $\mathfrak{B}$ is **a-contraction** with respect to some norm $\|\cdot\|$ since it can be shown that (Rust, 1997):

$$\|\mathfrak{B}s - \mathfrak{B}\bar{s}\| \leq a\|s - \bar{s}\| \quad (6.12)$$

Therefore it follows that:

1. The sequence $s, \mathfrak{B}s, \mathfrak{B}^2s, \dots$ converges for every s ;
2. $\mathfrak{B}$ has a unique fixed point s^* , which satisfies $\mathfrak{B}s^* = s^*$ and all sequences $s, \mathfrak{B}s, \mathfrak{B}^2s, \dots$ converge to this unique fixed point s^* .

Algorithm 3: Q-Learning with greedy policy.

inputs : trading universe of M -assets
initial portfolio vector $w_1 = a_0$
initial asset prices $p_0 = o_0$
initial
output: optimal action-value function q_*

```

1 initialize q-table:  $\hat{q}(s, a) \leftarrow 0, \forall s \in \mathcal{S}, a \in \mathcal{A}$ 
2 while convergence do
3   for  $t = 0, 1, \dots T$  do
4     select greedy action:  $a_t = \max_{a' \in \mathcal{A}} \hat{q}(s_t, a')$ 
5     observe tuple  $\langle s_{t+1}, r_t \rangle$ 
6     update q-table:
7        $\hat{q}(s_t, a_t) \leftarrow \hat{q}(s_t, a_t) + \alpha [r_t + \gamma \max_{a' \in \mathcal{A}} \hat{q}(s_{t+1}, a') - \hat{q}(s_t, a_t)]$ 
8   end
9 end

```

use case of Q-Learning in high-dimensional state spaces (Mnih *et al.*, 2015). Earlier work was limited to low-dimensional applications, where shallow neural network architectures were effective. Mnih *et al.* (2015) used a few algorithmic tricks and heuristics, and managed to stabilize the training process of the **Deep Q-Network** (DQN). The first demonstration was performed on Atari video games, where trained agents outperformed human players in most games.

Later, Hausknecht and Stone (2015) published a modified version of the DQN for partially observable environments, using a recurrent layer to construct the agent state, giving rise to the **Deep Recurrent Q-Network** (DRQN). Nonetheless, similar to the vanilla Q-Learning and enhanced DQN algorithms, the action space was always discrete.

Agent Model

Inspired by the breakthroughs in DQN and DRQN, we suggest a modification to the last layers to handle pseudo-continuous action spaces, as required for the portfolio management task. The current implementation, termed the **Deep Soft Recurrent Q-Network** (DSRQN) relies on a fixed, implicit policy (i.e., exponential normalization or *softmax* (McCullagh, 1984)) while the action-value function q is adaptively fitted.

A neural network architecture as in Figure 6.5 is used to estimate the action-value function. The two 2D-convolution (2D-CONV) layers are followed by

a max-pooling (MAX) layer, which aim to extract non-linear features from the raw historic log returns (LeCun and Bengio, 1995). The feature map is then fed to the gated recurrent unit (GRU), which is the state manager (see Section 5.3.2), responsible for reconstructing a meaningful agent state from a partially observable environment. The generated agent state, s_t , is then regressed along with the past action, a_t (i.e., current portfolio vector), in order to produce the action-value function estimates. Those estimates are used with the realized reward r_{t+1} to calculate the TD error δ_{t+1} (6.11) and train the DSRQN as in Algorithm 4. The action-values estimates $\hat{q}_{t+1}$ are passed to a **softmax** layer, which produces the agent action a_{t+1} . We select the softmax function because it provides the favourable property of forcing all the components (i.e., portfolio weights) to sum to unity (see Section 2.1). Analytically the actions are given by:

$$\forall i \in \{1, 2, \dots, M\} : \quad a_i = \frac{e^{a_i}}{\sum_{j=1}^M e^{a_j}} \implies \sum_{i=1}^M a_i = 1 \quad (6.14)$$

Note that the last layer (i.e, softmax) is not trainable (Goodfellow *et al.*, 2016), which means that it can be replaced by any function (i.e., deterministic or stochastic), even by a quadratic programming step, since differentiability is not required. For this experiment we did not consider more advanced policies, but anything is accepting as long as constraint (2.1) is satisfied. Moreover, comparing our implementation with the original DQN (Mnih *et al.*, 2015), no experience replay is performed, in order to avoid resetting the GRU hidden state for each batch, which will lead to an unused latent state, and hence poor state manager.

Evaluation

Figure 6.7 illustrates the results obtained on a small scale experiment with 12 assets from S&P 500 market using the DSRQN. The agent is trained on historic data between 2000-2005 for 5000 episodes, and tested on 2005-2018. The performance evaluation of the agent for different episodes e is highlighted. For $e = 1$ and $e = 10$, the agent acted completely randomly, leading to poor performance. For $e = 100$, the agent did not beat the market (i.e., S&P 500) but it learned how to follow it (hence high correlation) and to yield profit out of it (since the market is increasing in this case). Finally, for $e = 1000$, we note that the the agent is both profitable and less correlated with the market, compared to previous model-based agents (i.e., VAR and GRU-RNN),

Algorithm 4: Deep Soft Recurrent Q-Learning.

inputs : trading universe of M -assets
initial portfolio vector $w_1 = a_0$
initial asset prices $p_0 = o_0$
objective function $\mathcal{J}$
initial agent weights θ_0
output: optimal agent parameters θ_*

```
1 repeat
2   for  $t = 1, 2, \dots T$  do
3     observe tuple  $\langle o_t, r_t \rangle$ 
4     calculate TD error  $\delta_{t+1}$  // (6.11)
5     calculate gradients  $\nabla_{\theta_i} \mathcal{L}(\theta_i) = \delta_{t+1} \nabla_{\theta_i} q(s, a; \theta)$  // BPTT
6     update agent parameters  $\theta$ 
7       using adaptive gradient optimizers // ADAM
8     get estimate of value function  $q_t \approx \text{NN}(\vec{p}_{t-T \rightarrow t})$  // (6.11)
9     take action  $a_t \text{softmax}(q_t)$  // portfolio rebalance
10  end
11 until convergence
12 set  $\theta_* \leftarrow \theta$ 
```

which were highly impacted by 2008 market crash, while DSRQN was almost unaffected (e.g., 85 % drowdown).

In Figure 6.6, we illustrate the out-of-sample cumulative return of the DSRQN agent, which flattens for $e \gtrsim 1000$, and hence the neural network optimizer converges to a (local) minimum, thus we terminate training.

Figure 6.6 also highlights the importance of the optimization algorithm used in training the neural network, since ADAM (i.e., the adaptive optimizer) did not only converge faster than Stochastic Gradient Descent (SGD), but it also found a better (local) minimum.

Weaknesses

Despite the improved performance of DSRQN compared to the model-based agents, its architecture has severe weaknesses.

Firstly, the selection of the policy (e.g., softmax layer) is a manual process that can be only verified via empirical means, for example, cross-validation. This complicates the training process, without guaranteeing any global optimality of the selected policy.

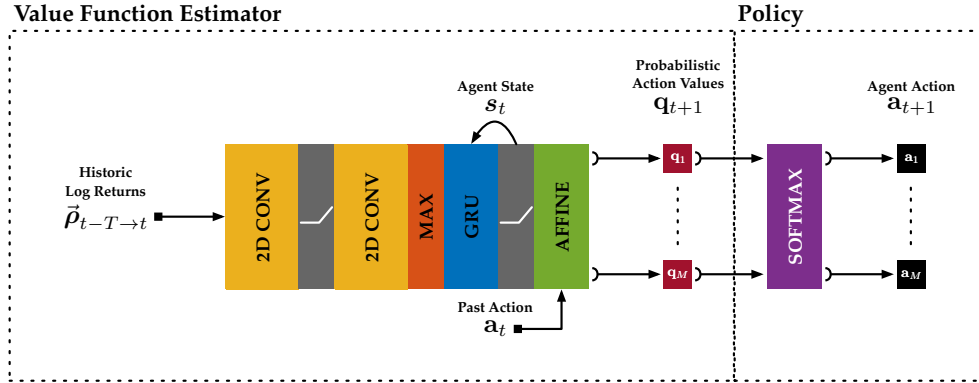


Figure 6.5: Deep Soft Recurrent Q-Network (DSRQN) architecture. The historic log returns $\vec{p}_{t-T \rightarrow t} \in \mathbb{R}^{M \times T}$ are passed through two 2D-convolution layers, which generate a feature map, which is, in turn, processed by the GRU state manager. The agent state produced is combined (via matrix flattening and vector concatenation) with the past action (i.e., current portfolio positions) to estimate action-values $q_1, q_2, \dots, q_M$. The action values are used both for calculating the TD error (6.11), showing up in the gradient calculation, as well as for determining the agents actions, after passed through a softmax activation layer.

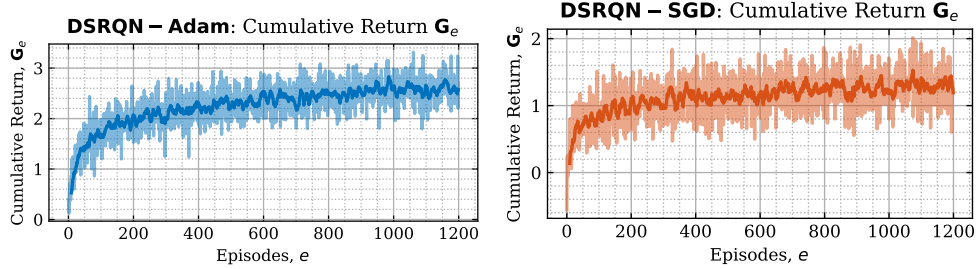


Figure 6.6: Out-of-sample cumulative returns per episode during training phase for DSRQN. Performance improvement saturates after $e \gtrsim 1000$. (Left) Adaptive Neural network optimization algorithm **ADAM** (Kingma and Ba, 2014). (Right) Neural network optimized with Stochastic Gradient Descent (SGD) (Mandic, 2004).

Expected Properties 6.4 (End-to-End Differentiable Architecture) *Agent policy should be part of the trainable architecture so that it adapts to (locally) optimal strategy via gradient optimization during training.*

Secondly, DSRQN is a Many-Input-Many-Output (MIMO) model, whose number of parameters grows polynomially as a function of the universe size (i.e., number of assets M), and hence its training complexity. Moreover,

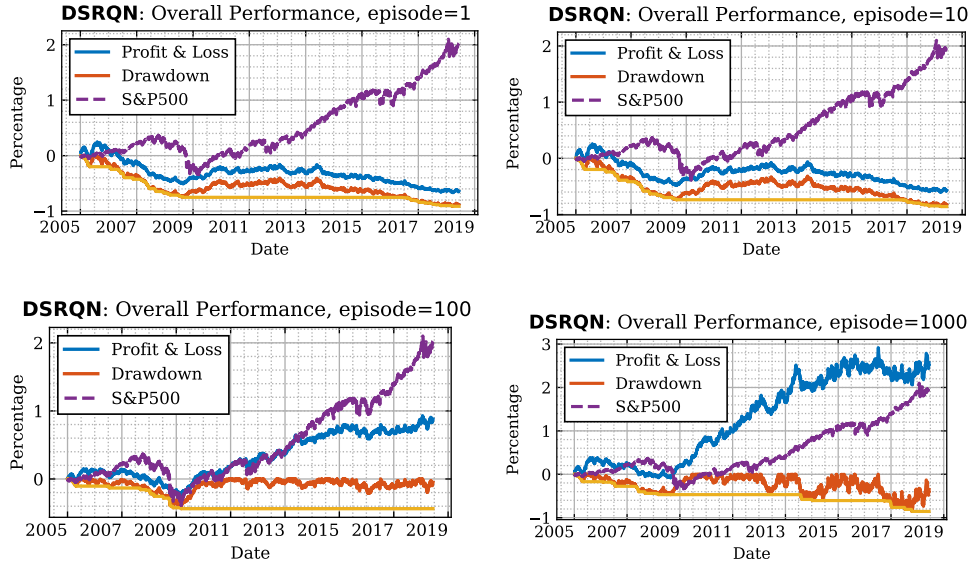


Figure 6.7: Deep Soft Recurrent Q-Network (DSRQN) model-free reinforcement learning agent on a 12-asset universe, trained on historic data between 2000-2005 and tested onward, for different number of episodes $e = \{1, 10, 100, 1000\}$. Visualization of cumulative rewards and (maximum) drawdown of the learned strategy, against the S&P 500 index (traded as SPY).

under this setting, the learned strategy is universe-specific, which means that the same trained network does not generalize to other universes. It even fails to work on permutations of the original universe; for example, if we interchange the order assets in the processed observation $\hat{s}_t$ after training, then DSRQN will break down.

Expected Properties 6.5 (Linear Scaling) *Model should scale linearly (i.e., computation and memory) with respect to the universe size.*

Expected Properties 6.6 (Universal Architecture) *Model should be universe-agnostic and replicate its strategy regardless the underlying assets.*

6.2.2 Monte-Carlo Policy Gradient (REINFORCE)

In order to address the first weakness of the DSRQN (i.e., manual selection of policy), we consider policy gradient algorithms, which directly address the learning of an agent policy, without intermediate action-value approximations, resulting in an end-to-end differentiable model.

Policy Gradient Theorem

In Section 4.2.2 we defined policy of an agent, π , as:

$$\pi : \mathcal{S} \rightarrow \mathcal{A} \quad (4.14)$$

In policy gradient algorithms, we parametrize the policy with parameters θ as π_θ and optimize them according to a long-term objective function $\mathcal{J}$, such as average reward per time-step, given by:

$$\mathcal{J}(\theta) \triangleq \sum_{s \in \mathcal{S}} \mathcal{P}^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \mathcal{R}_s^a \quad (6.15)$$

Note, that any differentiable parametrization of the policy is valid (e.g., neural network, linear model). Moreover, the freedom of choosing the reward generating function, $\mathcal{R}_s^a$, is still available.

In order to optimize the parameters, θ , the gradient, $\nabla_\theta \mathcal{J}$, should be calculated at each iteration. Firstly, we consider an one-step Markov Decision Process:

$$\mathcal{J}(\theta) = \mathbb{E}_{\pi_\theta}[\mathcal{R}_s^a] \quad (6.15)$$

$$= \sum_{s \in \mathcal{S}} \mathcal{P}^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \mathcal{R}_s^a \quad (6.16)$$

$$\begin{aligned} \nabla_\theta \mathcal{J}(\theta) &= \sum_{s \in \mathcal{S}} \mathcal{P}^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(s, a) \mathcal{R}_s^a \\ &= \sum_{s \in \mathcal{S}} \mathcal{P}^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \frac{\nabla_\theta \pi_\theta(s, a)}{\pi_\theta(s, a)} \mathcal{R}_s^a \\ &= \sum_{s \in \mathcal{S}} \mathcal{P}^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \nabla_\theta \log[\pi_\theta(s, a)] \mathcal{R}_s^a \\ &= \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log[\pi_\theta(s, a)] \mathcal{R}_s^a \right] \end{aligned} \quad (6.17)$$

The policy gradient calculation is extended to multi-step MDPs by replacing the instantaneous reward $\mathcal{R}_s^a$ with the long-term (action) value $q^\pi(s, a)$.

Theorem 6.2 (Policy Gradient Theorem) *For any differentiable policy $\pi_\theta(s, a)$ and for $\mathcal{J}$ the average discounted future rewards per step, the policy gradient is:*

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log[\pi_\theta(s, a)] q^\pi(s, a) \right] \quad (6.18)$$

where the proof is provided by Sutton *et al.* (2000b). The theorem applies to continuous settings (i.e., Infinite MDPs) (Sutton and Barto, 1998), where the summations are replaced by integrals.

Related Work

Policy gradient methods have gained momentum the past years due to their better convergence policies (Sutton and Barto, 1998), their effective in high-dimensional or continuous action spaces and natural fit to stochastic environments (Silver, 2015e). Apart from their extensive application in robotics (Smart and Kaelbling, 2002; Kohl and Stone, 2004; Kober and Peters, 2009), policy gradient methods have been used also in financial markets. Necchi (2016) develops a general framework for policy gradient agents to be trained to solve the asset allocation task, but only successful back-tests for synthetic market data are provided.

Agent Model

From equation (6.18), we note two challenges:

1. Calculation of an expectation over the (stochastic) policy, $\mathbb{E}_{\pi_\theta}$, leading to integration over unknown quantities;
2. Estimation of the unknown action-value, $q^\pi(s, a)$

The simplest, successful algorithm to address both of the challenges is **Monte-Carlo Policy Gradient**, also known as **REINFORCE**. In particular, different trajectories are generated following policy π_θ which are then used to estimate the expectation⁵ and the discounted future rewards, which are an unbiased estimate of $q^\pi(s, a)$. Hence we obtain Monte-Carlo estimates:

$$q^\pi(s, a) \approx G_t \triangleq \sum_{i=1}^t r_i \quad (6.19)$$

$$\mathbb{E}_{\pi_\theta} \left[\nabla_{\theta} \log[\pi_\theta(s, a)] q^\pi(s, a) \right] \approx \frac{1}{T} \left[\nabla_{\theta} \log[\pi_\theta(s, a)] \sum_{i=1}^T G_i \right] \quad (6.20)$$

where the gradient of the log term $\nabla_{\theta} \log[\pi_\theta(s, a)]$, is obtained by Backpropagation Through Time (Werbos, 1990).

We choose to parametrize the policy using a neural network architecture, illustrated in Figure 6.8. The configuration looks very similar to the DSRQN,

⁵The empirical mean is an unbiased estimate of expected value (Mandic, 2018b).

but the important difference is in the last two layers, where the REINFORCE network does not estimate the state action-values, but directly the agent actions. Algorithm 5 describes the steps for training a REINFORCE agent.

Algorithm 5: Model-Carlo Policy Gradient (REINFORCE).

inputs : trading universe of M -assets
 initial portfolio vector $w_1 = a_0$
 initial asset prices $p_0 = o_0$
 objective function $\mathcal{J}$
 initial agent weights θ_0
output: optimal agent policy parameters θ_*

```

1 initialize buffers:  $G, \Delta\theta_c \leftarrow 0$  repeat
2   for  $t = 1, 2, \dots T$  do
3     observe tuple  $\langle o_t, r_t \rangle$ 
4     sample and take action:  $a_t \sim \pi_\theta(\cdot | s_t; \theta)$            // portfolio
       rebalance
5     cache rewards:  $G \leftarrow G + r_t$                            // (6.19)
6     cache log gradients:  $\Delta\theta_c \leftarrow \Delta\theta_c + \nabla_\theta \log[\pi_\theta(s, a)]G$  // (6.20)
7   end
8   update policy parameters  $\theta$  using buffered
9     Monte-Carlo estimates via adaptive optimization // (6.18),
       ADAM
10  empty buffers:  $G, \Delta\theta_c \leftarrow 0$ 
11 until convergence
12 set  $\theta_* \leftarrow \theta$ 

```

Evaluation

In Figure 6.10, we present the results from an experiment performed on a small universe comprising of 12 assets from S&P 500 market using the REINFORCE agent. The agent is trained on historic data between 2000-2005 for 5000 episodes, and tested on 2005-2018. Similar to the DSRQN, at early episodes the REINFORCE agent performs poorly, but it learns a profitable strategy after a few thousands of episodes ($e \approx 7500$). Note that almost 8 times more episodes are required to train the REINFORCE agent compared to the DSRQN, however, the latter performs a parameters update at every step, while the former only once every episode (i.e., approximately steps 1260 in an episodes). The performance of the REINFORCE agent is significantly improved, with total cumulative returns 325.9% and 63.5% maximum drawdown.

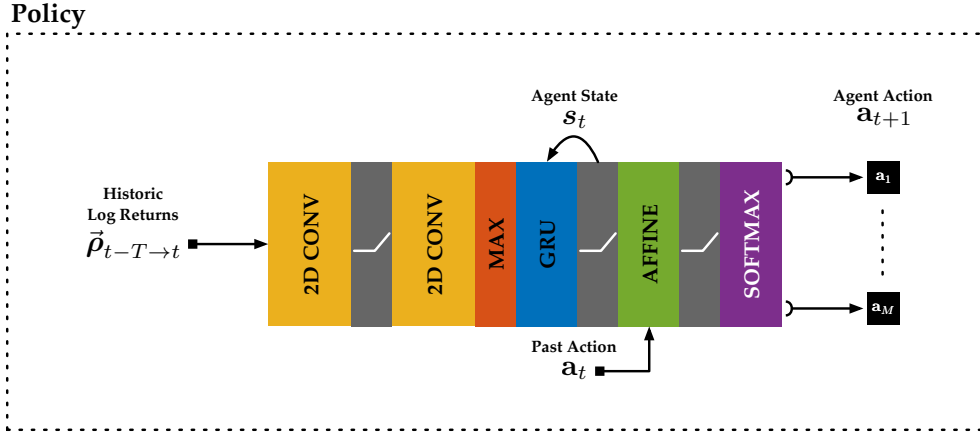


Figure 6.8: Monte-Carlo Policy Gradient (REINFORCE) architecture. The historic log returns $\vec{\rho}_{t-T \rightarrow t} \in \mathbb{R}^{M \times T}$ are passed through two 2D-convolution layers, which generate a feature map, which is, in turn, processed by the GRU state manager. The agent state produced and the past action (i.e., current portfolio positions) are non-linearly regressed and exponentially normalized by the affine and the softmax layer, respectively, to generate the agent actions.

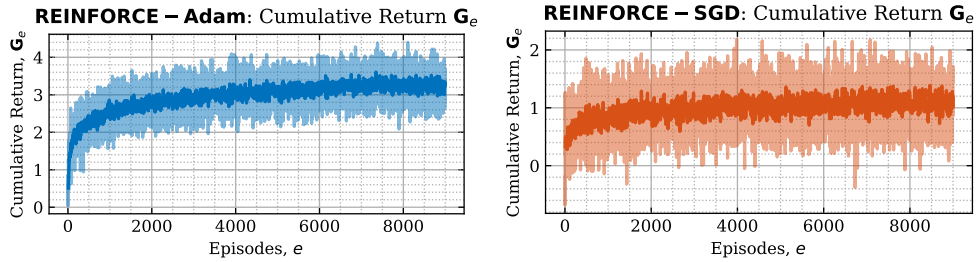


Figure 6.9: Out-of-sample cumulative returns per episode during training phase for REINFORCE. Performance improvement saturates after $e \gtrsim 7500$. (Left) Adaptive Neural network optimization algorithm **ADAM**. (Right) Neural network optimized with Stochastic Gradient Descent (SGD).

In Figure 6.6, we illustrate the out-of-sample cumulative return of the DSRQN agent, which flattens for $e \gtrsim 7500$, and hence the neural network optimizer converges to a (local) minimum, thus we terminate training.

Weaknesses

Policy gradient addressed only the end-to-end differentiability weakness of the DSRQN architecture, leading to significant improvements. Nonetheless, the polynomial scaling and universe-specific nature of the model are

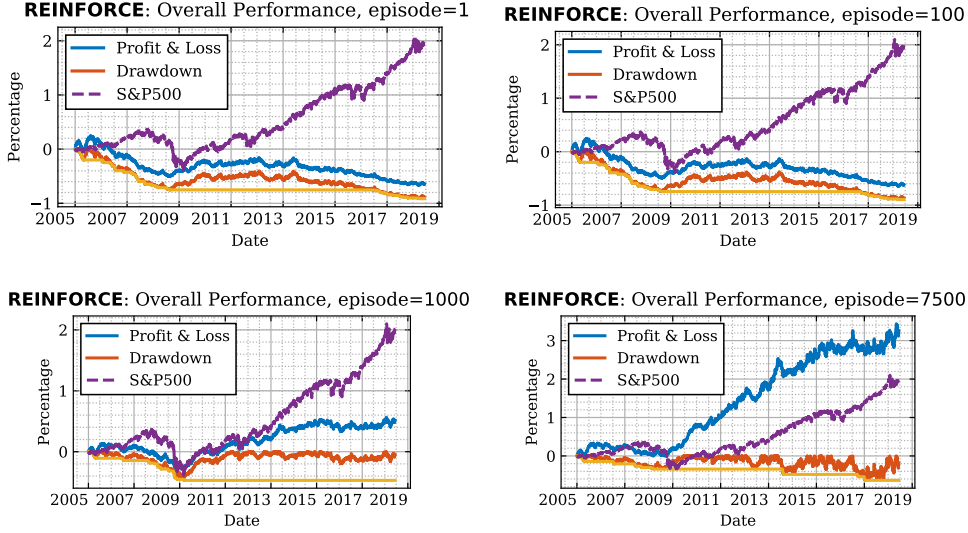


Figure 6.10: Model-Carlo Policy Gradient (REINFORCE) model-free reinforcement learning agent on a 12-asset universe, trained on historic data between 2000-2005 and tested onward, for different number of episodes $e = \{1, 100, 1000, 7500\}$. Visualization of cumulative rewards and (maximum) drawdown of the learned strategy, against the S&P 500 index (traded as SPY).

still restricting the applicability and generalization of the learned strategies. Moreover, the intractability of the policy gradient calculation given by (6.18) lead to the comprising solution of using Monte-Carlo estimates by running numerous simulations, leading to increased number of episodes required for convergence. Most importantly, the empirical estimation of the state action-value $q^{\pi_\theta}(s, a)$ in (6.19) has high-variance (see returns in Figure 6.9) (Sutton and Barto, 1998).

Expected Properties 6.7 (Low Variance Estimators) *Model should rely on low variance estimates.*

6.2.3 Mixture of Score Machines (MSM)

In this subsection, we introduce the **Mixture of Score Machines (MSM)** model with the aim to provide a universal model that reduces the agent model complexity and generalizes strategies across assets, regardless of the trained universe. These properties are obtained by virtue of principles of parameter sharing (Bengio *et al.*, 2003) and transfer learning (Pan and Yang, 2010a).

Related Work

Jiang *et al.* (2017) suggested a universal policy gradient architecture, the *Ensemble of Identical Independent Evaluators* (EIIE), which reduces significantly the model complexity, and hence enables larger-scale applications. Nonetheless, it operates only on independent (e.g., uncorrelated) time-series, which is a highly unrealistic assumption for real financial markets.

Agent Model

As a generalization to the universal model of Jiang *et al.* (2017), we introduce the **Score Machine** (SM), an estimator of statistical moments of stochastic multivariate time-series:

- A **First-Order Score Machine** SM(1) operates on univariate time-series, generating a score that summarizes the characteristics of the *location* parameters of the time-series (e.g., mode, median, mean). An M -components multivariate series will have $\binom{M}{1} = M$ first-order scores, one for each component;
- A **Second-Order Score Machine** SM(2) operates on bivariate time-series, generating a score that summarizes the characteristics of the *dispersion* parameters of the joint series (e.g., covariance, modal dispersion (Meucci, 2009)). An M -components multivariate series will have $\binom{M}{2} = \frac{M!}{2!(M-2)!}$ second-order scores, one for each distinct pair;
- An **N -Order Score Machine** SM(N) operates on N -component multivariate series and extracts information about the *N -order statistics* (i.e., statistical moments) of the joint series. An M -components multivariate series, for $M \geq N$, will have $\binom{M}{N} = \frac{M!}{N!(M-N)!}$ N -order scores, one for each distinct N combination of components.

Note that the extracted scores are *not* necessarily equal to the statistical (central) moments of the series, but a compressed and informative representation of the statistics of the time-series. The universality of the score machine is based on parameter sharing (Bengio *et al.*, 2003) across assets.

Figure 6.11 illustrates the first and second order score machines, where the transformations are approximated by neural networks. Higher order score machines can be used in order to captures higher-order moments.

By combining the score machines, we construct the **Mixture of Score Machines** (MSM), whose architecture is illustrated in Figure 6.12. We identify three main building blocks:

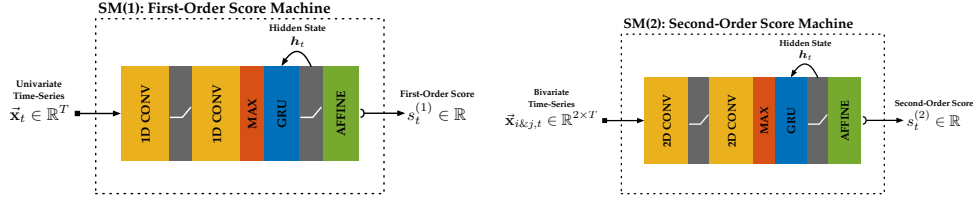


Figure 6.11: Score Machine (SM) neural network architecture. Convolutional layers followed by non-linearities (e.g., ReLU) and Max-Pooling (Giusti *et al.*, 2013) construct a feature map, which is selectively stored and filtered by the Gate Recurrent Unit (GRU) layer. Finally, a linear layer combines the GRU output components to a single scalar value, the score. (Left) First-Order Score Machine SM(1); given a univariate time-series $\vec{x}_t$ of T samples, it produces a scalar score value $s_t^{(1)}$. (Right) Second-Order Score Machine SM(2); given a bivariate time-series $\vec{x}_{i\&j,t}$, with components $\vec{x}_{i,t}$ and $\vec{x}_{j,t}$, of T samples each, it generates a scalar value $s_t^{(2)}$.

1. **SM(1)**: a first-order score machine that processes all single-asset log returns, generating the first-order scores;
2. **SM(2)**: a second-order score machine that processes all pairs of assets log-returns, generating the second-order scores;
3. **Mixture Network**: an aggregation mechanism that accesses the scores from SM(1) and SM(2) and infers the action-values.

We emphasize that there is only one SM for each order, shared across the network, hence during backpropagation the gradient of the loss function with respect to the parameters of each SM is given by the sum of all the paths that contribute to the loss (Goodfellow *et al.*, 2016) that pass through that particular SM. The mixture network, inspired by the Mixtures of Expert Networks by Jacobs *et al.* (1991), gathers all the extracted information from first and second order statistical moments and combines them with the past action (i.e., current portfolio vector) and the generated agent state (i.e., state manager hidden state) to determine the next optimal action.

Neural networks, and especially deep architectures, are very *data hungry* (Murphy, 2012), requiring thousands (or even millions) of data points to converge to meaningful strategies. Using daily market data (i.e., daily prices), almost 252 data points are only collected every year, which means that very few samples are available for training. Thanks to the parameter sharing, nonetheless, the SM networks are trained on orders of magnitude more data. For example, a 12-assets universe with 5 years history is given by the

dataset $\mathcal{D} \in \mathbb{R}^{12 \times 1260}$. The asset-specific agents (i.e., VAR, RNN, DSRQN, REINFORCE) have 1260 samples of a multivariate time series (i.e., with 12 components) from which they have to generalize. On the other hand, the MSM agent has $12 \cdot 1260 = 15120$ samples for training the SM(1) network and $\binom{12}{2} \cdot 1260 = 66 \cdot 1260 = 83160$ samples for training the SM(2) network.

The architecture is end-to-end differentiable and can be trained by back-propagating the policy gradient to the composite network. While the score machines (i.e., SM(1) and SM(2)) can be extended to any number of assets M without modification, by stacking more copies of the same machine with shared parameters, the *mixture network is universe-specific*. As a result, a different mixture network is required to be trained for different universes. Consider the case of an M -asset market, then the mixture network has the interface:

$$N_{\text{mixture-network}}^{\text{inputs}} = M + \binom{M}{2}, \quad N_{\text{mixture-network}}^{\text{outputs}} = M \quad (6.21)$$

Consequently, selecting a different number of assets would break the interface of the mixture network. Nonetheless, the score machines can be trained with different mixture networks hence when a new universe of assets is given, we freeze the training of the score machines and train only the mixture network. This operation is *cheap* since the mixture network comprises of only a small fraction of the total number of trainable parameters of the MSM.

Practically, the score machines are trained on large historic datasets and kept fixed, while transfer learning is performed on the mixture network. Therefore, the score machines can be viewed as rich, universal feature extractors, while the mixture network is the small (i.e., in size and capacity) mechanism that enables mapping from the abstract space of scores to the feasible action space, capable of preserving asset-specific information as well.

Evaluation

Figure 6.14 shows the results from an experiment performed on a small universe comprising of 12 assets from S&P 500 market using the MSM agent. The agent is trained on historic data between 2000-2005 for 10000 episodes, and tested on 2005-2018. Conforming with our analysis, the agent underperformed early in the training, but after 9000 episodes it became profitable and its performance saturated after 10000 episodes, with total cumulative returns of 283.9% and 68.5% maximum drawdown. It scored slightly worse

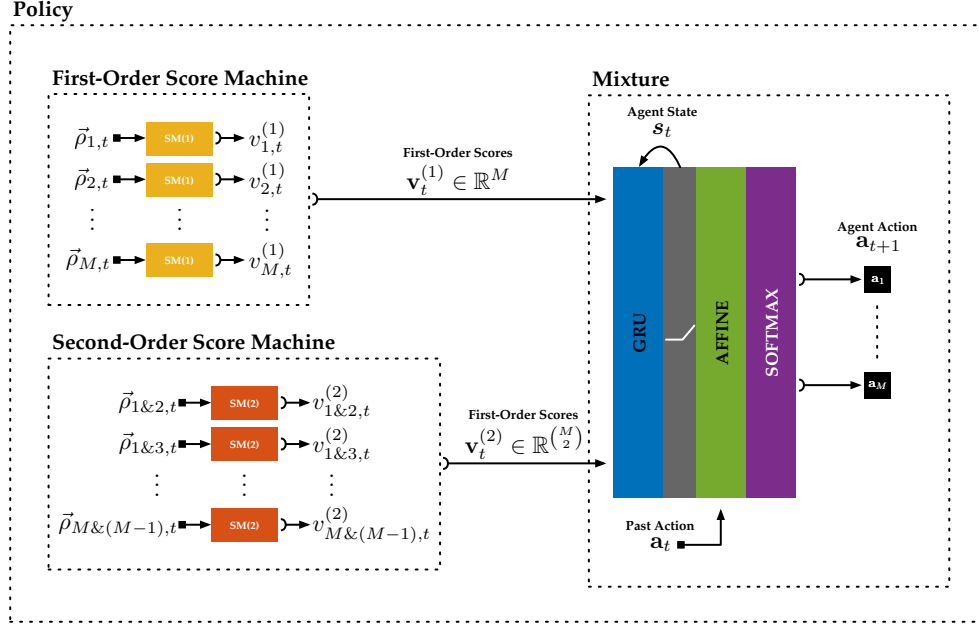


Figure 6.12: Mixture of Score Machines (MSM) architecture. The historic log returns $\vec{\rho}_{t-T \rightarrow t} \in \mathbb{R}^{M \times T}$ processes by the score machines SM(1) and SM(2), which assign scores to each asset ($v_t^{(1)}$) and pair of assets ($v_t^{(2)}$), respectively. The scores concatenated and passed to the mixture network, which combines them with the past action (i.e., current portfolio vector) and the generated agent state (i.e., state manager hidden state) to determine the next optimal action.

than the REINFORCE agent, but in Part III it is shown that in a larger scale experiments the MSM is both more effective and efficient (i.e., computationally and memory-wise).

Weaknesses

The Mixture of Score Machines (MSM) is an architecture that addresses the weaknesses of all the aforementioned approaches (i.e., VAR, LSTM, DSRQN and REINFORCE). However, it could be improved by incorporating the expected properties 6.8 and 6.9:

Expected Properties 6.8 (Short Sales) *Model should output negative portfolio weights, corresponding to short selling, as well.*

Expected Properties 6.9 (Optimality Guarantees) *Explore possible re-interpretations of the framework, which would allow proof of optimality (if applicable).*

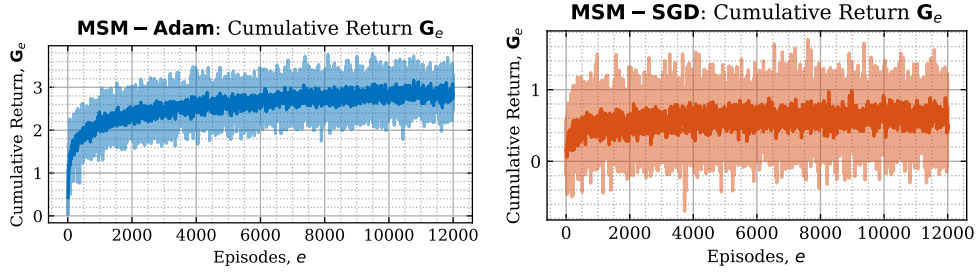


Figure 6.13: Out-of-sample cumulative returns per episode during training phase for MSM. Performance improvement saturates after $e \gtrsim 10000$. (Left) Adaptive Neural network optimization algorithm ADAM. (Right) Neural network optimized with Stochastic Gradient Descent (SGD).

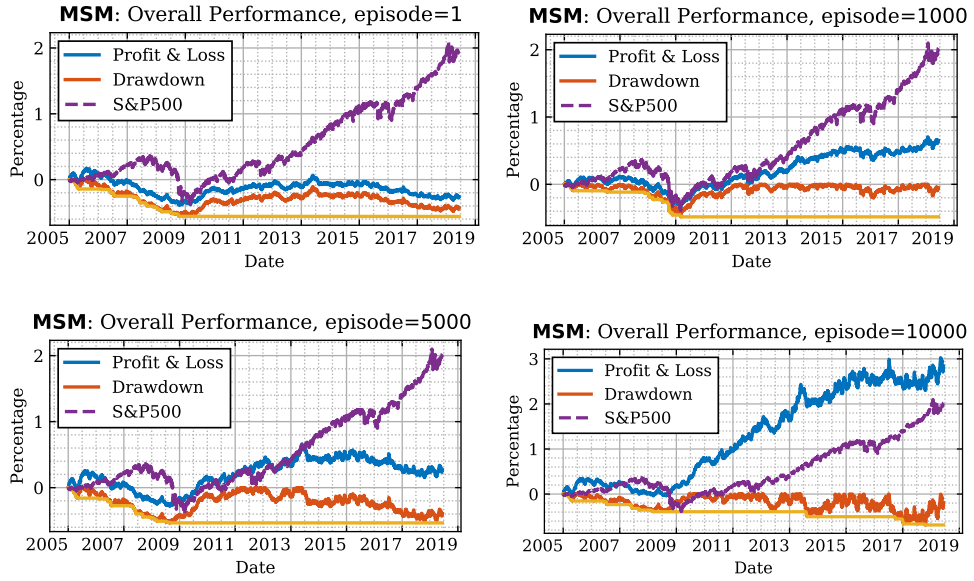


Figure 6.14: Mixture of Score Machines (MSM) model-free reinforcement learning agent on a 12-asset universe, trained on historic data between 2000-2005 and tested onward, for different number of episodes $e = \{1, 1000, 5000, 10000\}$. Visualization of cumulative rewards and (maximum) drawdown of the learned strategy, against the S&P 500 index (traded as SPY).

Trading Agents Comparison Matrix: 12-assets of S&P 500						
		Model-Based		Model-Free		
		VAR	RNN	DSRQN	REINFORCE	MSM
Metrics	Cumulative Returns (%)	185.1	221.0	256.7	325.9	283.9
	Sharpe Ratio (SNR)	1.53	1.62	2.40	3.02	2.72
	Max Drawdown (%)	179.6	198.4	85.6	63.5	68.5
Expected Properties	Non-Stationary Dynamics	×	✓	✓	✓	✓
	Long-Term Memory	×	✓	✓	✓	✓
	Non-Linear Model	×	✓	✓	✓	✓
	End-to-End	×	×	×	✓	✓
	Linear Scaling	×	×	×	×	✓
	Universality	×	×	×	×	✓
	Low Variance Estimators	×	×	×	×	×
	Short Sales	✓	✓	×	×	×

Table 6.1: Comprehensive comparison of evaluation metrics and their weaknesses (i.e., expected properties) of trading algorithms addressed in this chapter. Model-based agents (i.e., VAR and RNN) underperform, while the best performing agent is the REINFORCE. As desired, the MSM agent scores also well above the index (i.e., baseline) and satisfies most wanted properties.

Chapter 7

Pre-Training

In Chapter 6, model-based and model-free reinforcement learning agents were introduced, which address the asset allocation task. It was demonstrated (see comparison table 6.1) that model-based (i.e., VAR and RNN) and value-based model-free agents (i.e., DSRQN) are outperformed by the policy gradient agents (i.e., REINFORCE and MSM). However, policy gradient algorithms usually converge to local optima (Sutton and Barto, 1998).

Inspired by the approach taken by the authors of the original DeepMind AlphaGo paper (Silver and Hassabis, 2016), the local optimality of policy gradient agents is addressed via pre-training the policies in order to replicate the strategies of baseline models. It is shown that any one-step optimization method, discussed in Chapter 3 that reduces to a quadratic program, can be reproduced by the policy gradient networks (i.e., REINFORCE and MSM), when the networks are trained to approximate the quadratic program solution.

Because of the highly non-convex policy search space (Szepesvári, 2010), the randomly initialised agents (i.e., agnostic agents) tend to either get stuck to vastly sub-optimal local minima or need a lot more episodes and samples to converge to meaningful strategies. Therefore, the limited number of available samples (e.g., 10 years of market data is equivalent to approximately 2500 samples), motivates pre-training, which is expected to improve convergence speed and performance, assuming that the baseline model is sub-optimal but a proxy to the optimal strategy. Moreover, the pre-trained models can be viewed as *priors*¹ to the policies and episodic training with reinforcement learning steers them to the updated strategies, in a data-driven and data-efficient manner.

¹As in the context of Bayesian Inference.

In Section 7.1, a few candidate baseline models are introduced, including the algorithm for generating synthetic (supervised) datasets according to these baseline models as well as the corresponding steps for pre-training the agents using the generated data. In addition, in Section 7.2 the pre-training process is assessed as well as the performance gain compared to randomly initialized agents is quantified.

7.1 Baseline Models

In Chapter 3, the traditional one-step (i.e., static) portfolio optimization methods were described, derived from the Markowitz model (Markowitz, 1952). Despite the assumptions about covariance stationarity (i.e., time-invariance of first and second statistical moments) and myopic approach of those methods, they usually form the basis of other more complicated and effective strategies. As a result, it is attempted to replicate those strategies with the REINFORCE (see subsection 6.2.2) and the Mixture of Score Machines (MSM) (see subsection 6.2.3) agents. In both cases, the architecture of the agents (i.e., underlying neural networks) are treated as black boxes, represented by a set of parameters, which thanks to their end-to-end differentiability, can be trained via backpropagation. Figure 7.1 summarizes the pipeline used to (pre-)train the policy gradient agents.

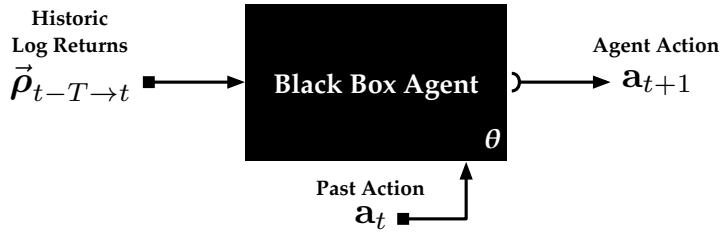


Figure 7.1: Interfacing with policy gradient agents as black boxes, with inputs (1) historic log returns $\vec{\rho}_{t-T \rightarrow t}$ and (2) past action (i.e., current portfolio vector) $\mathbf{a}_t$ and output the next agent actions $\mathbf{a}_{t+1}$. The black-box is parametrized by θ which can be updated and optimized.

7.1.1 Quadratic Programming with Transaction Costs

The one-step optimal portfolio for given commission rates (i.e., transaction costs coefficient β) and hyperparameters (e.g., risk-aversion coefficient) is obtained, by solving the optimization task in (3.13) or (3.14) via quadratic programming. The Sharpe Ratio with transaction costs objective function is

selected as the baseline for pre-training, since it has no hyperparameter to tune and inherently balances profit-risk trade-off.

Without being explicitly given the mean vector μ , the covariance matrix Σ and the transaction coefficient β , the black-box agents should be able to solve the optimization task (3.14), or equivalently:

$$\begin{aligned} \underset{\mathbf{a}_{t+1} \in \mathcal{A}}{\text{maximize}} \quad & \frac{\mathbf{a}_{t+1}^T \mu - \mathbf{1}_M^T \beta \|\mathbf{a}_t - \mathbf{a}_{t+1}\|_1}{\sqrt{\mathbf{a}_{t+1}^T \Sigma \mathbf{a}_{t+1}}} \\ \text{and} \quad & \mathbf{1}_M^T \mathbf{a}_{t+1} = 1 \\ \text{and} \quad & \mathbf{a}_{t+1} \succeq 0 \end{aligned}$$

7.1.2 Data Generation

Since there is a closed form formula that connects the black-box agents' inputs $\vec{\rho}_{t-T \rightarrow t}$ and $\mathbf{a}_t$ with the terms in the optimization problem (3.14), N supervised pairs $\{(\mathbf{X}_i, \mathbf{y}_i)\}_{i=1}^N$ are generated by solving the optimization for N distinct cases, such that:

$$\mathbf{X}_i = [\vec{\rho}_{t_i-T \rightarrow t_i}, \mathbf{a}_{t_i}] \quad (7.1)$$

$$\mathbf{y}_i = \mathbf{a}_{t_i+1} \quad (7.2)$$

Interestingly, myriad of examples (i.e., $\mathbf{X}_i, \mathbf{y}_i$ pairs) can be produced to enrich the dataset and allow convergence. This is a very rare situation where the generating process of the data is known and can be used to produce valid samples, which respect the dynamics of the target model. The data generation process is given in algorithm 6.

7.2 Model Evaluation

The parameters of the black-box agents are steered in the gradient direction that minimizes the **Mean Square Error** between the predicted portfolio weights, $\hat{\mathbf{y}}_{t_i}$, and the baseline model target portfolio weights, $\mathbf{y}_{t_i}$. An L^2 -norm weight decaying, **regularization**, term is also considered to avoid overfitting, obtaining the loss function:

$$\mathcal{L}(\theta) = \|\mathbf{y}_{t_i} - \hat{\mathbf{y}}_{t_i;\theta}\|_2^2 + \lambda \|\theta\|_2^2 \quad (7.3)$$

Algorithm 6: Pre-training supervised dataset generation.

inputs : number of pairs to generate N
 number of assets in portfolio M
 look back window size T
 transaction costs coefficient β

output: dataset $\{(X_i, y_i)\}_{i=1}^N$

```

1 for  $i = 1, 2, \dots, N$  do
2   sample valid random initial portfolio vector  $w_{t_i}$ 
3   sample random lower triangular matrix  $L \in \mathbb{R}^{M \times M}$  // Cholesky
   decomposition
4   sample randomly distributed log returns:  $\vec{\rho}_{t_i-T \rightarrow t_i} \sim \mathcal{N}(\mathbf{1}, LL^T)$ 
5   calculate empirical mean vector of log returns:  $\mu = \mathbb{E}[\vec{\rho}_{t_i-T \rightarrow t_i}]$ 
6   calculate empirical covariance matrix of log returns:
    $\Sigma = \text{Cov}[\vec{\rho}_{t_i-T \rightarrow t_i}]$ 
7   determine  $a_{t_i+1}$  by solving quadratic program (3.14)
8   set  $X_i = [\vec{\rho}_{t_i-T \rightarrow t_i}, a_{t_i}]$  and  $y_i = a_{t_i+1}$ 
9 end

```

The parameters are adaptively optimized by Adam (Kingma and Ba, 2014), while the network parameters gradients are obtained via Backpropagation Through Time (Werbos, 1990).

7.2.1 Convergence to Quadratic Programming

Figure 7.2 depicts the the learning curves, in-sample and out-of sample, of the supervised learning training process. Both the REINFORCE and the MSM converge after ≈ 400 epochs (i.e., iterations). The

7.2.2 Performance Gain

As suggested by Figure 7.3, the pre-training improves the cumulative returns and Sharpe Ratio of the policy gradient agents up to 21.02% and 13.61%, respectively.

7. PRE-TRAINING

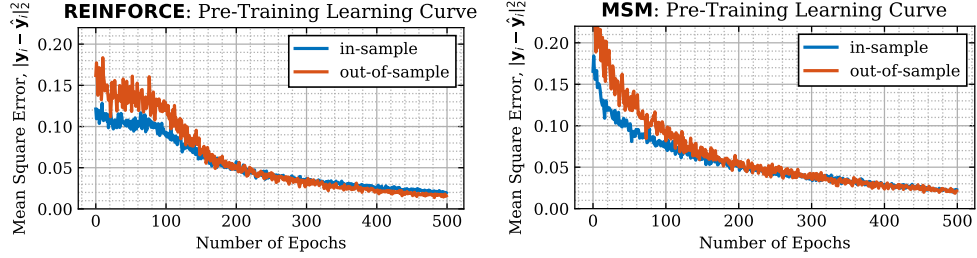


Figure 7.2: Mean square error (MSE) of Monte-Carlo Policy Gradient (REINFORCE) and Mixture of Score Machines (MSM) during pre-training. After ≈ 150 epochs the gap between the training (in-sample) and the testing (out-of-sample) errors is eliminated and error curve plateaus after ≈ 400 epochs, when training is terminated.

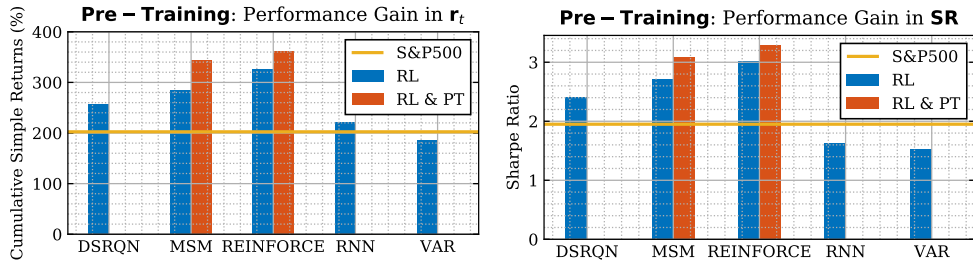


Figure 7.3: Performance evaluation of trading with reinforcement learning (RL) and reinforcement learning and pre-training (RL & PT). The Mixture of Score Machines (MSM) improves cumulative returns by 21.02% and Sharpe Ratio by 13.61%. The model-based (i.e., RNN and VAR) and the model-free value-based (i.e., DSRQN) agents are *not* end-to-end differentiable and hence cannot be pre-trained.

Part III

Experiments

Synthetic Data

It has been shown that the trading agents of Chapter 6, and especially REINFORCE and MSM, outperform the market index (i.e., S&P500) when tested in a small universe of 12-assets, see Table 6.1. For rigour, the validity and effectiveness of the developed reinforcement agents is investigated via a series of experiments on:

- **Deterministic** series, including sine, sawtooth and chirp waves, as in Section 8.1;
- **Simulated** series, using data surrogate methods, such as AAFT, as in Section 8.2.

As expected, it is demonstrated that model-based agents (i.e., VAR and RNN) excel in deterministic environments. This is attributed to the fact that given enough capacity they have the predictive power to accurately forecast the future values, based on which they can act optimally via planning.

On the other hand, on simulated (i.e., surrogate) time-series, it is shown that model-free agents score higher, especially after the pre-training process of Chapter 7, which contributes to up to 21% improvement in Sharpe Ratio and up to 40% reduction in the number of episodic runs.

8.1 Deterministic Processes

To begin with, via interaction with the environment (i.e., paper trading), the agents construct either an explicit (i.e., model-based reinforcement learning) or implicit model (i.e., model-free reinforcement learning) of the environment. In Section 6.1, It has been demonstrated that explicit modelling of financial time series is very challenging due to the stochasticity of the in-

volved time-series, and, as a result, model-based methods underperform. On the other hand, should the market series were sufficiently predictable, these methods would be expected to optimally allocate assets of the portfolio via dynamic programming and planning. In this section, we investigate the correctness of this hypothesis by generating a universe of deterministic time-series.

8.1.1 Sinusoidal Waves

A set of 100 sinusoidal waves of constant parameters (i.e., amplitude, circular frequency and initial phase) is generated, while example series are provided in Figure 8.1. Note the dominant performance of the model-based recurrent neural network (RNN) agent, which exploits its accurate predictions of future realizations and scores over three times better than the best-scoring model-free agent, the Mixture of Score Machines (MSM).

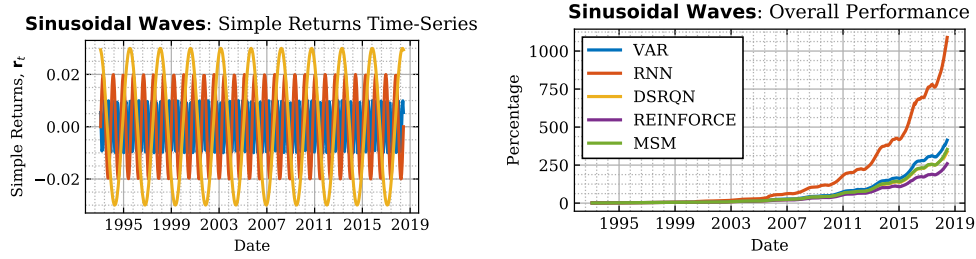


Figure 8.1: Synthetic universe of deterministic sinusoidal waves. (Left) Example series from universe. (Right) Cumulative returns of reinforcement learning trading agents.

For illustration purposes and in order to gain a finer insight into the learned trading strategies, a universe of only two sinusoids is generated the RNN agent is trained on binary trading the two assets; at each time step the agent puts all its budget on a single asset. As shown in Figure 8.2, the RNN agent learns the theoretically optimal strategy¹:

$$w_t = \begin{cases} w_{i,t} = 1, & \text{if } i = \operatorname{argmax}\{r_t\} \\ w_{i,t} = 0, & \text{otherwise} \end{cases} \quad (8.1)$$

or equivalently, the returns of the constructed portfolio is the max of the single asset returns at each time step.

¹Note that transaction costs are not considered in this experiment, in which case we would expect a time-shifted version of the current strategy so that it offsets the fees.

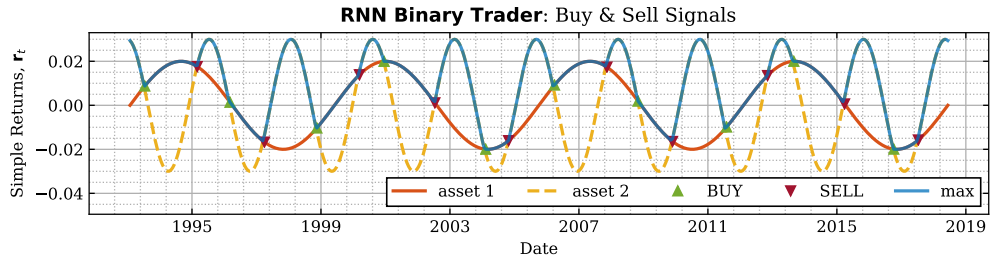


Figure 8.2: Recurrent neural network (RNN) model-based reinforcement learning agent trained on binary trading between two sinusoidal waves. The triangle trading signals (i.e., BUY or SELL) refer to *asset 1* (i.e., red), while opposite actions are taken for *asset 2*, but not illustrated.

8.1.2 Sawtooth Waves

A set of 100 deterministic sawtooth waves is generated next and examples are illustrated in Figure 8.3. Similar to the sinusoidal waves universe, the RNN agent outperforms the rest of the agents. Interestingly, it can be observed in the cumulative returns time series, right Figure 8.3, that all strategies have a low-frequency component, which corresponds to the highest amplitude sawtooth wave (i.e., yellow).

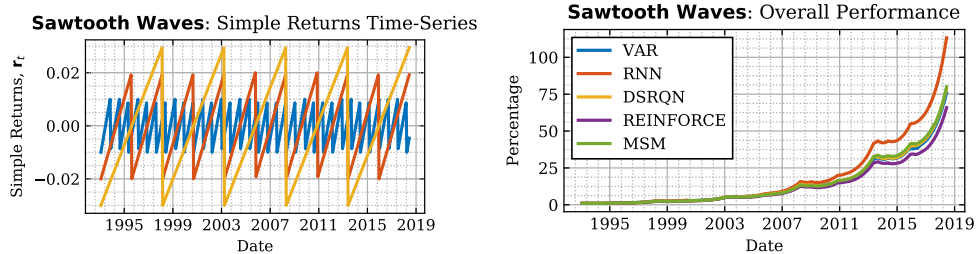


Figure 8.3: Synthetic universe of deterministic sawtooth waves. (Left) Example series from universe. (Right) Cumulative returns of reinforcement learning trading agents.

8.1.3 Chirp Waves

Last but not least, the experiment is repeated with a set of 100 deterministic chirp waves (i.e., sinusoidal wave with linearly modulated frequency). Three example series are plotted in 8.4, along with the cumulative returns of each trading agent. Note that the RNN agent is only 8.28% better than the second, the MSM, agent, compared to the $> 300\%$ marginal benefit in case of the sinusoidal waves. This is explained by the imperfect predictions of the RNN

due to the increased difficulty to learn the chirp signals.

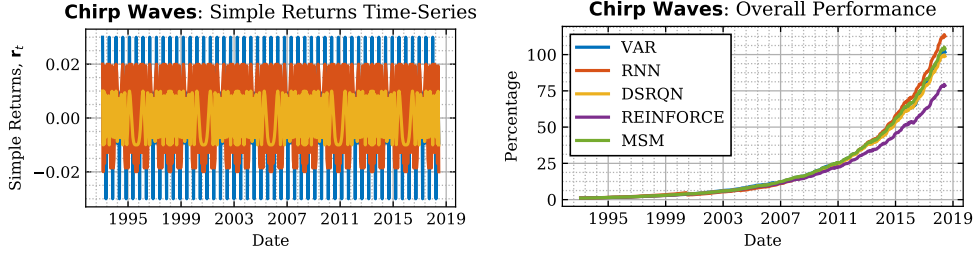


Figure 8.4: Synthetic universe of deterministic chirp waves. (Left) Example series from universe. (Right) Cumulative returns of reinforcement learning trading agents.

Remark 8.1 Overall, in a deterministic financial market, all trading agents learn profitable strategies and solve the asset allocation task. As expected, model-based agents, and especially the RNN, are significantly outperforming in case of well-behaving, easy-to-model and deterministic series (e.g., sinusoidal, sawtooth). On the other hand, in more complicated settings (e.g., chirp waves universe) the model-free agents perform almost as good as model-based agents.

8.2 Simulated Data

Having asserted the successfulness of reinforcement learning trading agents in deterministic universes, their effectiveness is challenged in stochastic universes, in this section. Instead of randomly selecting families of stochastic processes and corresponding parameters for them, real market data is used to learn the parameters of candidate generating processes that explain the data. The purpose of this approach is two-fold:

1. There is **no** need for **hyperparameter tuning**;
2. The training dataset is expanded, via **data augmentation**, giving the opportunity to the agents to gain more experience and further explore the joint state-action space.

It is worth highlighting that data augmentation improves overall performance, especially when strategies learned in the simulated environment are transferred and polished on real market data, via **Transfer Learning** (Pan and Yang, 2010b).

8.2.1 Amplitude Adjusted Fourier Transform (AAFT)

The simulated universe is generated using surrogates with random Fourier phases (Raeth and Monetti, 2009). In particular the **Amplitude Adjusted Fourier Transform** (AAFT) method (Prichard and Theiler, 1994) is used, explained in Algorithm 7. Given a real univariate time-series, the AAFT algorithm operates in Fourier (i.e., frequency) domain, where it preserves the amplitude spectrum of the series, but randomizes the phase, leading to a new realized signal.

AAFT can be explained by the **Wiener–Khinchin–Einstein Theorem** (Cohen, 1998), which states that the autocorrelation function of a wide-sense-stationary random process has a spectral decomposition given by the power spectrum of that process. In other words, first and second order statistical moments (i.e., due to autocorrelation) of the signal are encoded in its power spectrum, which is purely dependent on the amplitude spectrum. Consequently, the randomization of the phase does not impact the first and second order moments of the series, hence the surrogates share statistical properties of the original signal.

Since the original time-series (i.e., asset returns) are real-valued signals, their Fourier Transform after randomization of the phase should preserve *conjugate symmetry*, or equivalently, the randomly generated phase component should be an odd function of frequency. Then the Inverse Fourier Transform (IFT) returns real-valued surrogates.

Algorithm 7: Amplitude Adjusted Fourier Transform (AAFT).

inputs : M -variate original time-series $\vec{X}$
output: M -variate synthetic time-series $\hat{\vec{X}}$

```

1 for  $i = 1, 2, \dots, M$  do
2   calculate Fourier Transform of
3     univariate series  $\mathfrak{F}[\vec{X}_{:i}]$ 
4   randomize phase component      // preserve odd symmetry of
     phase
5   calculate Inverse Fourier Transform of
6     unchanged amplitude and randomized phase  $\hat{\vec{X}}_{:i}$ 
7 end
```

Remark 8.2 *Importantly, the AAFT algorithm works on univariate series, therefore the first two statistical moments of the single asset are preserved but the cross-*

asset dependencies (i.e., cross-correlation, covariance) are modified due to the data augmentation.

8.2.2 Results

Operating on the same 12-assets universe used in experiments of Chapter 6, examples of AAFT surrogates are given in Figure 8.5, along with the cumulative returns of each trading agent on this simulated universe. As expected, the model-free agents outperform the model-based agents, corroborating the results obtained in Chapter 6.

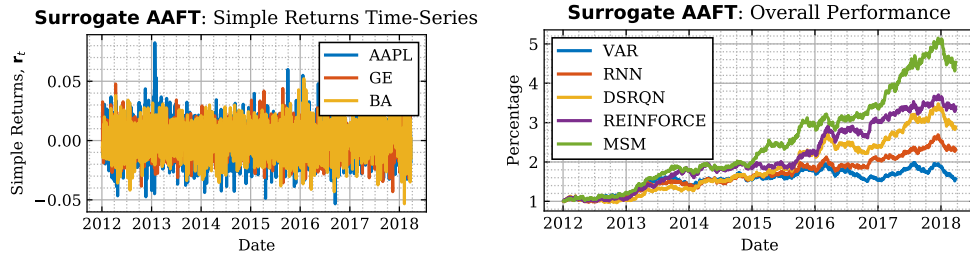


Figure 8.5: Synthetic, simulated universe of 12-assets from S&P500 via Amplitude Adjusted Fourier Transform (AAFT). (Left) Example series from universe. (Right) Cumulative returns of reinforcement learning trading agents.

Market Data

Having verified the applicability of the trading agents in synthetic environments (i.e., deterministic and stochastic) in Section 8, their effectiveness is challenged in real financial markets, namely the underlying stocks of the Standard & Poor's 500 (Investopedia, 2018f) and the EURO STOXX 50 (Investopedia, 2018a) indices. In detail, in this chapter:

- Candidate reward generating functions are explored, in Section 9.1;
- Paper trading experiments are carried out on U.S. and European most liquid assets (see Sufficient Liquidity Assumption 5.1), as in Sections 9.2 and 9.3, respectively;
- Comparison matrices and insights into the learned agent strategies are obtained.

9.1 Reward Generating Functions

Reinforcement learning relies fundamentally on the hypothesis that the goal of the agent can be fully described by the maximization of the cumulative reward over time, as suggested by the Reward Hypothesis 4.1. Consequently, the selection of the reward generating function can significantly affect the learned strategies and hence the performance of the agents. Motivated by the returns-risk trade-off arising in investments (see Chapter 3), two reward functions are implemented and tested: the log returns and the Differential Sharpe Ratio (Moody *et al.*, 1998).

9.1.1 Log Rewards

The agent at time step t observes asset prices $\mathbf{o}_t \equiv \mathbf{p}_t$ and computes the log returns, given by:

$$\boldsymbol{\rho}_t = \log(\mathbf{p}_t \oslash \mathbf{p}_{t-1}) \quad (2.12)$$

where $\oslash$ designates element-wise division and the $\log$ function is also applied element-wise, or equivalently:

$$\boldsymbol{\rho}_t \triangleq \begin{bmatrix} \rho_{1,t} \\ \rho_{2,t} \\ \vdots \\ \rho_{M,t} \end{bmatrix} = \begin{bmatrix} \log(\frac{p_{1,t}}{p_{1,t-1}}) \\ \log(\frac{p_{2,t}}{p_{2,t-1}}) \\ \vdots \\ \log(\frac{p_{M,t}}{p_{M,t-1}}) \end{bmatrix} \in \mathbb{R}^M \quad (9.1)$$

Using one-step log returns for reward, results in the multi-step maximization of **cumulative log returns**, which focuses only on the profit, without considering any risk (i.e., variance) metric. Therefore, agents are expected to be highly volatile when trained with this reward function.

9.1.2 Differential Sharpe Ratio

In Section 2.3, the Sharpe Ratio (Sharpe and Sharpe, 1970) was introduced, motivated by Signal-to-Noise Ratio (SNR), given by:

$$\mathbf{SR}_t \triangleq \sqrt{t} \frac{\mathbb{E}[\mathbf{r}_t]}{\sqrt{\text{Var}[\mathbf{r}_t]}} \in \mathbb{R} \quad (2.35)$$

where T is the number of samples considered in the calculation of the empirical mean and standard deviation. Therefore, empirical estimates of the mean and the variance of the portfolio are used in the calculation, making Sharpe Ratio an inappropriate metric for online (i.e., adaptive) episodic learning. Nonetheless, the **Differential Sharpe Ratio** (DSR), introduced by Moody *et al.* (1998), is a suitable reward function. DSR is obtained by:

1. Considering exponential moving averages of the returns and standard deviation of returns in 2.35;
2. Expanding to first order in the decay rate:

$$\mathbf{SR}_t \approx \mathbf{SR}_{t-1} + \eta \left. \frac{\partial \mathbf{SR}_t}{\partial \eta} \right|_{\eta=0} + O(\eta^2) \quad (9.2)$$

Noting that only the first order term in expansion (9.2) depends upon the return, r_t , at time step, t , the differential Sharpe Ratio, $\mathbf{D}_t$, is defined as:

$$\mathbf{D}_t \triangleq \frac{\partial \mathbf{SR}_t}{\partial \eta} = \frac{B_{t-1}\Delta A_t + \frac{1}{2}A_{t-1}\Delta B_t}{(B_{t-1} - A_{t-1}^2)^{\frac{3}{2}}} \quad (9.3)$$

where A_t and B_t are exponential moving estimates of the first and second moments of r_t , respectively, given by:

$$A_t = A_{t-1} + \eta\Delta A_t = A_{t-1} + \eta(r_t - A_{t-1}) \quad (9.4)$$

$$B_t = B_{t-1} + \eta\Delta B_t = B_{t-1} + \eta(r_t^2 - B_{t-1}) \quad (9.5)$$

Using differential Sharpe Ratio for reward, results in the multi-step maximization of **Sharpe Ratio**, which balances risk and profit, and hence it is expected to lead to better strategies, compared to log returns.

9.2 Standard & Poor's 500

9.2.1 Market Value

Publicly traded companies are usually also compared in terms of their **Market Value** or **Market Capitalization** (Market Cap), given by multiplying the number of their outstanding shares by the current share price (Investopedia, 2018d), or equivalently:

$$\mathbf{Market\ Cap}_{\text{asset } i} = \mathbf{Volume}_{\text{asset } i} \times \mathbf{Share\ Price}_{\text{asset } i} \quad (9.6)$$

The Standard & Poor's 500 Index (S&P 500) is a market capitalization weighted index of the 500 largest U.S. publicly traded companies by market value (Investopedia, 2018d). According to the Capital Asset Pricing Model (CAPM) (Luenberger, 1997) and the Efficient Market Hypothesis (EMH) (Fama, 1970), the market index, S&P 500, is efficient and portfolio derived by its constituent assets *cannot* perform better (as in the context of Section 3.1.2). Nonetheless, CAPM and EMH are not exactly satisfied and trading opportunities can be exploited via proper strategies.

9.2.2 Evaluation

In order to compare the different trading agents introduced in Chapter 6, as well as variants in Chapter 7 and Section 8.2, all agents are trained on the constituents of S&P 500 (i.e., 500 U.S. assets) and the results of their performance are provided in Figure 9.1 and Table 9.1. As expected, the differential Sharpe Ratio (DSR) is more stable than log returns, yielding higher Sharpe Ratio strategies, up to 2.77 for the pre-trained and experience transferred Mixture of Score Machines (MSM) agent.

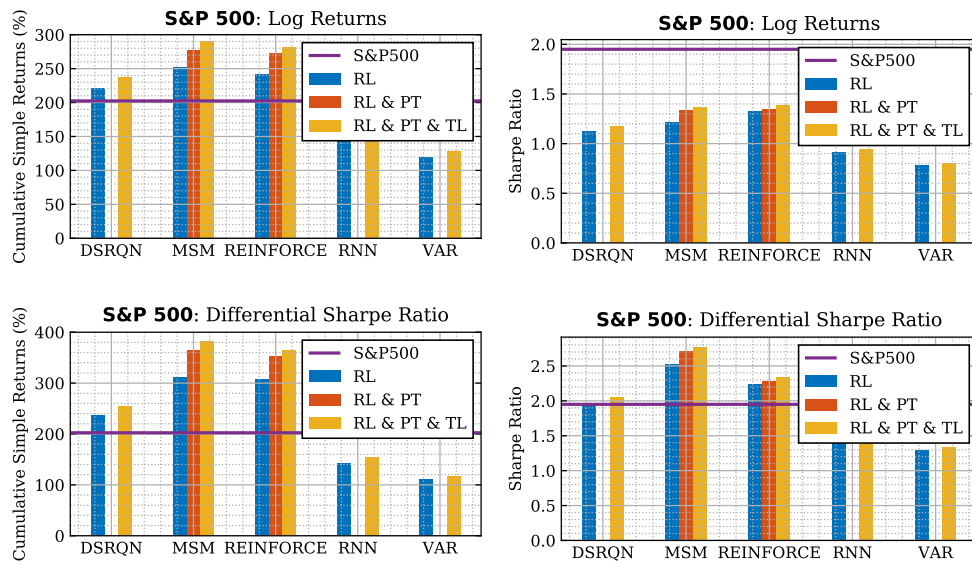


Figure 9.1: Comparison of reinforcement learning trading agents on cumulative returns and Sharpe Ratio, trained with: (RL) Reinforcement Learning; (RL & PT) Reinforcement Learning and Pre-Training; (RL & PT & TL) Reinforcement Learning, Pre-Training and Transfer Learning from simulated data.

Remark 9.1 The simulations confirm the superiority of the universal model-free reinforcement learning agents, Mixture of Score Machines (MSM), in asset allocation, with the achieved performance gain of as much as 9.2% in cumulative returns and 13.4% in Sharpe Ratio, compared to the most recent models in (Jiang et al., 2017) in the same universe.

9.3 EURO STOXX 50

Similar to S&P 500, the EURO STOXX 50 (SX5E) is a benchmark for the 50 largest publicly traded companies by market value in countries of Eurozone.

Trading Agents Comparison Matrix: S&P 500				
Reward Generating Function	Differential Sharpe Ratio		Log Returns	
	Cumulative Returns (%)	Sharpe Ratio	Cumulative Returns (%)	Sharpe Ratio
SPY	202.4	1.95	202.4	1.95
VAR	110.7	1.30	119.3	0.78
RNN	142.3	1.49	146.2	0.91
DSRQN	237.1	1.96	221.5	1.12
REINFORCE	307.5	2.24	241.3	1.32
MSM	310.8	2.53	251.9	1.21
REINFORCE & PT	353.6	2.29	272.7	1.33
MSM & PT	363.6	2.72	277.1	1.34
REINFORCE & PT & TL	364.2	2.33	280.9	1.38
MSM & PT & TL	381.7	2.77	291.0	1.36

Table 9.1: Comprehensive comparison of evaluation metrics of reinforcement learning trading algorithms and their variants, namely pre-training (PL) and transfer learning (TL).

In this section, the universality of the Mixture of Score Machines (MSM) agent is assessed against the Markowitz model (see Section 3.1).

9.3.1 Sequential Markowitz Model

A universal baseline agent is developed, based on the Sharpe Ratio with transaction costs (see optimization problem 3.14) extension of the Markowitz model, from Section 3.3. Therefore, a **Sequential Markowitz Model** (SMM) agent is derived by iteratively applying the one-step optimization program solver for each time step t . The Markowitz model is obviously a universal portfolio optimizer, since it does not make assumptions about the universe (i.e., underlying assets) it is applied upon.

9.3.2 Results

Given the EURO STOXX 50 market, *transfer learning* is performed for the *MSM agent trained on the S&P 500* (i.e., only the Mixture network is replaced and trained, while the parameters of the Score Machine networks are frozen). Figure 9.2 illustrates the cumulative returns of the market index (SX5E), the

Sequential Markowitz Model (SMM) agent and the Mixture of Score Machines (MSM) agent.

Remark 9.2 *As desired, the MSM agent outperformed both the market index (SX5E) and the SMM agent, reflecting the universality of the MSM learned strategies, which are both successful in the S&P 500 and EURO STOXX 50 markets.*

It is worth also noting that the cumulative returns of the MSM and the SMM agents were correlated, however, the MSM performed better, especially after 2009, when the SMM followed the declining market and the MSM became profitable. This fact can be attributed to the pre-training stage of the MSM agent, since during this stage, the policy gradient network converges to the Markowitz model, or effectively mimics the SMM strategies. Then, the reinforcement episodic training allows the MSM to improve itself so that it outperforms its initial strategy, the SMM.

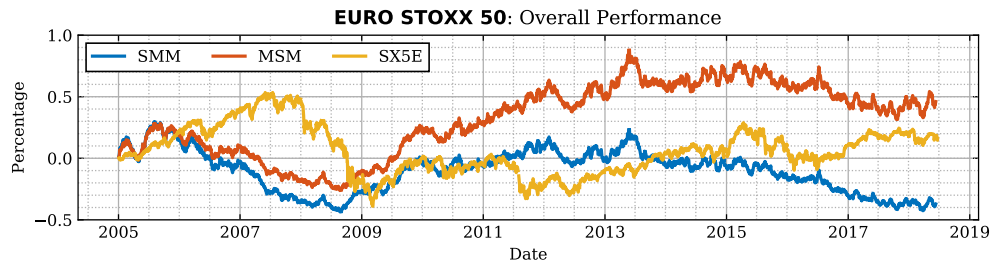


Figure 9.2: Cumulative Returns of Mixture of Score Machines (MSM) agent, trained on S&P 500 market and transferred experience to EURO STOXX 50 market (SX5E), along with the traditional Sequential Markowitz Model (SMM).

Conclusion

The main objective of this report was to investigate the effectiveness of Reinforcement Learning agents on Sequential Portfolio Management. To achieve this, many concepts from the fields of Signal Processing, Control Theory, Machine Intelligence and Finance have been explored, extended and combined. In this chapter, the contributions and achievements of the project are summarized, along with possible axis for future research.

10.1 Contributions

To enable episodic reinforcement learning, a mathematical formulation of financial markets as discrete-time stochastic dynamical systems is provided, giving rise to a unified, versatile framework for training agents and investment strategies.

A comprehensive account of reinforcement agents has been developed, including traditional, baseline agents from system identification (i.e., model-based methods) as well as context agnostic agents (i.e., model-free methods). A universal model-free reinforcement learning family of agents has been introduced, which was able to reduce the model computational and memory complexity (i.e., linear scaling with universe size) and to generalize strategies across assets and markets, regardless of the training universe. It also outperformed all trading agents, found in the open literature, in the S&P 500 and the EURO STOXX 50 markets.

Lastly, model pre-training, data augmentation and simulations enabled robust training of deep neural network architectures, even with a limited number of available real market data.

10.2 Future Work

Despite the performance gain of the developed strategies, the lack of interpretability (Rico-Martinez *et al.*, 1994) and the inability to exhaustively test the deep architectures used (i.e., Deep Neural Networks) discourage practitioner from adopting these solutions. As a consequence, it is worth investigating and interpreting the learned strategies by opening the deep “black box” and being able to reason for its decisions.

In addition, exploiting the large number of degrees of freedom given by the framework formulation of financial markets, further research on reward generating functions and state representation could improve the convergence properties and overall performance of the agents. A valid approach would be to construct the indicators typically used in the technical analysis of financial instruments (King and Levine, 1992). These measures would embed the expert knowledge acquired by financial analysts over decades of activity and could help in guiding the agent towards better decisions (Wilmott, 2007).

Furthermore, the flexible architecture of Mixtures of Score Machines (MSM) agent, the best-scoring universal trading agents, allows experimentation with both the Score Machines (SM) networks and their model-order, as well as the Mixture network, which, ideally, should be universally used without transfer learning.

The trading agents in this report are based on point estimates, provided by a deep neural network. However, due to the uncertainty of the financial signals, it would be appropriate to also model this uncertainty, incorporating it in the decision making process. Bayesian Inference, or more tractable variants of it, including Variation Inference (Titsias and Lawrence, 2010), can be used to train probabilistic models, capable of capturing the environment uncertainty (Vlassis *et al.*, 2012).

Last but not least, motivated by the recent¹ publication by (Fellows *et al.*, 2018) on exact calculation of the policy gradient by operating on the Fourier domain, employing an exact policy gradient method could eliminate the estimate variance and accelerate training.

¹At the time that report is submitted this paper has not been presented, but it is accepted in International Conference on Machine Learning (ICML) 2018.

Bibliography

- Markowitz, Harry (1952). "Portfolio selection". *The Journal of Finance* 7.1, pp. 77–91.
- Bellman, Richard (1957). *Dynamic programming*. Courier Corporation.
- Fama, Eugene F (1970). "Efficient capital markets: A review of theory and empirical work". *The Journal of Finance* 25.2, pp. 383–417.
- Sharpe, William F and WF Sharpe (1970). *Portfolio theory and capital markets*. Vol. 217. McGraw-Hill New York.
- Moylan, P and B Anderson (1973). "Nonlinear regulator theory and an inverse optimal control problem". *IEEE Transactions on Automatic Control* 18.5, pp. 460–465.
- Witten, Ian H (1977). "An adaptive optimal controller for discrete-time Markov environments". *Information and Control* 34.4, pp. 286–295.
- Gill, Philip E, Walter Murray, and Margaret H Wright (1981). "Practical optimization".
- Hopfield, John J (1982). "Neural networks and physical systems with emergent collective computational abilities". *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.
- Kroll, Yoram, Haim Levy, and Harry M Markowitz (1984). "Mean-variance versus direct utility maximization". *The Journal of Finance* 39.1, pp. 47–61.
- McCullagh, Peter (1984). "Generalized linear models". *European Journal of Operational Research* 16.3, pp. 285–292.
- Bollerslev, Tim (1986). "Generalized autoregressive conditional heteroskedasticity". *Journal of Econometrics* 31.3, pp. 307–327.
- Cybenko, George (1989). "Approximation by superpositions of a sigmoidal function". *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.
- Watkins, Christopher John Cornish Hellaby (1989). "Learning from delayed rewards". PhD thesis. King's College, Cambridge.

- Werbos, Paul J (1990). "Backpropagation through time: what it does and how to do it". *Proceedings of the IEEE* 78.10, pp. 1550–1560.
- Jacobs, Robert A *et al.* (1991). "Adaptive mixtures of local experts". *Neural Computation* 3.1, pp. 79–87.
- King, Robert Graham and Ross Levine (1992). *Financial indicators and growth in a cross section of countries*. Vol. 819. World Bank Publications.
- Prichard, Dean and James Theiler (1994). "Generating surrogate data for time series with several simultaneously measured variables". *Physical Review Letters* 73.7, p. 951.
- Rico-Martinez, R, JS Anderson, and IG Kevrekidis (1994). "Continuous-time nonlinear signal processing: a neural network based approach for gray box identification". *Neural Networks for Signal Processing [1994] IV. Proceedings of the 1994 IEEE Workshop*. IEEE, pp. 596–605.
- Bertsekas, Dimitri P *et al.* (1995). *Dynamic programming and optimal control*. Vol. 1. 2. Athena scientific Belmont, MA.
- LeCun, Yann, Yoshua Bengio, *et al.* (1995). "Convolutional networks for images, speech, and time series". *The Handbook of Brain Theory and Neural Networks* 3361.10, p. 1995.
- Tesauro, Gerald (1995). "Temporal difference learning and TD-Gammon". *Communications of the ACM* 38.3, pp. 58–68.
- Neuneier, Ralph (1996). "Optimal asset allocation using adaptive dynamic programming". *Advances in Neural Information Processing Systems*, pp. 952–958.
- Salinas, Emilio and LF Abbott (1996). "A model of multiplicative neural responses in parietal cortex". *Proceedings of the National Academy of Sciences* 93.21, pp. 11956–11961.
- Atkeson, Christopher G and Juan Carlos Santamaria (1997). "A comparison of direct and model-based reinforcement learning". *Robotics and Automation, 1997. Proceedings., 1997 IEEE International Conference on*. Vol. 4. IEEE, pp. 3557–3564.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long short-term memory". *Neural Computation* 9.8, pp. 1735–1780.
- Luenberger, David G *et al.* (1997). "Investment science". *OUP Catalogue*.
- Ortiz-Fuentes, Jorge D and Mikel L Forcada (1997). "A comparison between recurrent neural network architectures for digital equalization". *Acoustics, speech, and signal processing, 1997. ICASSP-97., 1997 IEEE International Conference on*. Vol. 4. IEEE, pp. 3281–3284.
- Rust, John (1997). "Using randomization to break the curse of dimensionality". *Econometrica: Journal of the Econometric Society*, pp. 487–516.

- Akaike, Hirotugu (1998). "Markovian representation of stochastic processes and its application to the analysis of autoregressive moving average processes". *Selected Papers of Hirotugu Akaike*. Springer, pp. 223–247.
- Cohen, Leon (1998). "The generalization of the wiener-khinchin theorem". *Acoustics, Speech and Signal Processing, 1998. Proceedings of the 1998 IEEE International Conference on*. Vol. 3. IEEE, pp. 1577–1580.
- Hochreiter, Sepp (1998). "The vanishing gradient problem during learning recurrent neural nets and problem solutions". *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 6.02, pp. 107–116.
- Moody, John *et al.* (1998). "Reinforcement learning for trading systems and portfolios: Immediate vs future rewards". *Decision Technologies for Computational Finance*. Springer, pp. 129–140.
- Papadimitriou, Christos H and Kenneth Steiglitz (1998). *Combinatorial optimization: Algorithms and complexity*. Courier Corporation.
- Sutton, Richard S and Andrew G Barto (1998). *Introduction to reinforcement learning*. Vol. 135. MIT press Cambridge.
- Gers, Felix A, Jürgen Schmidhuber, and Fred Cummins (1999). "Learning to forget: Continual prediction with LSTM".
- Ng, Andrew Y, Stuart J Russell, *et al.* (2000). "Algorithms for inverse reinforcement learning." *Icml*, pp. 663–670.
- Sutton, Richard S *et al.* (2000a). "Policy gradient methods for reinforcement learning with function approximation". *Advances in neural information processing systems*, pp. 1057–1063.
- (2000b). "Policy gradient methods for reinforcement learning with function approximation". *Advances in neural information processing systems*, pp. 1057–1063.
- Ghahramani, Zoubin (2001). "An introduction to hidden Markov models and Bayesian networks". *International Journal of Pattern Recognition and Artificial Intelligence* 15.01, pp. 9–42.
- LeBaron, Blake *et al.* (2001). "A builder's guide to agent-based financial markets". *Quantitative Finance* 1.2, pp. 254–261.
- Mandic, Danilo P, Jonathon A Chambers, *et al.* (2001). *Recurrent neural networks for prediction: Learning algorithms, architectures and stability*. Wiley Online Library.
- Tino, Peter, Christian Schittenkopf, and Georg Dorffner (2001). "Financial volatility trading using recurrent neural networks". *IEEE Transactions on Neural Networks* 12.4, pp. 865–874.
- Campbell, Murray, A Joseph Hoane Jr, and Feng-hsiung Hsu (2002). "Deep blue". *Artificial Intelligence* 134.1-2, pp. 57–83.

- Smart, William D and L Pack Kaelbling (2002). "Effective reinforcement learning for mobile robots". *Robotics and Automation, 2002. Proceedings. ICRA'02. IEEE International Conference on*. Vol. 4. IEEE, pp. 3404–3410.
- Bengio, Yoshua *et al.* (2003). "A neural probabilistic language model". *Journal of Machine Learning Research* 3.Feb, pp. 1137–1155.
- Greene, William H (2003). *Econometric analysis*. Pearson Education India.
- Boyd, Stephen and Lieven Vandenberghe (2004). *Convex optimization*. Cambridge university press.
- Kohl, Nate and Peter Stone (2004). "Policy gradient reinforcement learning for fast quadrupedal locomotion". *Robotics and Automation, 2004. Proceedings. ICRA'04. 2004 IEEE International Conference on*. Vol. 3. IEEE, pp. 2619–2624.
- Mandic, Danilo P (2004). "A generalized normalized gradient descent algorithm". *IEEE Signal Processing Letters* 11.2, pp. 115–118.
- Rasmussen, Carl Edward (2004). "Gaussian processes in machine learning". *Advanced lectures on machine learning*. Springer, pp. 63–71.
- Tsay, Ruey S (2005). *Analysis of financial time series*. Vol. 543. John Wiley & Sons.
- Gatev, Evan, William N Goetzmann, and K Geert Rouwenhorst (2006). "Pairs trading: Performance of a relative-value arbitrage rule". *The Review of Financial Studies* 19.3, pp. 797–827.
- Nasrabadi, Nasser M (2007). "Pattern recognition and machine learning". *Journal of Electronic Imaging* 16.4, p. 049901.
- Wilmott, Paul (2007). *Paul Wilmott introduces quantitative finance*. John Wiley & Sons.
- Kober, Jens and Jan R Peters (2009). "Policy search for motor primitives in robotics". *Advances in neural information processing systems*, pp. 849–856.
- Meucci, Attilio (2009). *Risk and asset allocation*. Springer Science & Business Media.
- Raeth, Christoph and R Monetti (2009). "Surrogates with random Fourier phases". *Topics On Chaotic Systems: Selected Papers from CHAOS 2008 International Conference*. World Scientific, pp. 274–285.
- Ahmed, Nesreen K *et al.* (2010). "An empirical comparison of machine learning models for time series forecasting". *Econometric Reviews* 29.5-6, pp. 594–621.
- Nair, Vinod and Geoffrey E Hinton (2010). "Rectified linear units improve restricted boltzmann machines". *Proceedings of the 27th international conference on machine learning (ICML-10)*, pp. 807–814.
- Pan, Sinno Jialin and Qiang Yang (2010a). "A survey on transfer learning". *IEEE Transactions on Knowledge and Data Engineering* 22.10, pp. 1345–1359.

- Pan, Sinno Jialin and Qiang Yang (2010b). "A survey on transfer learning". *IEEE Transactions on Knowledge and Data Engineering* 22.10, pp. 1345–1359.
- Poole, David L and Alan K Mackworth (2010). *Artificial intelligence: Foundations of computational agents*. Cambridge University Press.
- Szepesvári, Csaba (2010). "Algorithms for reinforcement learning". *Synthesis Lectures on Artificial Intelligence and Machine Learning* 4.1, pp. 1–103.
- Titsias, Michalis and Neil D Lawrence (2010). "Bayesian Gaussian process latent variable model". *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pp. 844–851.
- Deisenroth, Marc and Carl E Rasmussen (2011). "PILCO: A model-based and data-efficient approach to policy search". *Proceedings of the 28th International Conference on machine learning (ICML-11)*, pp. 465–472.
- Hénaff, Patrick *et al.* (2011). "Real time implementation of CTRNN and BPTT algorithm to learn on-line biped robot balance: Experiments on the standing posture". *Control Engineering Practice* 19.1, pp. 89–99.
- Johnston, Douglas E and Petar M Djurić (2011). "The science behind risk management". *IEEE Signal Processing Magazine* 28.5, pp. 26–36.
- Farmer, Roger EA (2012). "The stock market crash of 2008 caused the Great Recession: Theory and evidence". *Journal of Economic Dynamics and Control* 36.5, pp. 693–707.
- Murphy, Kevin P. (2012). *Machine Learning: A Probabilistic Perspective*. The MIT Press. ISBN: 0262018020, 9780262018029.
- Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio (2012). "Understanding the exploding gradient problem". *CoRR*, [abs/1211.5063](https://arxiv.org/abs/1211.5063).
- Tieleman, Tijmen and Geoffrey Hinton (2012). "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude". *COURSERA: Neural Networks for Machine Learning* 4.2, pp. 26–31.
- Vlassis, Nikos *et al.* (2012). "Bayesian reinforcement learning". *Reinforcement Learning*. Springer, pp. 359–386.
- Aldridge, Irene (2013). *High-frequency trading: A practical guide to algorithmic strategies and trading systems*. Vol. 604. John Wiley & Sons.
- Giusti, Alessandro *et al.* (2013). "Fast image scanning with deep max-pooling convolutional neural networks". *Image Processing (ICIP), 2013 20th IEEE International Conference on*. IEEE, pp. 4034–4038.
- Michalski, Ryszard S, Jaime G Carbonell, and Tom M Mitchell (2013). *Machine learning: An artificial intelligence approach*. Springer Science & Business Media.
- Roberts, Stephen *et al.* (2013). "Gaussian processes for time-series modelling". *Phil. Trans. R. Soc. A* 371.1984, p. 20110550.

- Xia, Yili and Danilo P Mandic (2013). "Augmented MVDR spectrum-based frequency estimation for unbalanced power systems". *IEEE Transactions on Instrumentation and Measurement* 62.7, pp. 1917–1926.
- Kingma, Diederik P and Jimmy Ba (2014). "Adam: A method for stochastic optimization". *arXiv preprint arXiv:1412.6980*.
- Almeida, Náthalee C, Marcelo AC Fernandes, and Adrião DD Neto (2015). "Beamforming and power control in sensor arrays using reinforcement learning". *Sensors* 15.3, pp. 6668–6687.
- Archer, Evan *et al.* (2015). "Black box variational inference for state space models". *arXiv preprint arXiv:1511.07367*.
- Chen, Kai, Yi Zhou, and Fangyan Dai (2015). "A LSTM-based method for stock returns prediction: A case study of China stock market". *Big Data (Big Data)*, 2015 IEEE International Conference on. IEEE, pp. 2823–2824.
- Hausknecht, Matthew and Peter Stone (2015). "Deep recurrent q-learning for partially observable mdps". *CoRR*, abs/1507.06527.
- Hills, Thomas T *et al.* (2015). "Exploration versus exploitation in space, mind, and society". *Trends in Cognitive Sciences* 19.1, pp. 46–54.
- Lillicrap, Timothy P *et al.* (2015). "Continuous control with deep reinforcement learning". *arXiv preprint arXiv:1509.02971*.
- Mnih, Volodymyr *et al.* (2015). "Human-level control through deep reinforcement learning". *Nature* 518.7540, p. 529.
- Silver, David (2015a). *Integrating learning and planning*. URL: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching_files/dyna.pdf.
- (2015b). *Introduction to reinforcement learning*. URL: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching_files/intro_RL.pdf.
- (2015c). *Markov decision processes*. URL: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching_files/MDP.pdf.
- (2015d). *Model-free control*. URL: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching_files/control.pdf.
- (2015e). *Policy gradient*. URL: http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching_files/pg.pdf.
- Feng, Yiyong, Daniel P Palomar, *et al.* (2016). "A signal processing perspective on financial engineering". *Foundations and Trends® in Signal Processing* 9.1–2, pp. 1–231.
- Gal, Yarin (2016). "Uncertainty in deep learning". *University of Cambridge*.
- Gal, Yarin and Zoubin Ghahramani (2016). "A theoretically grounded application of dropout in recurrent neural networks". *Advances in neural information processing systems*, pp. 1019–1027.

- Gal, Yarin, Rowan McAllister, and Carl Edward Rasmussen (2016). "Improving PILCO with Bayesian neural network dynamics models". *Data-Efficient Machine Learning workshop, ICML*.
- Goodfellow, Ian *et al.* (2016). *Deep learning*. Vol. 1. MIT press Cambridge.
- Heaton, JB, NG Polson, and Jan Hendrik Witte (2016). "Deep learning in finance". *arXiv preprint arXiv:1602.06561*.
- Kennedy, Douglas (2016). *Stochastic financial models*. Chapman and Hall/CRC.
- Levine, Sergey *et al.* (2016). "End-to-end training of deep visuomotor policies". *The Journal of Machine Learning Research* 17.1, pp. 1334–1373.
- Liang, Yitao *et al.* (2016). "State of the art control of atari games using shallow reinforcement learning". *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, pp. 485–493.
- Mnih, Volodymyr *et al.* (2016). "Asynchronous methods for deep reinforcement learning". *International Conference on Machine Learning*, pp. 1928–1937.
- Necchi, Pierpaolo (2016). *Policy gradient algorithms for asset allocation problem*. URL: https://github.com/pnecchi/Thesis/blob/master/MS_Thesis_Pierpaolo_Necchi.pdf.
- Nemati, Shamim, Mohammad M Ghassemi, and Gari D Clifford (2016). "Optimal medication dosing from suboptimal clinical examples: A deep reinforcement learning approach". *Engineering in Medicine and Biology Society (EMBC), 2016 IEEE 38th Annual International Conference of the. IEEE*, pp. 2978–2981.
- Silver, David and Demis Hassabis (2016). "AlphaGo: Mastering the ancient game of Go with Machine Learning". *Research Blog*.
- Bao, Wei, Jun Yue, and Yulei Rao (2017). "A deep learning framework for financial time series using stacked autoencoders and long-short term memory". *PloS One* 12.7, e0180944.
- Deng, Yue *et al.* (2017). "Deep direct reinforcement learning for financial signal representation and trading". *IEEE Transactions on Neural Networks and Learning Systems* 28.3, pp. 653–664.
- Heaton, JB, NG Polson, and Jan Hendrik Witte (2017). "Deep learning for finance: Deep portfolios". *Applied Stochastic Models in Business and Industry* 33.1, pp. 3–12.
- Jiang, Zhengyao, Dixing Xu, and Jinjun Liang (2017). "A deep reinforcement learning framework for the financial portfolio management problem". *arXiv preprint arXiv:1706.10059*.
- Navon, Ariel and Yosi Keller (2017). "Financial time series prediction using deep learning". *arXiv preprint arXiv:1711.04174*.

- Noonan, Laura (2017). *JPMorgan develops robot to execute trades*. URL: <https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c>.
- Quantopian (2017). *Commission models*. URL: <https://www.quantopian.com/help>.
- Schinckus, Christophe (2017). "An essay on financial information in the era of computerization". *Journal of Information Technology*, pp. 1–10.
- Zhang, Xiao-Ping Steven and Fang Wang (2017). "Signal processing for finance, economics, and marketing: Concepts, framework, and big data applications". *IEEE Signal Processing Magazine* 34.3, pp. 14–35.
- Fellows, Matthew, Kamil Ciosek, and Shimon Whiteson (2018). "Fourier Policy Gradients". *CoRR*.
- Investopedia (2018a). *Euro STOXX 50 index*. URL: <https://www.investopedia.com/terms/d/dowjoneseurostoxx50.asp>.
- (2018b). *Law of supply and demand*. URL: <https://www.investopedia.com/terms/l/law-of-supply-demand.asp>.
- (2018c). *Liquidity*. URL: <https://www.investopedia.com/terms/l/liquidity.asp>.
- (2018d). *Market value*. URL: <https://www.investopedia.com/terms/m/marketvalue.asp>.
- (2018e). *Slippage*. URL: <https://www.investopedia.com/terms/s/slippage.asp>.
- (2018f). *Standard & Poor's 500 index - S&P 500*. URL: <https://www.investopedia.com/terms/s/sp500.asp>.
- (2018g). *Transaction costs*. URL: <https://www.investopedia.com/terms/t/transactioncosts.asp>.
- Mandic, Danilo P. (2018a). *Advanced signal processing: Linear stochastic processes*. URL: http://www.commsp.ee.ic.ac.uk/~mandic/ASP_Slides/ASP_Lecture_2_ARMA_Modelling_2018.pdf.
- (2018b). *Introduction to estimation theory*. URL: http://www.commsp.ee.ic.ac.uk/~mandic/ASP_Slides/AASP_Lecture_3_Estimation_Intro_2018.pdf.
- Spooner, Thomas *et al.* (2018). "Market Making via Reinforcement Learning". *arXiv preprint arXiv:1804.04216*.
- Wikipedia (2018a). *London Stock Exchange*. URL: https://en.wikipedia.org/wiki/London_Stock_Exchange.
- (2018b). *NASDAQ*. URL: <https://en.wikipedia.org/wiki/NASDAQ>.
- (2018c). *New York Stock Exchange*. URL: https://en.wikipedia.org/wiki/New_York_Stock_Exchange.
- Graves, Alex. "Supervised sequence labelling with recurrent neural networks. 2012". ISBN 9783642212703. URL <http://books.google.com/books>.